

Technische Universität Chemnitz

Exploring Reinforcement learning strategies in automated wheel truing

Master's Thesis

Toni Rozsahegyi

First Supervisor: M.Sc. Antje Ahrens

Second Supervisor: M.Sc. Oliver Maith

last updated on: January 22, 2026

Abstract

Spoked bicycle wheel truing, the process of correcting rim deviations by adjusting spoke tensions, presents an interesting testbed for reinforcement learning due to its high-dimensional state space, non-local action effects, and multi-objective reward structure. These characteristics mirror challenges found in broader industrial applications such as residual stress optimization in manufacturing.

This thesis investigates whether reinforcement learning agents can learn effective wheel truing policies in simulation, with the longer-term goal of enabling sim-to-real transfer to physical truing machines.

We conduct a systematic ablation study comparing three algorithmic paradigms: value-based (Rainbow DQN), policy-gradient (PPO), and model-based (TD-MPC2) across four state-space formulations, three action-space designs, and four reward functions. Follow-up experiments investigate reward weighting, extended training dynamics, and temporal augmentation through frame stacking and recurrent architectures (LSTM, GRU). The insights from these experiments inform the selection of a final algorithm configuration, which undergoes dedicated hyperparameter optimization to produce a candidate policy for future work and eventual deployment on a physical truing test station.

TODO: KEY RESULT

TODO: KEY CONCLUSION

Contents

Abstract	i
1 Introduction	1
2 Theory	4
2.1 Foundations of Reinforcement Learning	4
2.1.1 The Markov Decision Process Framework	4
2.1.2 Evaluating Policies through Value Functions	5
2.1.3 Finding Optimal Policies	6
2.1.4 The Exploration-Exploitation Dilemma	6
2.1.5 The Curse of Dimensionality	7
2.2 Deep Reinforcement Learning	7
2.2.1 Challenges of Function Approximation	7
2.3 Three Algorithmic Paradigms	9
2.3.1 Value-Based Methods	9
2.3.2 Policy Gradient Methods	10
2.3.3 Model-Based Methods	12
3 Methods	14
3.1 Environment	14
3.1.1 Wheel Truing Physics	14
3.1.2 On State-Space Formulations	21
3.1.3 On the Starting State	23
3.1.4 On Action-Space Formulations	25
3.1.5 On Reward Functions	26
3.1.6 Episode Length	28
3.2 Algorithm Configurations	29
3.2.1 Rainbow DQN	29
3.2.2 PPO	32
3.2.3 TD-MPC2	35
3.2.4 Common Training Configuration	38

3.3	Experimental Design	39
3.3.1	Ablation Study Design	39
3.4	Follow-up Experiments	40
3.4.1	Reward Weighting Parameter Sweep	41
3.4.2	Extended Training and Temporal Augmentation	41
3.4.3	Hyperparameter Optimization	43
3.5	Methodological Limitations	45
4	Results and Analysis	47
4.1	Baseline Metrics and Reference Points	47
4.1.1	Initial State Distribution	47
4.1.2	Interpretation Framework	48
4.2	Ablation Study Results	48
4.2.1	Algorithm Comparison	48
4.2.2	Action Space Analysis	50
4.2.3	State Representation Analysis	51
4.2.4	Reward Function Analysis	53
4.3	Alpha Parameter Sweep	53
4.4	Extended Training and Temporal Augmentation	55
4.4.1	Convergence Behavior	55
4.4.2	Temporal Augmentation Results	55
4.4.3	Implications for Effective Markovianity	56
4.5	Synthesis	56
4.6	Synthesis	56
4.6.1	Explaining Algorithm Performance Differences	56
4.6.2	Explaining the State Representation Paradox	57
4.6.3	Best-Performing Configuration	57
4.7	Hyperparameter Optimization	57
5	Discussion and Conclusion	58
	Bibliography	59
	Appendix	61
	PPO Figures	65
	TD-MPC Figures	78
.1	Code Listings	87

List of Figures

1.1	Schematic illustration of the mapping from spoke tension configurations to rim displacements. Distinct tension configurations $\mathbf{t}_i$ that are far apart in tension space can map to similar displacement profiles $\mathbf{d}_i$, creating partial observability when only rim geometry is measured.	3
2.1	The policy iteration algorithm alternates between evaluating the current policy and improving it by acting greedily with respect to the computed values. This process converges to the optimal value function V^* and optimal policy π^*	6
3.1	Schematic of a typical bicycle wheel. (a) Side view, looking at the hub. (b) Rim cross-section showing local coordinate system at the centroid and vector spoke offset b_s . (c) Rim cross-section after deformation. Tangential displacement ω is not shown. Reproduced from [1].	15
3.2	MSE and calculation time depending on mode Number	21
3.3	Fourier reconstruction analysis for the wheel displacement state. Left: Combined mean squared error between original and reconstructed displacement versus number of harmonics, shown for different levels of initial wheel damage (3–12 randomly detuned spokes). Right: Reconstruction accuracy (variance explained) for radial and lateral displacement components separately, with shaded regions indicating standard deviation across 50 random initial states. The vertical dashed line marks the chosen value of $N = 30$ harmonics.	23
3.4	Distribution of initial state components across 500 sampled wheel configurations. (a) Radial displacements are approximately an order of magnitude smaller than (b) lateral displacements. (c) Spoke tensions are normalized by the initial tension $T_0 = 800$ N, centering the distribution around 1.0. (d) The Fourier magnitude spectrum shows that lower harmonics dominate the initial disturbances. Red lines indicate mean and ± 1 standard deviation.	24

4.1	Distribution of initial state norm (integrated rim displacement) across training episodes. The mean of 0.702 mm establishes the baseline against which learned policies are evaluated.	48
4.2	Distribution of initial maximum tension deviation from the nominal 800 N. The mean of 171.99 N provides context for evaluating tension uniformity.	48
4.3	Rainbow DQN ablation results. The rimpoints state representation with raw or combined reward achieves the best geometric accuracy across all tested configurations.	49
4.4	PPO with discrete action space. Performance is systematically worse than DQN but follows similar patterns across state representations and reward functions.	49
4.5	TD-MPC2 with continuous action space. Performance hovers near the initial state mean, indicating minimal effective learning within the training budget.	50
4.6	PPO with all-spokes action space. All configurations show catastrophic divergence, with final state norms exceeding initial values by factors of 10^3 – 10^4	51
4.7	TD-MPC2 with all-spokes action space. While performance is poor, the divergence is far less catastrophic than PPO.	52
4.8	Alpha parameter sweep for DQN. The rimpoints representation (orange) consistently outperforms rimandspokes (blue) across all α values.	54
4.9	Alpha parameter sweep for PPO. Performance is noisier and generally worse than DQN, but the optimal α region is similar.	54
4.10	Extended training and temporal augmentation results over 5×10^6 steps. Final state norm (left) and maximum tension delta (right) across PPO (blue) and Rainbow (red) with different temporal processing methods: baseline (darkest), frame stacking, LSTM, and GRU (lightest). Raw data points shown with 20% transparency, overlaid with 100-step moving averages. Neither temporal augmentation method improves over the feedforward baseline.	55

List of Tables

3.1	Wheel simulation parameters	22
3.2	State-space formulations investigated in this work. Code identifiers correspond to labels used in experimental plots. Statistics are computed over 500 sampled initial configurations.	23
3.3	Algorithm–action space compatibility matrix. Checkmarks indicate valid combinations tested in this work.	26
3.4	Rainbow DQN hyperparameters	31
3.5	PPO hyperparameters	34
3.6	TD-MPC2 network architectures. All hidden layers use Mish activation with layer normalization.	35
3.7	TD-MPC2 planning hyperparameters	37
3.8	TD-MPC2 training hyperparameters	38
3.9	Shared training configuration across all algorithms	38
3.10	Temporal augmentation hyperparameters.	43
4.1	Best-performing configuration for each algorithm, evaluated by final state norm (mean of last 100 training episodes). Improvement percentage is calculated relative to the initial mean of 0.702 mm.	48
4.2	Action space comparison for PPO (rimpoints state, combined reward). The discrete formulation provides dramatic advantages over continuous alternatives.	50
4.3	State representation comparison for DQN (combined reward). Adding spoke tensions to the observation consistently degrades performance despite providing strictly more information.	52
4.4	Reward function comparison (rimpoints state, discrete actions). Dense reward signals (raw, combined) substantially outperform sparse alternatives.	53

Chapter 1

Introduction

Broader Motivation Many industrial manufacturing processes produce components that retain internal stresses from thermal or mechanical treatment. These residual stresses pose significant challenges in sectors where large monolithic structures are employed, such as shipbuilding and aircraft manufacturing. If left unaddressed, internal tensions can lead to gradual deformation, reduced fatigue life, or catastrophic material failure during operation.

In shipbuilding, for instance, large hull sections and structural components that have undergone welding or thermal cutting often require stress relief before assembly. This remains a predominantly manual task, demanding considerable expertise. Skilled specialists iteratively apply localized heating to specific regions of the workpiece, allowing the material to settle into a lower-energy configuration upon cooling. The process is inherently sequential: the technician observes the current state of the structure, decides where to apply heat, and evaluates the result before proceeding to the next intervention. This cycle continues until the component reaches an acceptable stress distribution.

While the physical mechanisms differ substantially, the wheel truing problem studied in this thesis shares conceptual similarities with such stress optimization tasks. Both involve iteratively adjusting tension within a coupled system, where local interventions propagate effects throughout the structure. Both require balancing competing constraints in wheel truing, achieving geometric trueness while maintaining uniform spoke tensions; in stress relief, minimizing residual stresses without introducing new distortions. These parallels suggest that methods developed for automated wheel truing may offer insights applicable to a broader class of iterative tension optimization problems.

Wheel Truing A spoked bicycle wheel consists of a hub connected to a rim through a network of tensioned spokes. During use or as a result of manufacturing imperfections, the rim can deviate from its ideal circular and planar geometry a condition known as being "out of true." Wheel truing is the process of restoring geometric accuracy by selectively adjusting spoke tensions. Tightening or loosening individual spokes pulls the

rim in specific directions, and through careful iterative adjustments, a skilled mechanic can correct both lateral (side-to-side) and radial (up-and-down) deviations.

This task presents an interesting challenge for reinforcement learning. The state space is high-dimensional, encoding the displacement field around the rim and the tensions of all spokes. Actions are discrete choices of which spoke to adjust and by how much, yet their effects propagate non-locally through the coupled spoke-rim system. The reward signal must balance multiple objectives: minimizing rim displacement, maintaining spoke tensions within safe bounds, and achieving uniform tension distribution. These characteristics make wheel truing a compelling testbed for studying sequential decision-making in physically grounded environments.

The longer-term goal of this research program, though beyond the scope of the present thesis, is to achieve sim-to-real transfer: training an agent entirely in simulation that can subsequently be deployed on a physical wheel truing machine. Such a system would iteratively measure the wheel’s geometry, select a spoke to adjust, execute the adjustment, and repeat until the wheel meets trueness specifications. Automated wheel truing machines do exist commercially, but to our knowledge, current systems rely on inverse models calibrated to specific wheel configurations and do not generalize across different wheel geometries or spoke patterns. A reinforcement learning approach trained on diverse simulated wheels could potentially overcome this limitation, learning policies that transfer to novel wheel types without explicit re-calibration.

This horizon goal informs several design decisions throughout this work. The choice of state representations considers what measurements would be feasible on a physical truing stand. The action space formulations reflect realistic constraints on spoke adjustment granularity.

Simulation Environment To enable systematic experimentation, this work employs a physics-based simulation of the wheel truing process, described in detail in Section 3.1.1. The simulator models the mechanical coupling between spoke tensions and rim geometry, allowing agents to be trained on diverse wheel configurations without requiring access to physical hardware. This simulation-first approach is a prerequisite for eventual real-world deployment, where a policy trained in simulation would be transferred to a physical truing machine.

Hypothesis: Partial Observability Preliminary experiments with simpler function approximators revealed that wheel truing presents significant learning challenges beyond its high dimensionality. One hypothesis for this difficulty concerns the relationship between spoke tensions and rim displacements: the mapping may be many-to-one, meaning that multiple distinct spoke-tension configurations could produce nearly identical rim displacement profiles, as illustrated schematically in Figure 1.1. If this is the

case, an agent observing only the rim geometry, the natural measurement available on a physical truing stand, would face a partially observable decision problem, where the optimal action depends not just on the current rim shape but on the underlying tension distribution, which cannot be uniquely inferred from displacements alone. This hypothesized partial observability motivates the exploration of recurrent architectures and alternative state representations in later chapters, which may help the agent implicitly reconstruct relevant aspects of the hidden state from observation history.

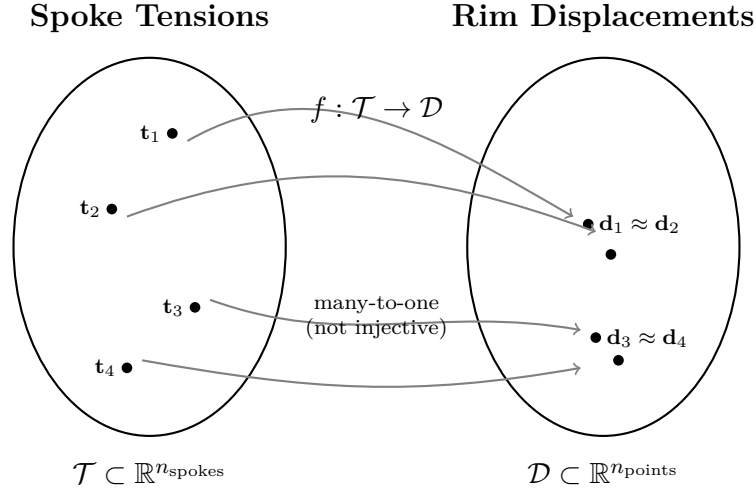


Figure 1.1: Schematic illustration of the mapping from spoke tension configurations to rim displacements. Distinct tension configurations $\mathbf{t}_i$ that are far apart in tension space can map to similar displacement profiles $\mathbf{d}_i$, creating partial observability when only rim geometry is measured.

Chapter 2

Theory

TODO:

- make more use of paragraphs instead of subsections

2.1 Foundations of Reinforcement Learning

Reinforcement learning addresses the problem of sequential decision-making under uncertainty. Unlike supervised learning, where a model learns from labeled examples, a reinforcement learning agent must discover effective behavior through interaction with its environment. The agent receives no direct instruction on which actions to take; instead, it observes the consequences of its actions through a scalar reward signal. This feedback is often delayed, a decision made now may only reveal its quality many steps later. Furthermore, the agent's actions influence not only immediate rewards but also the future states it will encounter.

2.1.1 The Markov Decision Process Framework

The standard formalization of this problem is the Markov decision process (MDP). An MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, p, r)$, where $\mathcal{S}$ denotes the set of possible states, $\mathcal{A}$ the set of available actions, $p(s' | s, a)$ the probability of transitioning to the next state s' when taking action a in state s , and $r(s, a, s')$ the reward received upon this transition. The defining characteristic of this framework is the Markov property: the distribution over next states depends only on the current state and action, not on the history of how that state was reached. This memorylessness is what makes the problem tractable and allows for mathematical rigor of the underlying optimization procedure. All information relevant to future outcomes is encoded in the current state, allowing the agent to make decisions without maintaining an explicit record of the past.

The agent's behavior is governed by a policy π , which specifies a probability distribution over actions for each state: $\pi(a | s) = P(a_t = a | s_t = s)$. A policy

can be deterministic, always selecting the same action in a given state, or stochastic, randomizing over multiple actions. The central objective of reinforcement learning is to find a policy that maximizes the cumulative reward the agent receives over time.

2.1.2 Evaluating Policies through Value Functions

To compare policies, we need a way to quantify their long-term performance. The discounted return starting from time t is defined as

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad (2.1)$$

where $\gamma \in [0, 1)$ is the discount factor. This formulation captures two important intuitions. First, rewards received sooner are generally preferable to the same rewards received later. Second, the discount factor ensures that the infinite sum remains bounded, which is necessary for the problem to be well-defined.

The state-value function $V^\pi(s)$ measures the expected return when starting in state s and subsequently following policy π :

$$V^\pi(s) = \mathbb{E}_\pi[R_t \mid s_t = s]. \quad (2.2)$$

Similarly, the action-value function $Q^\pi(s, a)$ captures the expected return when taking action a in state s and then following π :

$$Q^\pi(s, a) = \mathbb{E}_\pi[R_t \mid s_t = s, a_t = a]. \quad (2.3)$$

These two functions are intimately related: the state-value is the expectation of the action-value under the policy, $V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a)$.

A crucial property of value functions is their recursive structure, expressed by the Bellman equation:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) [r(s, a, s') + \gamma V^\pi(s')]. \quad (2.4)$$

This equation states that the value of a state equals the expected immediate reward plus the discounted value of the successor state. The recursive nature of this relationship forms the foundation for reinforcement learning algorithms.

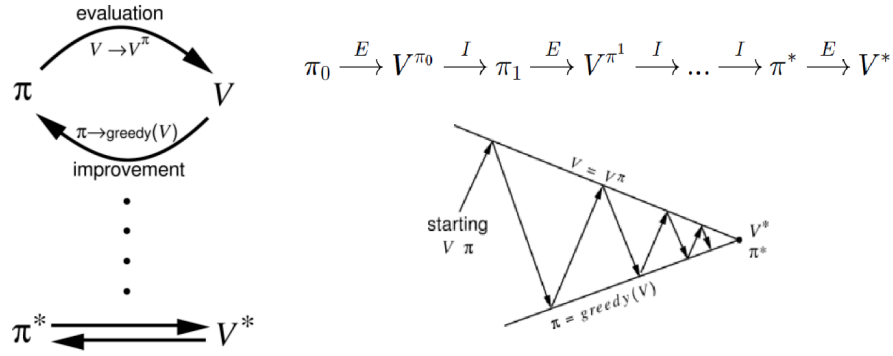


Figure 2.1: The policy iteration algorithm alternates between evaluating the current policy and improving it by acting greedily with respect to the computed values. This process converges to the optimal value function V^* and optimal policy π^* .

2.1.3 Finding Optimal Policies

Among all possible policies, at least one achieves the maximum expected return from every state. The corresponding optimal value functions are defined as

$$V^*(s) = \max_{\pi} V^{\pi}(s), \quad Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a). \quad (2.5)$$

If the optimal action-value function Q^* were known, the optimal policy would follow immediately: simply select the action with the highest Q -value in each state. The challenge, of course, is that Q^* is not given - it must be learned.

One classical approach to finding optimal policies is policy iteration, which alternates between two phases. In policy evaluation, the value function V^{π} is computed for the current policy by solving the Bellman equation. In policy improvement, a new policy is derived by acting greedily with respect to the computed values: selecting the action that maximizes expected return according to V^{π} . This improved policy is then evaluated, and the cycle repeats. Under mild conditions, this process converges to the optimal policy. Figure 2.1 illustrates this iterative refinement, showing how the value function progressively improves until reaching V^* .

2.1.4 The Exploration-Exploitation Dilemma

A fundamental tension pervades all of reinforcement learning: the conflict between exploration and exploitation. Exploitation means selecting actions that the agent currently believes to be best, based on its learned value estimates. Exploration means trying actions whose values are uncertain, potentially discovering better strategies. An agent that exploits too aggressively may settle for mediocre policies, never discovering superior alternatives. An agent that explores too freely wastes time on suboptimal actions even when good ones are known.

This dilemma has no universal solution. Different algorithms adopt different strategies, from simple approaches like ϵ -greedy action selection, which occasionally chooses random actions, to sophisticated methods that quantify and seek out uncertainty. The appropriate balance depends on the problem structure and the available learning budget. What remains constant is that effective learning requires both: pure exploitation leads to premature convergence, while pure exploration never capitalizes on acquired knowledge.

2.1.5 The Curse of Dimensionality

The ideas described so far assume that value functions can be represented exactly - for instance, as tables storing one value per state or state-action pair. This tabular approach works well for small problems with discrete states. A gridworld with a hundred cells requires only a hundred table entries; policy iteration can solve such problems efficiently.

In many real-world problems as for instance our bicycle wheel with continuous spoke tensions and rim displacements has an effectively infinite state space. The number of entries required for a tabular representation grows exponentially with the dimensionality of the state a phenomenon known as the curse of dimensionality. Exact methods become computationally infeasible long before reaching problems of practical interest.

This limitation motivates the use of function approximation: rather than storing values explicitly for each state, we approximate the value function with a parameterized model. Under the assumption that a suitable function from states to "true" values exists, this model generalizes from visited states to unvisited ones, enabling learning in vast state spaces. As universal function approximators, neural networks have proven particularly effective for this purpose, capable of representing complex, nonlinear relationships between these states and values. This combination of reinforcement learning with neural network function approximation defines the field of deep reinforcement learning.

2.2 Deep Reinforcement Learning

Replacing tables with neural networks fundamentally changes the learning problem. Function approximation introduces challenges absent from the tabular setting, and much of modern deep reinforcement learning research concerns methods for overcoming these difficulties.

2.2.1 Challenges of Function Approximation

Three interrelated problems arise when neural networks approximate value functions or policies.

The first is non-stationarity of the learning target. In supervised learning, the relationship between inputs and outputs is fixed: a cat remains a cat regardless of how well the classifier performs. In reinforcement learning, the target values depend on the current policy, which itself changes as learning progresses. When approximating Q^π , the "true" values shift as π improves. The network chases a moving target, and updates that improve accuracy for one policy may be counterproductive for the next.

The second challenge stems from correlated samples. Standard supervised learning assumes that training examples are independent and identically distributed. Experience collected by an agent violates this assumption profoundly: consecutive states in a trajectory are highly correlated, and the distribution of visited states depends on the current policy. Training on sequential batches of correlated experience can cause the network to overfit to recent states, forgetting what it learned earlier.

The third problem is bootstrapping instability. Many reinforcement learning algorithms update value estimates using other value estimates - the Bellman equation, for instance, expresses $V(s)$ in terms of $V(s')$. When the same network provides both the prediction and the target, small errors can amplify: an overestimated value propagates to predecessor states, which then propagate further, potentially leading to divergence.

These challenges are not independent. Non-stationarity exacerbates instability; correlated samples accelerate overfitting to erroneous values. Effective deep reinforcement learning algorithms must address all three.

The correlation problem admits a universal solution applicable across algorithmic paradigms: experience replay. Rather than training on transitions as they occur, the agent stores experiences in a replay buffer and samples mini-batches randomly for training. This breaks temporal correlations and allows each transition to contribute to multiple updates. Prioritized variants improve efficiency further by sampling transitions proportionally to their prediction error, focusing computation on experiences the network currently models poorly. Experience replay has become a standard component in both value-based and model-based methods, and similar buffer-based approaches appear in some policy gradient implementations.

The remaining challenges, non-stationarity and bootstrapping instability, are addressed differently by each algorithmic paradigm. Value-based methods employ target networks and double estimation to stabilize the bootstrap targets. Policy gradient methods sidestep bootstrapping entirely by estimating returns from complete trajectories, and constrain policy changes to maintain stability despite the shifting objective. Model-based methods learn an explicit dynamics model, allowing them to generate synthetic experience and plan ahead rather than relying solely on cached value estimates. The following sections examine each approach in detail.

2.3 Three Algorithmic Paradigms

Deep reinforcement learning algorithms can be broadly categorized into three families based on what they learn and how they derive behavior.

Value-based methods learn an approximation to the optimal action-value function Q^* . The policy is implicit: in each state, the agent selects the action with the highest estimated Q-value. Because this requires evaluating Q for every possible action, value-based methods are most naturally suited to discrete action spaces where actions can be enumerated.

Policy gradient methods parameterize the policy directly as $\pi_\theta(a | s)$ and optimize the expected return with respect to the policy parameters θ . Note that this θ is unrelated to the variable used in the wheel state. This approach handles continuous action spaces naturally: the network outputs parameters of a continuous distribution, such as the mean and variance of a Gaussian. The policy can be stochastic, which provides a natural mechanism for exploration.

Model-based methods learn a model of the environment’s dynamics: given the current state and action, predict the next state and reward. This model enables planning by simulating the consequences of actions before committing to them. By extracting more information from each real interaction, model-based approaches can achieve higher sample efficiency than their model-free counterparts.

The following sections examine each paradigm in greater detail, focusing on the mechanisms that address the challenges outlined above.

2.3.1 Value-Based Methods

Value-based methods seek to learn the optimal action-value function $Q^*(s, a)$ directly. Once learned, the optimal policy follows immediately: select $\arg \max_a Q^*(s, a)$ in each state. The foundational algorithm in this family is Q-learning, which updates value estimates using the Bellman optimality equation. Deep Q-Networks (DQN) extend this approach using neural networks as function approximators, introducing several mechanisms to stabilize training.

Stabilization Techniques

Target networks tackle bootstrapping instability by separating the network used for action selection from the network used to compute target values. The target network is a copy of the main network, updated only periodically—for instance, every several thousand steps. This separation prevents the feedback loop where changes to the network immediately affect its own training targets, providing a more stable learning signal.

Double Q-learning addresses a systematic overestimation bias in standard Q-learning. Because the same network both selects the best action and evaluates its value, noise in the estimates tends to favor overestimated actions. Double Q-learning decouples these roles: the main network selects the action, but the target network evaluates it. This simple modification significantly reduces overestimation and improves learning stability.

Distributional Reinforcement Learning and Exploration

Standard value functions represent expected returns as single number summarizing the distribution of possible outcomes. Distributional reinforcement learning instead learns the full distribution of returns. This richer representation captures information about risk and uncertainty that expectations discard. Empirically, distributional methods often learn faster and achieve better final performance, even when the distribution itself is ultimately reduced to an expectation for action selection.

Implicit Quantile Networks (IQN) provide a flexible approach to distributional learning. Rather than predicting a fixed set of quantiles, IQN learns to map any quantile fraction $\tau \in [0, 1]$ to its corresponding return value. This implicit representation adapts to the shape of the return distribution without requiring manual specification of its support.

Exploration in value-based methods traditionally relies on ε -greedy action selection: with probability ε , choose a random action; otherwise, act greedily. Noisy networks offer a learned alternative by adding parameterized noise to the network weights. The magnitude and structure of the noise adapt during training, providing state-dependent exploration that can be more efficient than uniform random action selection.

The Rainbow algorithm combines these advances — experience replay, target networks, double Q-learning, distributional values, noisy exploration, and several others into a unified framework. The specific configuration for this and the other two algorithms employed in this work is detailed in the later Methods section.

2.3.2 Policy Gradient Methods

Policy gradient methods take a fundamentally different approach: rather than learning values and deriving a policy indirectly, they parameterize the policy $\pi_\theta(a \mid s)$ and aim to optimize it directly through a locally approximating surrogate function. This formulation handles continuous action spaces naturally. The network outputs the parameters of a probability distribution—for instance, the mean and variance of a Gaussian—from which actions are sampled. Stochasticity is intrinsic to the policy rather than added externally for exploration.

The Policy Gradient Theorem

The objective is to maximize the expected return under the policy:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R_0]. \quad (2.6)$$

The policy gradient theorem provides a formula for the gradient of this objective:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t | s_t) \cdot R_t]. \quad (2.7)$$

This expression can be estimated from sampled trajectories without requiring knowledge of the environment dynamics. Actions that lead to high returns are reinforced; their probabilities increase. Actions leading to low returns are suppressed.

The main challenge with this estimator is variance. Returns can fluctuate dramatically between episodes, and multiplying log-probabilities by these noisy signals produces gradient estimates that vary wildly. High variance slows learning and can destabilize training.

Variance Reduction through Advantage Estimation

A key insight is that the gradient formula remains valid if a state-dependent baseline $b(s_t)$ is subtracted from the return:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t | s_t) \cdot (R_t - b(s_t))]. \quad (2.8)$$

If the baseline approximates the expected return $V^\pi(s_t)$, the quantity $R_t - b(s_t)$ estimates the advantage $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$: how much better action a_t was compared to the average action in state s_t . Subtracting the baseline does not change the expected gradient but substantially reduces its variance.

Generalized Advantage Estimation (GAE) provides a practical method for computing advantages that balances bias and variance through a parameter λ . Low values of λ yield low-variance but potentially biased estimates; high values reduce bias at the cost of increased variance. This tunability allows practitioners to adapt the algorithm to the characteristics of their problem.

Trust Regions and Proximal Policy Optimization

Policy gradient updates face a different danger: because the gradient is estimated using samples from the current policy, the estimate becomes unreliable if the policy changes substantially. A large update might move the policy far from the region where the gradient estimate is accurate, potentially causing catastrophic performance collapse.

Trust region methods address this by constraining how much the policy can change in a single update. The idea is to improve the policy while staying close to the current one, measured by a divergence such as the Kullback-Leibler divergence. Trust Region Policy Optimization (TRPO) enforces this constraint explicitly but requires computationally expensive second-order optimization.

Proximal Policy Optimization (PPO) achieves similar stability through a simpler mechanism. It tracks the probability ratio between the updated and original policies:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (2.9)$$

The objective function clips this ratio to the interval $[1 - \epsilon, 1 + \epsilon]$, preventing large policy changes regardless of how advantageous they might appear. This clipping serves as a soft trust region: updates that would change the policy too much are truncated. The resulting algorithm is simple to implement, requires only first-order optimization, and has proven remarkably effective across a wide range of problems.

2.3.3 Model-Based Methods

Model-based reinforcement learning learns a model of the environment predicting how states evolve and what rewards result from actions, and uses this model to inform decision-making. While model-free methods learn to act well through trial and error, model-based methods learn how the world works and leverage this understanding for more efficient learning.

World Models and Latent Representations

Learning to predict raw observations directly can be challenging, particularly for high-dimensional inputs. World models address this by learning a compact latent representation of states together with dynamics that operate in this latent space. An encoder maps observations to latent states; a dynamics model predicts how latent states evolve under actions; and auxiliary networks predict rewards and values from latent states.

Operating in latent space offers several advantages. The representation can discard irrelevant details, focusing on aspects of the state that matter for decision-making. Dynamics in latent space are often smoother and more predictable than in observation space. And the compact representation reduces the computational burden of both learning and planning.

Planning with Learned Models

Given a learned model, the agent can simulate the consequences of candidate action sequences without actually executing them. Model Predictive Control (MPC) optimizes actions over a finite planning horizon: starting from the current state, the model predicts trajectories for many candidate action sequences, evaluates their expected returns, and selects the first action of the best trajectory. After executing this action and observing the new state, planning repeats from scratch.

Re-planning at every step provides robustness to model errors. Even if predictions drift from reality over long horizons, the agent corrects course before errors accumulate catastrophically. Combining model-based planning with a learned value function further helps: rather than predicting rewards over an infinite horizon, the planner need only look a few steps ahead, using the value function to estimate the worth of terminal states.

TD-MPC2

TD-MPC2 exemplifies this hybrid approach. It jointly learns five components: an encoder mapping observations to latent states, a dynamics model predicting latent state transitions, a reward model predicting immediate rewards, a policy network suggesting promising actions, and a value network estimating long-term returns. At decision time, the algorithm uses the policy network to propose candidate action sequences, refines them through model-based trajectory optimization, and executes the best action found.

This architecture balances the sample efficiency of model-based learning with the robustness of model-free value estimation. The learned model enables efficient planning, while the value function limits the horizon over which the model must remain accurate.

Chapter 3

Methods

TODO:

- maybe thinking behind different reward functions
- maybe add something to showcase computational speedup
- improve on the reasoning for stopping condition + make it more clear that this was changed early in the ablation study
- (low priority) use figures to show the network structure and value/policy approximation functions / gradient flow or learning

3.1 Environment

3.1.1 Wheel Truing Physics

The environment used in this thesis is built upon a simulation based on a physical model of spoked bicycle wheels presented by Ford [1]. This section provides a brief overview of the problem formulation and numerical solution to the spoked wheel system, following the derivations in Chapter 2 of that work.

Structural Components and Modeling Assumptions

The spoked wheel consists of three structural components: the hub, rim, and spokes. The hub is treated as a rigid body due to its high stiffness relative to the other components; it provides fixed boundary conditions for the spoke attachments through its flange diameter d_h and width w_h .

The rim is modeled as a thin-walled curved beam with constant radius R measured at the shear center of the cross-section. The cross-section may exhibit monosymmetric geometry (symmetric about the radial axis but not the lateral axis), which introduces

coupling between lateral bending and torsional deformations. The rim material is assumed to be linearly elastic, obeying Hooke's law with Young's modulus E and shear modulus G . Following Euler-Bernoulli beam theory, shear deformations not associated with uniform torsion are neglected, and the cross-section is assumed to remain rigid in its own plane.

The spokes are modeled as ideal linear elastic bar elements that deform only along their length, characterized by Young's modulus E_s , cross-sectional area A_s , and length l_s . The spoke-rim and spoke-hub connections are idealized as moment-free ball-and-socket joints, meaning spokes transmit only axial forces.

To describe deformations, a local coordinate triad is defined with basis vectors $\{e_1, e_2, e_3\}$ pointing in the lateral (out-of-plane), radial (toward hub), and tangential (along rim circumference) directions respectively. The angular position along the rim is denoted by $\theta \in [-\pi, \pi]$. Each spoke's orientation is characterized by its unit direction vector $\mathbf{n} = [n_1, n_2, n_3]^T$ pointing from the rim attachment toward the hub.

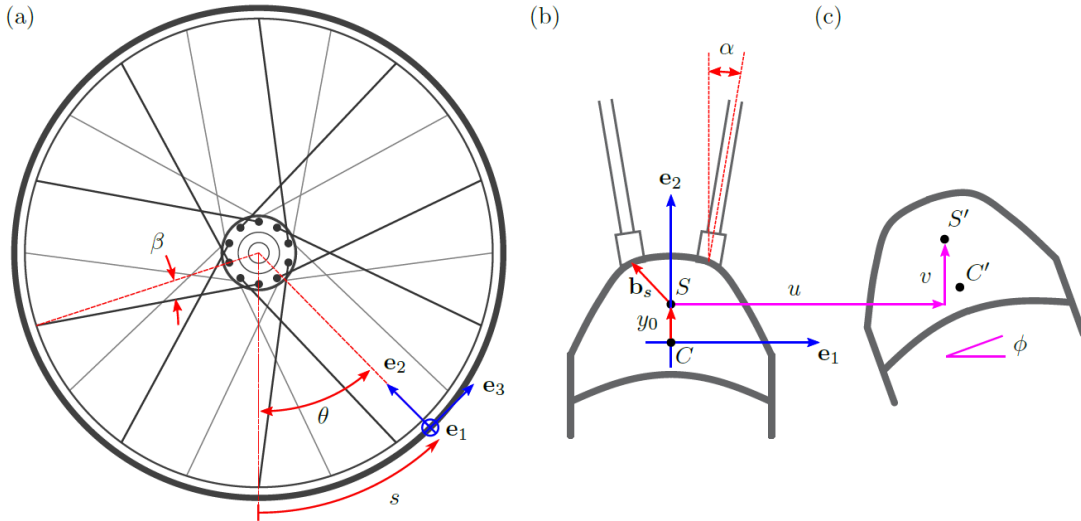


Figure 3.1: Schematic of a typical bicycle wheel. (a) Side view, looking at the hub. (b) Rim cross-section showing local coordinate system at the centroid and vector spoke offset b_s . (c) Rim cross-section after deformation. Tangential displacement ω is not shown. Reproduced from [1].

Displacement Field

The rim's deformation is fully characterized by the displacement and rotation of its shear center. At angular position θ , the displacement has three translational components:

- $u(\theta)$: Lateral displacement (in e_1 direction, parallel to hub axis)
- $v(\theta)$: Radial displacement (in e_2 direction, positive toward hub)

- $w(\theta)$: Tangential displacement (in e_3 direction, along circumference)

and one rotational component:

- $\phi(\theta)$: Twist angle about the tangential axis e_3

These collect into the augmented displacement vector:

$$\mathbf{d}(\theta) = [u, v, w, \phi]^T \quad (3.1)$$

which constitutes the fundamental unknown of the governing equations. The rotations of the cross-section are related to the displacement gradients through:

$$\begin{aligned} \omega_1 &= v' + \frac{w}{R} \\ \omega_2 &= u' \\ \omega_3 &= \phi \end{aligned} \quad (3.2)$$

where primes denote differentiation with respect to arc length $s = R\theta$. The deformation curvatures, found by differentiating the rotation vector with respect to s , are:

$$\begin{aligned} \kappa_1 &= v'' + \frac{w'}{R} && \text{(in-plane bending)} \\ \kappa_2 &= u'' + \frac{\phi}{R} && \text{(out-of-plane bending)} \\ \kappa_3 &= \phi' - \frac{u'}{R} && \text{(twist)} \end{aligned} \quad (3.3)$$

Strain Energy

The strain energy stored in the rim is derived from the elastic strain energy density. For a linear elastic material, the longitudinal strain at a point (x, y) in the cross-section is given by:

$$\varepsilon_{33} = w' - \frac{v}{R} - x \left(u'' + \frac{\phi}{R} \right) + (y - y_0) \left(v'' + \frac{w'}{R} \right) + \left(\phi'' - \frac{u''}{R} \right) \alpha_s \quad (3.4)$$

where y_0 is the shear center offset and α_s is the normalized warping function. This warping function describes how the thin-walled beam cross-sections deform out-of-plane when subjected to torsion; for closed box-section rims that resist warping effectively, this term can be neglected. The first term represents axial strain at the centroid, while subsequent terms capture the contributions from bending and warping. The shear strain associated with uniform torsion is:

$$\gamma = 2\xi \left(\phi' - \frac{u'}{R} \right) \quad (3.5)$$

where ξ is the distance from the section midplane.

The total rim strain energy is obtained by integrating the strain energy density $\frac{1}{2}(E\varepsilon_{33}^2 + G\gamma^2)$ over the cross-section and around the circumference:

$$U_{\text{rim}} = \frac{1}{2} \int_0^{2\pi R} \int_A (E\varepsilon_{33}^2 + G\gamma^2) dA ds \quad (3.6)$$

Substituting the strain expressions and carrying out the cross-sectional integration using the definitions of the section properties (A , I_1 , I_2 , J , I_ω) yields:

$$U_{\text{rim}} = \frac{1}{2} \int_0^{2\pi R} [EA\varepsilon_c^2 + EI_1\kappa_1^2 + EI_2\kappa_2^2 + GJ\kappa_3^2 + EI_\omega(\kappa_3')^2] ds \quad (3.7)$$

where $\varepsilon_c = w' - v/R - y_0(v'' + w'/R)$ is the axial strain at the centroid. The five terms represent energy contributions from axial deformation, radial bending (in-plane), lateral bending (out-of-plane), uniform torsion, and warping torsion, respectively.

Spoke Stiffness and Incremental Formulation

For the spokes, an incremental formulation distinguishes between a prestressed reference configuration and small deformations about that state. The wheel is assumed to be uniformly tensioned to a pretension T^p in the reference configuration. The goal is to find the additional displacement $\delta\mathbf{u}$ and force $\delta\mathbf{f}$ when the wheel deforms from this prestressed state. This incremental approach, rooted in the principle of virtual work, allows linearization of the spoke response about the prestressed equilibrium.

Each spoke contributes stiffness through two mechanisms. The *material stiffness* arises from elastic elongation along the spoke axis:

$$K_s = \frac{E_s A_s}{l_s} \quad (3.8)$$

The *geometric stiffness* arises from the pretension: when a tensioned spoke undergoes transverse displacement $\delta u_\perp$, it rotates through angle $\delta u_\perp/l_s$, causing the tension force to acquire a restoring component $(T^p/l_s)\delta u_\perp$ perpendicular to the original direction. This is the same effect that gives a tensioned string its transverse stiffness.

Taking the vector sum of these contributions, the incremental spoke force is:

$$\delta\mathbf{f} = \mathbf{k}_f \delta\mathbf{u}_n \quad (3.9)$$

where the 3×3 spoke stiffness matrix is:

$$\mathbf{k}_f = \frac{E_s A_s}{l_s} (\mathbf{n} \otimes \mathbf{n}) + \frac{T^p}{l_s} (\mathbf{I} - \mathbf{n} \otimes \mathbf{n}) \quad (3.10)$$

Here $\mathbf{n} \otimes \mathbf{n}$ denotes the outer product (dyadic product) of the spoke direction vector

with itself, and $\mathbf{I}$ is the 3×3 identity matrix. The first term provides stiffness only along the spoke axis, while the second term provides stiffness in the plane perpendicular to the spoke.

The incremental displacement $\delta \mathbf{u}_n$ at the spoke nipple depends on both the rim translation and twist. If the nipple is offset from the shear center by vector $\mathbf{b}_s$:

$$\delta \mathbf{u}_n = [\delta u, \delta v, \delta w]^T + \delta \phi (\mathbf{e}_3 \times \mathbf{b}_s) \quad (3.11)$$

Following Ford's simplification, the tangential component of this displacement (proportional to gradients of $\delta \mathbf{u}_s$) is neglected due to the small tangential projection of the spokes. To couple the spoke response with the full rim displacement vector $\mathbf{d} = [u, v, w, \phi]^T$, the stiffness is extended to a 4×4 augmented form:

$$\mathbf{k}_i = \begin{bmatrix} \mathbf{k}_f & \mathbf{k}_f(\mathbf{e}_3 \times \mathbf{b}_s) \\ (\mathbf{e}_3 \times \mathbf{b}_s)^T \mathbf{k}_f & (\mathbf{e}_3 \times \mathbf{b}_s)^T \mathbf{k}_f (\mathbf{e}_3 \times \mathbf{b}_s) \end{bmatrix} \quad (3.12)$$

Modal Decomposition

Equilibrium requires minimizing the total potential energy $\Pi = U_{\text{rim}} + U_{\text{spokes}} - W_{\text{ext}}$ with respect to the displacement field. A Rayleigh-Ritz approximation is employed by expanding the displacement components in truncated Fourier series. Since the rim forms a closed loop, periodicity is naturally satisfied:

$$u(\theta) = u_0 + \sum_{n=1}^N [u_n^c \cos(n\theta) + u_n^s \sin(n\theta)] \quad (3.13)$$

$$v(\theta) = v_0 + \sum_{n=1}^N [v_n^c \cos(n\theta) + v_n^s \sin(n\theta)] \quad (3.14)$$

$$w(\theta) = w_0 + \sum_{n=1}^N [w_n^c \cos(n\theta) + w_n^s \sin(n\theta)] \quad (3.15)$$

$$\phi(\theta) = \phi_0 + \sum_{n=1}^N [\phi_n^c \cos(n\theta) + \phi_n^s \sin(n\theta)] \quad (3.16)$$

where N is the truncation order. The Fourier coefficients are collected into the mode coefficient vector:

$$\mathbf{d}_m = [u_0, v_0, w_0, \phi_0, u_1^c, u_1^s, v_1^c, v_1^s, w_1^c, w_1^s, \phi_1^c, \phi_1^s, \dots]^T \quad (3.17)$$

which has dimension $4 + 8N$. The physical displacement at any angle θ is recovered through the mode shape matrix $\mathbf{B}(\theta)$:

$$\mathbf{d}(\theta) = \mathbf{B}(\theta) \mathbf{d}_m \quad (3.18)$$

where $\mathbf{B}(\theta)$ is a $4 \times (4 + 8N)$ matrix with entries composed of sines and cosines evaluated at θ .

Stiffness Matrices

Substituting the modal expansion into the rim strain energy (3.7) yields a quadratic form:

$$U_{\text{rim}} = \frac{1}{2} \mathbf{d}_m^T \mathbf{K}_{\text{rim}} \mathbf{d}_m \quad (3.19)$$

Due to the orthogonality of the Fourier basis functions over the circle, the rim stiffness matrix $\mathbf{K}_{\text{rim}}$ is block-diagonal, with separate blocks for each mode number. This structure reflects the fact that, for the rim alone, different Fourier modes do not interact.

The spoke strain energy transforms similarly. Evaluating the displacement at each spoke location θ_i using (3.18) and summing over all spokes:

$$U_{\text{spokes}} = \frac{1}{2} \sum_{i=1}^{n_s} \mathbf{d}(\theta_i)^T \mathbf{k}_i \mathbf{d}(\theta_i) = \frac{1}{2} \mathbf{d}_m^T \left(\sum_{i=1}^{n_s} \mathbf{B}_i^T \mathbf{k}_i \mathbf{B}_i \right) \mathbf{d}_m = \frac{1}{2} \mathbf{d}_m^T \mathbf{K}_{\text{spk}} \mathbf{d}_m \quad (3.20)$$

where $\mathbf{B}_i = \mathbf{B}(\theta_i)$ is the mode shape matrix evaluated at the i -th spoke's rim attachment angle. The spoke stiffness matrix in modal coordinates is therefore:

$$\mathbf{K}_{\text{spk}} = \sum_{i=1}^{n_s} \mathbf{B}_i^T \mathbf{k}_i \mathbf{B}_i \quad (3.21)$$

Unlike $\mathbf{K}_{\text{rim}}$, this matrix is *not* block-diagonal: the discrete spoke locations introduce coupling between different Fourier modes, since products of trigonometric functions evaluated at discrete points do not satisfy the same orthogonality relations as their continuous integrals. This discrete formulation is used in the implementation rather than the “smeared spokes” approximation discussed by Ford, which averages the spoke stiffness into a continuous elastic foundation. The discrete approach better captures mode coupling effects that are important for accurate truing simulation.

The Truing Problem

For wheel truing, the case of no external loads ($\mathbf{F}_{\text{ext}} = \mathbf{0}$) but with non-uniform spoke adjustments is considered. When a spoke nipple is rotated, it changes the spoke's effective rest length. Let a_i denote the length adjustment for spoke i (positive for tightening, which shortens the rest length). This adjustment creates an internal force along the spoke direction that must be balanced by rim deformation.

The adjustment forces enter the equilibrium equation through the spoke adjustment matrix $\mathbf{A}_{\text{adj}}$, whose i -th column represents the modal force produced by a unit adjustment

of spoke i :

$$\mathbf{A}_{\text{adj}}[:, i] = \frac{E_s A_s}{l_i} \mathbf{B}_i^T \begin{bmatrix} \mathbf{n}_i \\ \mathbf{e}_3 \cdot (\mathbf{b}_i \times \mathbf{n}_i) \end{bmatrix} \quad (3.22)$$

Physically, tightening a spoke creates a force pulling the rim toward the hub along the spoke direction.

The governing equation for wheel truing is therefore:

$$\boxed{(\mathbf{K}_{\text{rim}} + \mathbf{K}_{\text{spk}}) \mathbf{d}_m = \mathbf{A}_{\text{adj}} \mathbf{a}} \quad (3.23)$$

where $\mathbf{a} = [a_1, a_2, \dots, a_{n_s}]^T$ is the vector of spoke adjustments. Given a set of spoke adjustments, the rim displacement is found by solving this linear system:

$$\mathbf{d}_m = (\mathbf{K}_{\text{rim}} + \mathbf{K}_{\text{spk}})^{-1} \mathbf{A}_{\text{adj}} \mathbf{a} \quad (3.24)$$

Once the mode coefficients $\mathbf{d}_m$ are known, the physical rim displacement at any angle θ is computed via (3.18), and the resulting spoke tension changes are:

$$T_i = T_0 + \Delta T_i, \quad \Delta T_i = \frac{E_s A_s}{l_i} (a_i - \mathbf{n}_i \cdot \mathbf{u}_{n,i}) \quad (3.25)$$

where T_0 is the initial uniform pretension and $\mathbf{u}_{n,i}$ is the displacement of the i -th spoke nipple.

Linearization and Computational Implementation

The formulation above assumes that the stiffness matrices $\mathbf{K}_{\text{rim}}$ and $\mathbf{K}_{\text{spk}}$ are computed for a reference configuration with uniform spoke tension T_0 . This linearization is justified for wheel truing because:

1. Typical truing adjustments cause tension variations of ± 100 – 200 N around the nominal $T_0 = 800$ N, representing relative changes of less than 25%.
2. The geometric stiffness terms (proportional to T/l_s) contribute primarily to lateral stiffness and are secondary to the material stiffness terms for in-plane deformations.
3. The goal of truing is to return the wheel to a nearly uniform tension state, so large deviations are transient.

In the implementation, the shear center offset y_0 and warping moment of inertia I_ω are set to zero, which is appropriate for symmetric, closed box-section rims typical of modern bicycles. The stiffness matrices $\mathbf{K}_{\text{rim}}$, $\mathbf{K}_{\text{spk}}$, and the adjustment matrix $\mathbf{A}_{\text{adj}}$ are precomputed during environment initialization. Each simulation step then requires

only solving the linear system (3.23) for the current adjustment vector $\mathbf{a}$, evaluating rim displacements via (3.18), and computing spoke tension changes via (3.25). With $N = 40$ Fourier modes, the system matrix has dimension 324×324 .

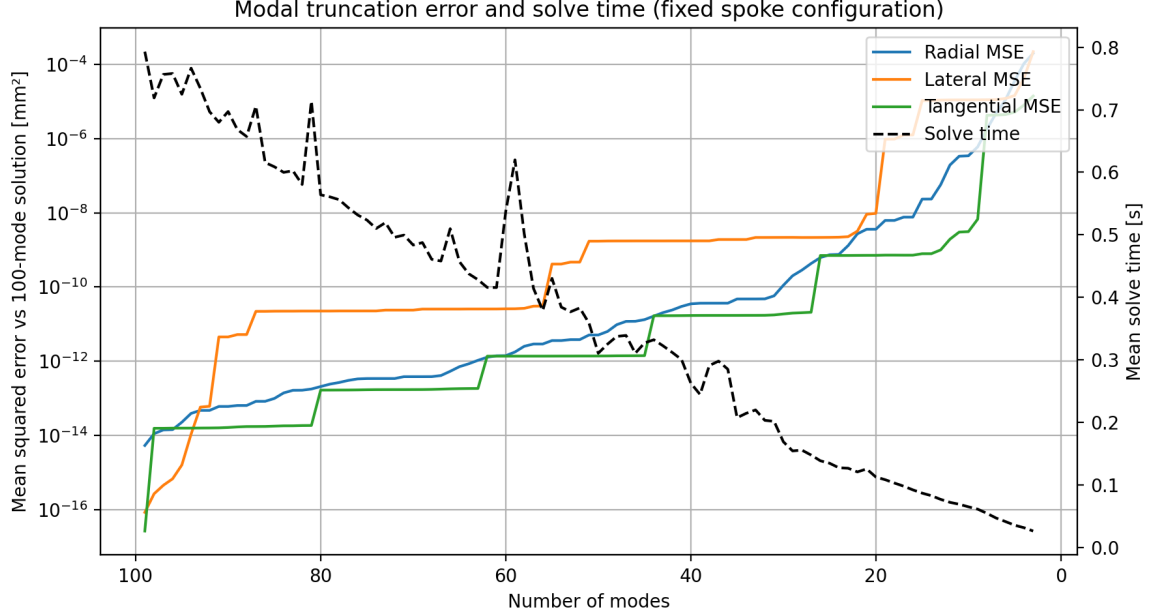


Figure 3.2: MSE and calculation time depending on mode Number

To achieve the computational throughput required for reinforcement learning, where millions of environment steps may be needed, the core simulation loop is implemented using Numba’s just-in-time (JIT) compilation. The `@njit` decorator compiles the displacement and tension calculations to optimized machine code, yielding approximately two orders of magnitude speedup compared to pure Python/NumPy execution. This optimization enables rapid iteration during training while maintaining the full fidelity of the discrete-spoke formulation.

The properties of the wheel simulation are characterized by the parameters shown in Table 3.1.

3.1.2 On State-Space Formulations

The complete state is defined by the displacement field and spoke tensions in equilibrium as explained above. For representing the state we can choose an arbitrary number of points around the rim in terms of the rotation angle θ . We make this choice to be 360 points, equidistant around the circle, i.e.: $s = [\theta_1, \theta_1, \dots, \theta_{360}]$. For each of these points, we can simulate the displacement triad, giving us the local lateral, tangential and radial displacement. As it is not realistic to discern the tangential displacement by eye when truing a wheel, or even measure it through sensors, it is omitted from the state. The spoke tensions in equilibrium are calculated by the simulation in units of Newtons.

Table 3.1: Wheel simulation parameters

Parameter	Value	Description
n_spokes	36	Number of spokes
hub_width	50 mm	Hub width (distance between flanges)
hub_diameter	40 mm	Hub flange diameter
rim_radius	300 mm	Rim radius (at shear center)
rim_area	100 mm ²	Rim cross-sectional area
rim_I_lat	1500 mm ⁴	Rim lateral moment of inertia (I_2)
rim_I_rad	3000 mm ⁴	Rim radial moment of inertia (I_1)
rim_J_tor	500 mm ⁴	Rim torsion constant (J)
rim_young_mod	69 GPa	Rim Youngs modulus (aluminum)
rim_shear_mod	26 GPa	Rim shear modulus
rim_I_warp	0.0 mm ⁶	Rim warping constant (I_w)
spokes_crossings	3	Number of spoke crossings (lacing pattern)
spokes_diameter	2.0 mm	Spoke wire diameter
spokes_young_mod	210 GPa	Spoke Youngs modulus (steel)
number_modes	40	Number of Fourier modes (N)
init_tension	800 N	Initial spoke pretension (T_p)

To reduce the variation in absolute values in the state (displacement vectors on the order of a few millimeters vs. spoke tensions on the order of hundreds of Newtons), the tensions are normalized by using only their deviation from the initial tension (800N) and dividing by their maximum range on reset (400). Since the displacement around the wheel is periodic by nature, we can also express this displacement in terms of its Fourier coefficients. For the lateral and radial displacement fields $u_{\text{lat}}(\theta)$ and $u_{\text{rad}}(\theta)$, we compute the discrete Fourier transform coefficients up to N harmonics:

$$c_k = \frac{1}{M} \sum_{j=0}^{M-1} u(\theta_j) e^{-2\pi i k j / M}, \quad k = 0, 1, \dots, N \quad (3.26)$$

where $M = 360$ is the number of sample points, $\theta_j = 2\pi j / M$ are the equidistant angular positions, and $u(\theta_j)$ represents either the lateral or radial displacement at position j . The Fourier representation encodes the periodic rim displacement using harmonic coefficients, reducing the state dimension from $2 \times 360 = 720$ rim points to $2 + 4N$ features (two DC components plus magnitude and phase for each harmonic of both radial and lateral displacement). To select an appropriate number of harmonics N , we analyze the reconstruction accuracy, the percentage of variance in the original displacement signal captured by the truncated Fourier series. Figure 3.3 shows that reconstruction accuracy exceeds 99% at $N = 10$ and 99.99% at $N = 15$, with diminishing returns beyond this point. We select $N = 30$ to ensure negligible information loss ($> 99.999\%$ reconstruction) while maintaining a compact state representation of 122 features, corresponding to a $5.9\times$ compression ratio.

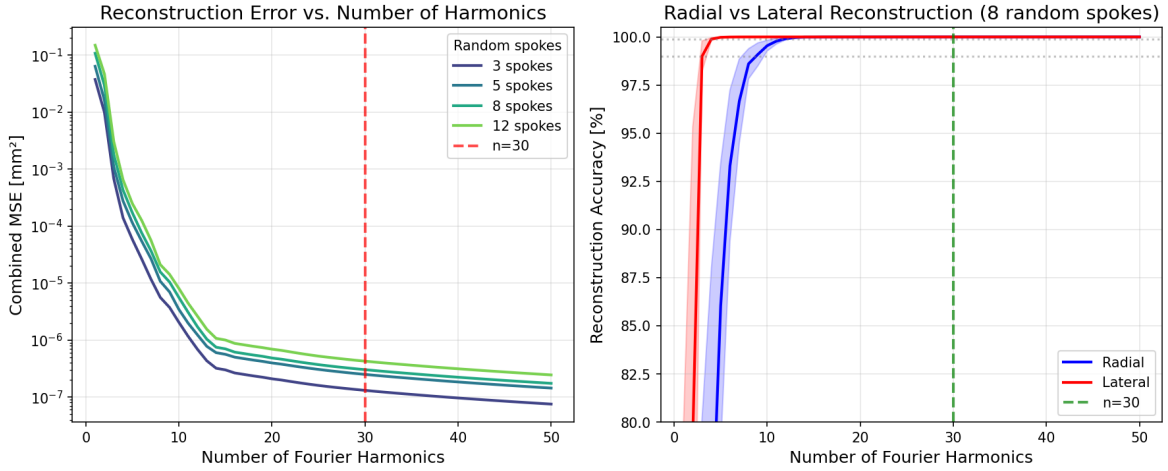


Figure 3.3: Fourier reconstruction analysis for the wheel displacement state. Left: Combined mean squared error between original and reconstructed displacement versus number of harmonics, shown for different levels of initial wheel damage (3–12 randomly detuned spokes). Right: Reconstruction accuracy (variance explained) for radial and lateral displacement components separately, with shaded regions indicating standard deviation across 50 random initial states. The vertical dashed line marks the chosen value of $N = 30$ harmonics.

To explore how much of the full state is required for the agent to learn and if we could get away without measuring the spokes in a potential physical implementation, we combine these parts into the four state-space formulations summarized in Table 3.2.

Table 3.2: State-space formulations investigated in this work. Code identifiers correspond to labels used in experimental plots. Statistics are computed over 500 sampled initial configurations.

State Space	Code Identifier	Dim.	Component Statistics
Displacement only	<code>rimpoints</code>	720	Radial: $\mu \approx 0$, $\sigma = 0.025$ mm; Lateral: $\mu \approx 0$, $\sigma = 0.29$ mm
Displacement + tensions	<code>rimandspokes</code>	756	As above, plus tensions: $\mu = 1.0$, $\sigma = 0.077$
Fourier only	<code>fourier</code>	124	Magnitude range: $[10^{-5}, 1]$
Fourier + tensions	<code>fourierandspokes</code>	160	As above, plus tensions: $\mu = 1.0$, $\sigma = 0.077$

3.1.3 On the Starting State

On reset of the environment, i.e., start of a new episode, we need to construct an out-of-true wheel. We do this by starting with a true wheel with completely uniform tensions and then bringing it out of true by choosing n_{choice} spokes and turning them

some random uniformly sampled value from $[-\max_{\text{turn}}, \max_{\text{turn}}]$. We choose somewhat arbitrarily $n_{\text{choice}} = 5$ and $|\max_{\text{turn}}| = |-\max_{\text{turn}}| = 1$.

Distribution of Initial State Components

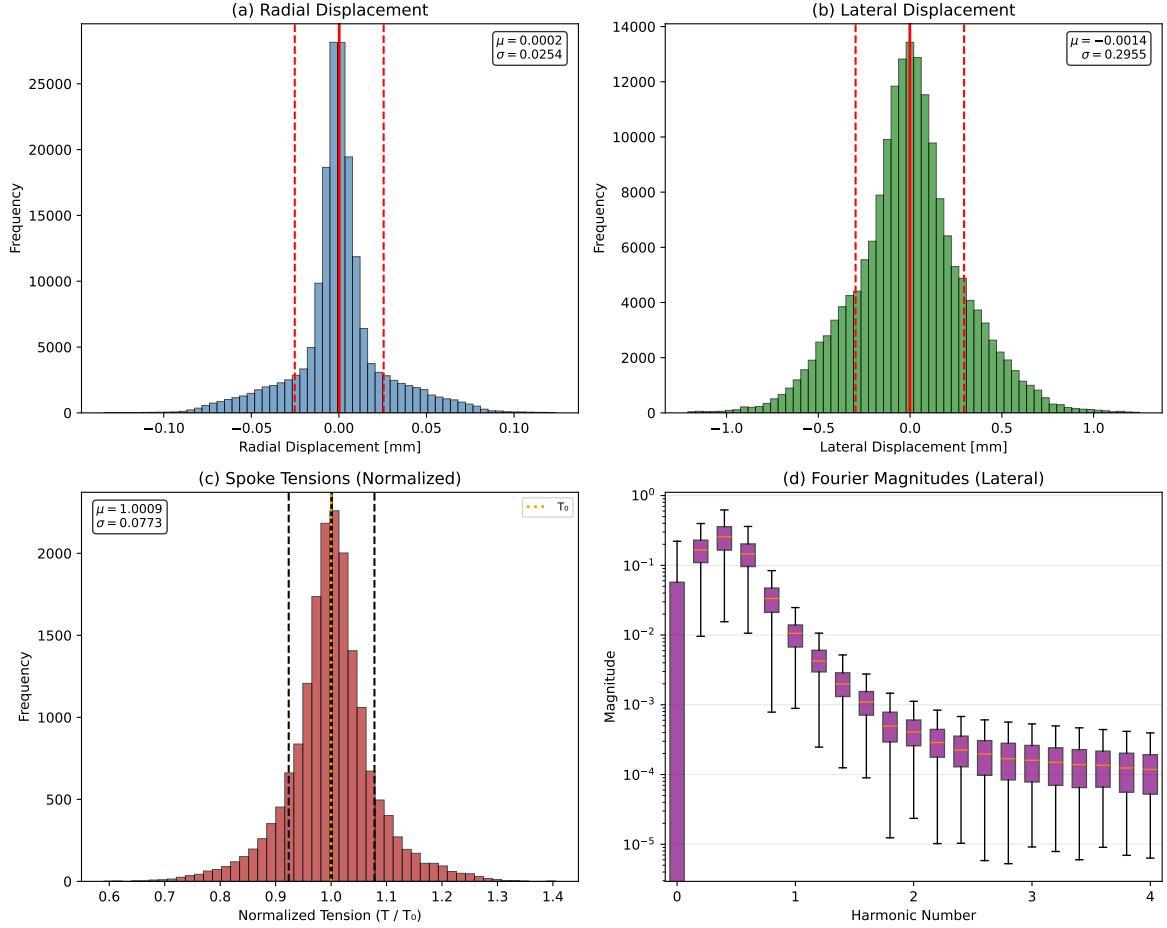


Figure 3.4: Distribution of initial state components across 500 sampled wheel configurations. (a) Radial displacements are approximately an order of magnitude smaller than (b) lateral displacements. (c) Spoke tensions are normalized by the initial tension $T_0 = 800$ N, centering the distribution around 1.0. (d) The Fourier magnitude spectrum shows that lower harmonics dominate the initial disturbances. Red lines indicate mean and ± 1 standard deviation.

3.1.4 On Action-Space Formulations

Since DQN variants only work with discrete and TDMPC2 only with continuous action spaces, we need at least two formulations. Starting with the discrete formulation, we let the agent choose from $[2 \cdot n_{\text{spokes}}]$ actions corresponding to a 0.1 turn in $+/-$ direction of the corresponding spoke. We can transform this directly into a 2D continuous action-space by letting the first variable control which spoke should be adjusted and letting the second control the adjustment size. Additionally, we can also formulate a continuous action by letting the agent adjust all the spokes at once with a vector of length n_{spokes} , giving us the following three action spaces $\mathcal{A}$:

- **Continuous spoke selection:** two-dimensional continuous space

$$\mathcal{A}_{\text{continuous}} = [0, n_{\text{spokes}} - 1] \times [-1, 1] \subset \mathbb{R}^2 \quad (3.27)$$

where n_s is the number of spokes. An action $a = (s, \Delta t) \in \mathcal{A}_{\text{continuous}}$ specifies a continuous spoke index s and a tension adjustment Δt .

- **Discrete spoke selection:**

$$\mathcal{A}_{\text{discrete}} = \{0, 1, 2, \dots, 71\} \quad (3.28)$$

representing 72 distinct action choices, where each action corresponds to a specific spoke-tension adjustment pair.

- **All spokes adjustment:** n_{spokes} -dimensional continuous space

$$\mathcal{A}_{\text{all_spokes}} = [-1, 1]^{n_{\text{spokes}}} \subset \mathbb{R}^{n_s} \quad (3.29)$$

where an action $a = (\Delta t_1, \Delta t_2, \dots, \Delta t_{n_s}) \in \mathcal{A}_{\text{all_spokes}}$ specifies simultaneous tension adjustments for all spokes.

For the continuous action space formulation we make a special choice for the PPO-Algorithm: Treating both dimensions with a shared Gaussian distribution conflates two fundamentally different quantities, a discrete choice among the spokes and a bounded continuous adjustment making it difficult to balance exploration appropriately across both. We therefore opt for a two-headed network design to reflect this discrete-continuous dichotomy.

Algorithm-Action Space Compatibility

Not all algorithms can be evaluated on all action space formulations due to inherent design constraints. Table 3.3 summarizes the valid combinations.

The constraints arise from fundamental algorithmic requirements:

Table 3.3: Algorithm–action space compatibility matrix. Checkmarks indicate valid combinations tested in this work.

Algorithm	Discrete	Continuous	All-Spokes
Rainbow DQN	✓	—	—
PPO	✓	✓	✓
TD-MPC2	—	✓	✓

- **Rainbow DQN:** Requires enumeration of Q-values for all actions, restricting it to discrete formulations.
- **TD-MPC2:** Performs gradient-based trajectory optimization, requiring differentiable (continuous) action spaces.
- **PPO:** Accommodates both discrete (categorical policy) and continuous (Gaussian policy) actions through appropriate policy parameterization.

This compatibility structure has implications for cross-algorithm comparisons: direct comparison is only possible where action spaces overlap. PPO serves as a bridge, being the only algorithm evaluated on all three action formulations.

3.1.5 On Reward Functions

The formulation of the reward is notoriously important for successful reinforcement learning.

Generally, we can construct a scalar reward from a given wheel-state by either looking at the remaining spoke turns from its initial true state or by measuring the total displacement by summing over the displacement vectors weighted by $d\theta$:

$$\|s_t\| \approx \sqrt{\sum_{k=1}^{N-1} \frac{\Delta\theta_k}{2} [\|u(\theta_k)\|_2^2 + \|u(\theta_{k+1})\|_2^2]} \quad (3.30)$$

where $\|u(\theta)\|_2^2 = \sum_{d=1}^D u_d^2(\theta)$ is the squared Euclidean norm of the displacement vector at angle θ , and define the step-improvement I_{step} as $\|s_{t-1}\| - \|s_t\|$.

- **Raw reward:** Directly penalizes the remaining displacement in the rim

$$r_t^{\text{raw}} = -\|s_t\| \quad (3.31)$$

- **Normalized reward:** Provides binary feedback based on step-wise improvement

$$r_t^{\text{normalized}} = \begin{cases} 0 & \text{if } I_{\text{step}} > 0 \\ -1 & \text{if } I_{\text{step}} \leq 0 \end{cases} \quad (3.32)$$

- **Spokes reward:** Provides binary feedback based on spoke adjustment magnitude

$$r_t^{\text{spokes}} = \begin{cases} 0 & \text{if } |\Delta t_i^{(t)}| \leq |\Delta t_i^{(t-1)}| \text{ for all } i \\ -1 & \text{otherwise} \end{cases} \quad (3.33)$$

where $\Delta t_i^{(t)}$ denotes the turns applied to spoke i at time step t .

- **Combined reward:** Integrates both geometric displacement and spoke tension deviation into a single signal

$$r_t^{\text{combined}} = - \left(\|s_t\| + \frac{1}{n_s} \sum_{j=1}^{n_s} \frac{|T_j - T_0|}{400} \right) \quad (3.34)$$

where T_0 is the initial spoke tension and the scaling factor of 400 normalizes the tension deviation to be comparable in magnitude to the displacement norm. This formulation provides continuous feedback on both objectives simultaneously, encouraging the agent to reduce rim displacement while also maintaining spoke tensions close to their nominal value. The motivation for this combined signal is that optimizing displacement alone may lead to solutions with uneven tension distributions, while the tension term guides the agent toward states that satisfy both geometric and mechanical constraints.

Additionally, we introduce two stopping conditions to reward the agent for producing a true wheel or punish for exceeding a tension threshold. Some research into actual manual wheel truing shows that the "trueness" is often evaluated only by the rim displacement with less concern for the uniformity of spoke tensions. The state of a true wheel is often seen as dependent on the material quality of the wheel. In this context, we often see discussion around the concepts of "decent," "good," and "golden" wheels, where these generally have the following specifications:

- A decent wheel refers to a common quality wheel which is considered reasonably true with a maximum rim displacement of 1–2 mm and a relative tension deviation of 15–20%
- A good wheel refers to a good quality wheel with a maximum rim displacement of 0.5–1 mm and a relative tension deviation of 5–10%
- A golden wheel refers to an excellent quality wheel with a maximum rim displacement of less than 0.5 mm and a relative tension deviation of under 5%

Since in simulation we can assume no errors or inherent tension in the material of the rim, we can orient ourselves close to the golden standard. From this we introduce the success condition as:

$$|u_d(\theta_i)| < 0.3 \text{ mm} \quad \forall d \in \{1, \dots, D\}, \forall i \in \{1, \dots, N\} \quad (3.35)$$

$$700 \text{ N} \leq T_j \leq 900 \text{ N} \quad \forall j \in \{1, \dots, n_s\} \quad (3.36)$$

$$\max_j T_j - \min_j T_j \leq 0.1 \cdot \bar{T} \quad (3.37)$$

where $u_d(\theta_i)$ represents the displacement in dimension d at discrete angle θ_i , T_j is the tension in spoke j , and $\bar{T} = \frac{1}{n_s} \sum_{j=1}^{n_s} T_j$ is the mean spoke tension. When these conditions are met, the episode terminates with a bonus reward of $r_t = 20$.

For the penalty condition we consider the fact that most wheel manufacturers give a maximum tension of 120 kg force on a spoke. We consider the episode as failed when a single spoke exceeds this threshold and we punish the agent with a reward of $r_t = -20$. To avoid filling the replay buffer with exceptionally bad states at the beginning, we also terminate the episode after exceeding this threshold. So our second stopping condition can be formulated as:

$$\text{if } \exists j \in \{1, \dots, n_s\} : T_j > T_{\max} \quad \text{then} \quad r_t \leftarrow r_t - 20, \text{ truncated} = \text{True} \quad (3.38)$$

where $T_{\max}$ is the maximum tension threshold.

Important: *This truncation condition has shown itself to be exploitable in the continuous action spaces and was changed for PPO and TD-MPC2 to:*

$$\text{if } \exists j \in \{1, \dots, n_s\} : T_j > T_{\max} \quad \text{then} \quad r_t \leftarrow r_t - 2, \text{ truncated} = \text{False} \quad (3.39)$$

3.1.6 Episode Length

The maximum episode length must balance several competing considerations. From the starting state distribution we can see that the initial wheel configurations are, on average, approximately 25 adjustment steps of 0.1 turns away from the perfectly true state. This provides a baseline for the minimum episode length required.

However, the episode length should exceed this minimum to accommodate exploration during learning and to allow suboptimal policies sufficient time to eventually true the wheel. At the same time, excessively long episodes would cause the agent to frequently visit highly suboptimal states early in training, potentially degrading learning efficiency by filling the replay buffer with low-quality experiences.

Balancing these considerations, we set the maximum episode length to 41 steps. This provides approximately 60% headroom above the average optimal path length, giving the agent adequate room for exploration while preventing episodes from persisting

indefinitely in poor states.

3.2 Algorithm Configurations

This section describes the specific configurations of each reinforcement learning algorithm used in the experiments. While the algorithmic foundations were introduced in Section 2.3, the focus here is on implementation choices, network architectures, and hyperparameter settings.

3.2.1 Rainbow DQN

Implementation Basis The Rainbow DQN implementation is adapted from the open-source PyTorch implementation by Kaixhin¹. The primary modification replaces the original Categorical DQN (C51) distributional component with Implicit Quantile Networks (IQN) [2], which avoids the need to predefine a fixed support for the value distribution and instead learns the full quantile function by conditioning on sampled quantile fractions $\tau \in [0, 1]$.

Rainbow Components The implementation incorporates the following components from the Rainbow framework:

- **Double DQN** [3]: action selection uses the online network while evaluation uses the target network which is updated every 8000 steps, reducing overestimation bias.
- **Dueling architecture** [4]: the network separately estimates state-value $V(s)$ and advantage $A(s, a)$ streams, combining them as $Q(s, a) = V(s) + A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a')$.
- **Distributional RL**: Implicit Quantile Networks (IQN) [2] to avoid having to choose a viable base for the distribution, rather than learning fixed quantiles, IQN learns the full quantile function by conditioning on sampled quantile fractions $\tau \in [0, 1]$.
- **Prioritized Experience Replay** [5]: transitions are sampled proportionally to their temporal-difference error with priority exponent $\alpha = 0.5$.
- **Noisy networks** [6]: all linear layers use factorized Gaussian noise with initial standard deviation $\sigma_0 = 0.1$, replacing ε -greedy exploration.
- **Multi-step returns**: 3-step returns are used, providing a balance between bias and variance in the bootstrap target.

¹<https://github.com/Kaixhin/Rainbow>

Network Architecture The Q-network follows a dueling architecture with IQN-style quantile conditioning. The architecture processes states through parallel value and advantage streams:

- **Input layer:** State dimension varies by representation (see Table 3.2). With `history_length = 1`, the input dimension equals the raw state dimension.
- **Value stream:** One noisy linear layer mapping input to 500 hidden units with ReLU activation, followed by quantile-conditioned processing and a final noisy linear layer outputting a scalar value.
- **Advantage stream:** Parallel structure to the value stream, with the final layer outputting 72 advantage values (one per action).
- **Quantile embedding:** A cosine embedding network with embedding dimension 64, mapping sampled quantile fractions τ to hidden representations via:

$$\phi(\tau) = \text{ReLU} \left(\sum_{i=0}^{63} \cos(\pi i \tau) \cdot \mathbf{w}_i + b \right) \quad (3.40)$$

The quantile embeddings are combined with state features through element-wise multiplication.

The complete forward pass samples $N = 32$ quantile fractions uniformly from $[0, 1]$, producing Q-value distributions represented as $(|\mathcal{A}| \times N)$ quantile values per state.

Noisy Layer Implementation The noisy linear layers implement factorized Gaussian noise [6]. For a layer with p inputs and q outputs, the weight noise is factorized as:

$$\varepsilon_{i,j}^w = f(\varepsilon_i) f(\varepsilon_j), \quad \text{where } f(x) = \text{sgn}(x) \sqrt{|x|} \quad (3.41)$$

This reduces the number of random variables from $pq + q$ to $p + q$ while maintaining sufficient exploration.

Hyperparameters Table 3.4 summarizes the hyperparameters used for Rainbow DQN training.

Loss Function The network is trained using the quantile Huber loss. For a transition (s, a, r, s') with sampled quantile fractions $\tau_1, \dots, \tau_N$ for the online network and $\hat{\tau}_1, \dots, \hat{\tau}_{N'}$ for the target, the loss is:

$$\mathcal{L} = \frac{1}{N \cdot N'} \sum_{i=1}^N \sum_{j=1}^{N'} \rho_{\tau_i}^{\kappa}(\delta_{ij}) \quad (3.42)$$

Table 3.4: Rainbow DQN hyperparameters

Parameter	Value
<i>Learning</i>	
Learning rate	5.25×10^{-5}
Adam ε	1.5×10^{-4}
Discount factor γ	0.99
Gradient norm clipping	10.0
<i>Replay Buffer</i>	
Replay buffer capacity	1×10^6
Batch size	64
Learning starts	20,000 steps
Replay frequency	Every 4 steps
Priority exponent α	0.5
Priority weight β	0.4
<i>Target Network</i>	
Target network update	Every 8,000 steps
<i>Architecture</i>	
Hidden layer size	500
Noisy layer σ_0	0.1
Multi-step returns n	3
<i>IQN</i>	
Training quantiles N	32
Evaluation quantiles K	32
Quantile embedding dimension	64
Huber loss threshold κ	1.0

where $\delta_{ij} = r + \gamma Z_{\hat{\tau}_j}(s', a^*) - Z_{\tau_i}(s, a)$ is the temporal-difference error, $a^* = \arg \max_a \mathbb{E}[Z(s', a)]$ follows Double DQN, and ρ_τ^κ is the quantile Huber loss:

$$\rho_\tau^\kappa(\delta) = |\tau - \mathbf{1}_{\delta < 0}| \cdot \mathcal{L}_\kappa(\delta), \quad \mathcal{L}_\kappa(\delta) = \begin{cases} \frac{1}{2}\delta^2 & \text{if } |\delta| \leq \kappa \\ \kappa(|\delta| - \frac{1}{2}\kappa) & \text{otherwise} \end{cases} \quad (3.43)$$

3.2.2 PPO

Implementation Basis The PPO implementation is adapted from the open-source PyTorch implementation by Saqib². Several modifications were made to support the wheel truing task:

- **Hybrid actor architecture:** A dual-headed actor network (`HybridPPOAgent`) was implemented to support the continuous action space formulation, combining discrete spoke selection (categorical distribution) with continuous turn magnitude (Gaussian distribution).
- **Vectorized rollout collection:** The training loop was extended to support parallel environment execution using subprocess-based vectorization, improving sample throughput and GPU utilization.

Network Architecture The policy uses an actor-critic architecture with a shared feature extractor:

- **Shared feature extractor:** Two fully-connected layers with 500 units each and Tanh activations, processing the input state.
- **Actor head:** Action-space-dependent output layer (see below).
- **Critic head:** Single linear layer outputting a scalar value estimate.

The actor head is parameterized differently for each action space formulation:

- **Discrete (72 actions):** Linear layer followed by softmax, outputting a categorical distribution over 72 actions (36 spokes $\times$ 2 directions).
- **Continuous/Hybrid (2D):** A hybrid architecture using a separate `HybridPPOAgent` that combines discrete spoke selection (categorical distribution over 36 spokes) with a continuous turn magnitude (Gaussian distribution). This formulation allows the agent to select which spoke to adjust and by how much in a single action.

²<https://github.com/saqib1707/RL-PPO-PyTorch>

- **All-spokes (36D)**: Linear layer outputting 36 means for independent Gaussian distributions, one per spoke. The policy outputs simultaneous adjustments for all spokes.

Action Distribution For continuous action dimensions, the policy uses a multivariate Gaussian distribution with diagonal covariance:

$$\pi_{\theta}(a|s) = \mathcal{N}(\mu_{\theta}(s), \text{diag}(\sigma^2)) \quad (3.44)$$

where $\mu_{\theta}(s)$ is the network output and σ^2 is a *learned parameter* initialized to $\sigma_0^2 = 0.1^2$. The variance is implemented as a trainable `nn.Parameter`, allowing the policy to adapt its exploration over training.

Vectorized Training To improve sample throughput, training employs $n_{\text{envs}} = 8$ parallel environment instances using subprocess-based vectorization. Each policy update uses

$$T_{\text{update}} = T_{\text{interval}} = 1000 \text{ steps} \quad (3.45)$$

collected across all environments, corresponding to 125 steps per environment.

Generalized Advantage Estimation Advantages are computed using Generalized Advantage Estimation (GAE) [7]:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \text{where } \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (3.46)$$

with discount factor $\gamma = 0.99$ and GAE parameter $\lambda = 0.95$. Advantages are normalized to zero mean and unit variance before each update.

Optimization The policy is updated using the clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = -\mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (3.47)$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio and $\varepsilon = 0.2$ is the clip parameter.

The total loss combines policy, value, and entropy terms:

$$\mathcal{L} = \mathcal{L}^{\text{CLIP}} + c_v \mathcal{L}^{\text{VF}} - c_e H[\pi_{\theta}] \quad (3.48)$$

where $\mathcal{L}^{\text{VF}} = (V_{\theta}(s_t) - R_t)^2$ is the value function loss, $H[\pi_{\theta}]$ is the policy entropy, $c_v = 0.5$ is the value loss coefficient, and $c_e = 0.01$ is the entropy coefficient.

A single Adam optimizer is used with parameter-group-specific learning rates:

- Shared feature extractor: default learning rate

- Actor head: 3×10^{-4}
- Critic head: 1×10^{-3}

Hyperparameters Table 3.5 summarizes the hyperparameters used for PPO training.

Table 3.5: PPO hyperparameters

Parameter	Value
<i>Learning</i>	
Actor learning rate	3×10^{-4}
Critic learning rate	1×10^{-3}
Discount factor γ	0.99
GAE λ	0.95
Max gradient norm	0.5
<i>PPO-Specific</i>	
Clip ratio ε	0.2
Entropy coefficient c_e	0.01
Value loss coefficient c_v	0.5
<i>Batching</i>	
Number of parallel environments	8
Update interval (total steps)	1000
Batch size	256
Epochs per update	10
<i>Architecture</i>	
Hidden layer size	500
Number of hidden layers	2
Activation function	Tanh
Action std init σ_0 (continuous)	0.1

Action Spaces Tested PPO is the only algorithm evaluated on all three action space formulations, enabled by a factory pattern that instantiates the appropriate agent variant based on the `action_space_type` configuration parameter:

- **Discrete:** Standard `PPOAgent` with categorical policy over 72 actions
- **Continuous/Hybrid:** `HybridPPOAgent` combining discrete spoke selection with continuous turn magnitude
- **All-spokes:** Standard `PPOAgent` with 36-dimensional Gaussian policy

This flexibility makes PPO the only algorithm that enables direct comparison across all action formulations.

3.2.3 TD-MPC2

Implementation Basis The TD-MPC2 implementation is based on the JAX/Flax re-implementation by Flandermeyer³, which itself follows the original TD-MPC2 algorithm [8]. The implementation was left largely unchanged, with only minor modifications to support configurable network sizes for the encoder and world model components.

Architecture Overview TD-MPC2 jointly learns five neural network components that together form a world model for latent-space planning:

- **Encoder** h_θ : Maps observations to latent states $z = h_\theta(s)$
- **Dynamics model** d_θ : Predicts next latent state $z' = d_\theta(z, a)$
- **Reward model** r_θ : Predicts reward $\hat{r} = r_\theta(z, a)$
- **Policy model** π_θ : Outputs action distribution $\pi_\theta(a|z)$
- **Value model** Q_θ : Estimates action-value $Q_\theta(z, a)$

All networks use normalized linear layers (**NormedLinear**) with layer normalization and Mish activation [9]:

$$\text{Mish}(x) = x \cdot \tanh(\ln(1 + e^x)) \quad (3.49)$$

The latent space uses SimNorm [8] as the final activation, which projects representations onto a simplex:

$$\text{SimNorm}(x) = \text{softmax}(\text{reshape}(x, [L, V])) \quad (3.50)$$

where $L = \text{latent_dim}/V$ and $V = 8$ is the simplex dimension.

Network Architectures Table 3.6 details the architecture of each component.

Table 3.6: TD-MPC2 network architectures. All hidden layers use Mish activation with layer normalization.

Component	Architecture	Output
Encoder	$s \rightarrow 256 \rightarrow 512$ (SimNorm)	Latent state z
Dynamics	$(z, a) \rightarrow 512 \rightarrow 512 \rightarrow 512$ (SimNorm)	Next latent z'
Reward	$(z, a) \rightarrow 512 \rightarrow 512 \rightarrow 101$	Reward bins
Policy	$z \rightarrow 512 \rightarrow 512 \rightarrow 2 \cdot A $	Mean, log-std
Value	$(z, a) \rightarrow 512 \rightarrow 512 \rightarrow 101$	Q-value bins

³<https://github.com/ShaneFlandermeyer/tdmpc2-jax>

Distributional Value and Reward Prediction Following TD-MPC2, both reward and value predictions use a distributional representation with two-hot encoding. Values are discretized into 101 bins spanning $[-10, 10]$ in symlog space:

$$\text{symlog}(x) = \text{sign}(x) \cdot \ln(1 + |x|) \quad (3.51)$$

The two-hot encoding provides soft targets for regression, and predictions are recovered via:

$$\hat{v} = \text{symexp} \left(\sum_{i=1}^{101} \text{softmax}(\text{logits})_i \cdot b_i \right) \quad (3.52)$$

where b_i are the bin centers and $\text{symexp}(x) = \text{sign}(x)(e^{|x|} - 1)$.

Value Ensemble The value function uses an ensemble of $N_Q = 5$ networks with dropout ($p = 0.01$) for uncertainty estimation. During policy optimization, the mean across ensemble members is used; during TD-target computation, two randomly sampled members are used with minimum aggregation to reduce overestimation.

Planning with MPPI At inference time, actions are selected via Model Predictive Path Integral (MPPI) control in latent space. The planning procedure:

1. Encode current observation: $z_0 = h_\theta(s)$
2. Initialize action sequences: 24 from policy rollouts, 488 from Gaussian noise
3. For each MPPI iteration (6 total):
 - (a) Evaluate all trajectories using learned dynamics and rewards
 - (b) Select top 64 elites based on estimated returns
 - (c) Update mean and std via softmax-weighted aggregation
4. Sample final action from elite distribution using Gumbel-softmax

The return estimate for a trajectory combines imagined rewards with a terminal value:

$$G = \sum_{t=0}^{H-1} \gamma^t \hat{r}_t + \gamma^H Q_\theta(z_H, \pi_\theta(z_H)) \quad (3.53)$$

Training Objective The world model is trained to minimize a weighted combination of losses:

$$\mathcal{L} = \lambda_c \mathcal{L}_{\text{consistency}} + \lambda_r \mathcal{L}_{\text{reward}} + \lambda_v \mathcal{L}_{\text{value}} \quad (3.54)$$

where:

- $\mathcal{L}_{\text{consistency}}$: MSE between predicted and encoded next latent states

- $\mathcal{L}_{\text{reward}}$: Soft cross-entropy for reward prediction
- $\mathcal{L}_{\text{value}}$: Soft cross-entropy for Q-value prediction with TD targets

The policy is trained separately to maximize Q-values with entropy regularization:

$$\mathcal{L}_{\pi} = \mathbb{E}_{z,a \sim \pi} [\alpha \log \pi(a|z) - Q(z, a)] \quad (3.55)$$

Hyperparameters Tables 3.7 and 3.8 summarize the hyperparameters.

Table 3.7: TD-MPC2 planning hyperparameters

Parameter	Value
Planning horizon H	3
MPPI iterations	6
Population size	512
Policy prior samples	24
Number of elites	64
Minimum plan std	0.05
Maximum plan std	2.0
Temperature	1.0

Table 3.8: TD-MPC2 training hyperparameters

Parameter	Value
<i>Architecture</i>	
Latent dimension	512
MLP hidden dimension	512
Encoder hidden dimension	256
Number of encoder layers	2
SimNorm dimension V	8
Number of value networks N_Q	5
Value dropout	0.01
Number of bins	101
Symlog range	$[-10, 10]$
<i>Optimization</i>	
Learning rate (all networks)	3×10^{-4}
Batch size	256
Discount factor γ	0.99
Horizon decay ρ	0.5
Max gradient norm	20
Target network τ	0.01
<i>Loss Coefficients</i>	
Consistency λ_c	20
Reward λ_r	0.1
Value λ_v	0.1
Entropy α	10^{-4}
<i>Data Collection</i>	
Number of parallel environments	7
Replay buffer size	300,000
Seed steps	$\max(5 \cdot 250, 1000) \cdot 7 \cdot 1.0$
Update-to-data ratio	1.0

3.2.4 Common Training Configuration

To ensure comparability, all algorithms share the following training configuration:

Table 3.9: Shared training configuration across all algorithms

Parameter	Value
Total environment steps	1×10^6
Maximum episode length	41 steps
Training runs per configuration	1

Logging Training progress is logged using TensorBoard. The following metrics are recorded for every episode:

- Episodic return
- Final state norm (rim displacement)
- Maximum rim displacement component
- Absolute sum of spoke tension deviations from 800
- Maximum spoke tension deviation from 800
- Episode termination type (success or failure)

3.3 Experimental Design

This chapter describes the experimental methodology used to analyze the influence of state representation, action formulation, reward design, and optimization strategy on reinforcement learning performance in the wheel truing task. The experiments are structured as a controlled ablation study, followed by an investigations into temporal state augmentation and lastly a targeted hyperparameter optimization on the best obtained configuration from the previous experiments.

The primary goal of the experimental design is not to obtain optimal absolute performance, but to systematically compare modeling and algorithmic to inform the choice of the final model for further use.

3.3.1 Ablation Study Design

The ablation study forms the core of the experimental analysis. Its purpose is to identify which components of the reinforcement learning formulation are most influential for learning a successful wheel truing policy. To this end, multiple design dimensions are varied systematically while all other aspects of the environment and training procedure are held constant.

Ablation dimensions

The ablation study systematically varies the following components of the reinforcement learning setup:

- **State-space representation:** Rim displacement representations (point-based and Fourier-based), spoke-tension-only representations, and combined representations including both geometric and tension information.
- **Action-space formulation:** Discrete spoke adjustments, continuous single-spoke actions, and continuous multi-spoke adjustments.

- **Reward function design:** Four reward formulations are tested: raw displacement penalty, normalized step-wise improvement, spoke adjustment magnitude penalty, and a combined signal integrating both displacement and tension deviation.
- **Reinforcement learning algorithm:** Value-based (DQN), policy-gradient (PPO), and model-based (TD-MPC2) approaches.

These dimensions were chosen to reflect both practical considerations in wheel truing (e.g., observability and control granularity) and common design choices in reinforcement learning. All experiments in the ablation study employ feedforward neural network policies without explicit recurrence. This restriction is intentional, as it allows the influence of state representation and reward design to be analyzed without the confounding effects of learned temporal memory. Temporal augmentation strategies are investigated separately in a later experimental phase.

Controlled comparison strategy

To isolate the influence of individual design choices, experiments are conducted under a controlled comparison protocol. Wherever possible, only a single ablation dimension is varied at a time, while all other aspects of the training setup are held fixed.

Specifically, episode length, termination conditions, wheel parameters, and training budgets are identical across all configurations. Each configuration is trained once under stochastic but fixed conditions, reflecting a single realization of the learning process. Due to computational constraints imposed by the available resources, stochastic variability across multiple training runs is not explicitly characterized. Instead, the experimental design prioritizes breadth of methodological comparison over statistical estimation of expected performance. Conclusions are therefore drawn based on relative performance trends and qualitative behavioral differences between configurations rather than precise quantitative comparisons.

3.4 Follow-up Experiments

Based on observations from the ablation study, three follow-up investigations were conducted: a reward weighting parameter sweep to optimize the geometry-tension trade-off in the "combined" reward function, an extended training study with temporal augmentation to investigate convergence behavior and the role of temporal context, and a hyperparameter optimization study to identify performance limits of the best-performing algorithm.

3.4.1 Reward Weighting Parameter Sweep

The ablation study revealed a trade-off between the raw and combined reward functions: while the raw reward achieved lower final state norms (geometric displacement), the combined reward produced states with smaller maximum spoke tension deviations. To investigate this trade-off systematically and identify an optimal balance, the combined reward was generalized to include a tunable weighting parameter $\alpha \in [0, 1]$:

$$r_t^{\text{weighted}} = - \left(\alpha \|s_t\| + (1 - \alpha) \frac{1}{n_s} \sum_{j=1}^{n_s} \frac{|T_j - T_0|}{400} \right) \quad (3.56)$$

where $\alpha = 1$ recovers a pure displacement penalty and $\alpha = 0$ penalizes only tension deviation. The original combined reward corresponds to $\alpha = 0.5$.

A parameter sweep was conducted over $\alpha \in \{0.0, 0.1, 0.2, \dots, 1.0\}$ using both the `rimpoints` and `rimandspokes` state representations with the Rainbow DQN and PPO algorithms. Due to the substantially higher computational cost of TD-MPC2, the sweep was not repeated for this algorithm; we assume the relationship between α and performance generalizes across algorithms given the shared reward structure.

Results of the sweep (Chapter 4) suggest $\alpha = 0.7$ provides the best balance between geometric accuracy and tension uniformity. This value is adopted for subsequent experiments.

3.4.2 Extended Training and Temporal Augmentation

Analysis of the ablation study learning curves revealed that several algorithm–configuration combinations had not plateaued at the 1×10^6 step budget, raising two questions: (1) what is the asymptotic performance of each algorithm under extended training, and (2) does providing temporal context improve learning, particularly given that the wheel truing environment may violate the Markov assumption due to the many-to-one mapping from spoke tensions to rim displacements.

To address both questions efficiently, extended training experiments to 5×10^6 steps were conducted as a unified study that simultaneously compared baseline feedforward policies with temporally augmented variants for the two best-performing algorithms from the ablation study: Rainbow DQN and PPO. Due to restrictions in available computational resources, additional algorithms were not included in this extended training and temporal augmentation analysis.

Temporal Augmentation Methods

In the standard formulation, the agent observes only the current rim displacement (and optionally spoke tensions). However, the optimal corrective action may depend on the

history of previous adjustments, particularly because the relationship between spoke tensions and rim displacement might be many-to-one: different tension distributions can yield similar displacement profiles. This suggests that the environment may be a Partially Observable Markov Decision Process (POMDP) rather than a fully observable MDP.

Two approaches for providing temporal context were evaluated: frame stacking for feedforward networks and recurrent neural network architectures.

Frame Stacking. Frame stacking augments the observation by concatenating the k most recent observations and the $k - 1$ most recent actions:

$$\tilde{s}_t = [s_{t-k+1}, s_{t-k+2}, \dots, s_t, \text{onehot}(a_{t-k+1}), \dots, \text{onehot}(a_{t-1})] \quad (3.57)$$

A wrapper class maintains a sliding window buffer, with zero-padding at episode boundaries until sufficient history accumulates. With the `rimpoints` state representation ($d_s = 720$), discrete action space ($|\mathcal{A}| = 72$), and $k = 4$ frames, the augmented observation dimension is:

$$\dim(\tilde{s}) = 720 \times 4 + 72 \times 3 = 3096 \quad (3.58)$$

Recurrent Architectures. Both Long Short-Term Memory (LSTM) [10] and Gated Recurrent Unit (GRU) [11] architectures were evaluated. For Rainbow DQN, the recurrent layer processes sequences of state-action pairs, with the final hidden state passed to the dueling value and advantage streams. The replay memory was modified to store and sample sequences of length $L = 4$ rather than individual transitions. For PPO, the recurrent layer is inserted between the observation input and the shared feature extractor, with hidden state persisting across timesteps within episodes.

Experimental Configuration

All extended training and temporal augmentation experiments used the best-performing configuration from the ablation study:

- State representation: `rimpoints` (720-dimensional rim displacement)
- Reward function: weighted combined with $\alpha = 0.7$
- Action space: discrete (72 actions)
- Training budget: 5×10^6 environment steps

For each algorithm (Rainbow DQN and PPO), four conditions were compared:

1. **Baseline:** Standard feedforward network (no temporal augmentation)
2. **Frame stacking:** Feedforward network with $k = 4$ stacked frames and action history
3. **LSTM:** Recurrent network with single-layer LSTM (256 hidden units)
4. **GRU:** Recurrent network with single-layer GRU (256 hidden units)

Table 3.10 summarizes the hyperparameters specific to temporal augmentation. All other hyperparameters matched those used in the ablation study.

Table 3.10: Temporal augmentation hyperparameters.

Parameter	Frame Stack	LSTM/GRU
Stack size / Sequence length	4	4
Include action history	Yes	Yes
Hidden layer size	512	512
Recurrent hidden size	—	256
Recurrent layers	—	1
Recurrent dropout	—	0.0

Methodological Note. The recurrent architectures were trained using hyperparameters originally tuned for feedforward networks. Recurrent networks may benefit from different learning rates, batch sizes, or sequence lengths. The results therefore represent lower bounds on what temporal augmentation methods might achieve with dedicated hyperparameter optimization. The goal of these experiments is to test whether temporal context provides *any* benefit for this task, rather than to optimize the temporal augmentation approach.

3.4.3 Hyperparameter Optimization

While the ablation study and extended training experiments establish relative algorithm performance, the true potential of the best-performing algorithm (Rainbow DQN) may be obscured by suboptimal hyperparameter choices. To assess its performance ceiling, a dedicated hyperparameter optimization (HPO) study was conducted.

Optimization Target

The optimization focused on Rainbow DQN in the configuration identified as optimal in the ablation study:

- Action space: discrete (72 actions)

- Reward function: weighted combined with $\alpha = 0.7$

The objective of each trial was a multi-objective evaluation over three metrics:

1. Final state norm (lower is better)
2. Maximum tension deviation (lower is better)
3. Success rate (higher is better)

For integration with Optunas minimization framework, success rate was negated so that all objectives could be minimized.

Search Space

Hyperparameters were selected based on known sensitivities of Rainbow DQN and considerations specific to the wheel truing task. The following parameters were optimized:

- **Learning rate** $\eta \in [10^{-5}, 10^{-3}]$ (log scale)
- **Batch size** $\in 32, 64, 128$
- **Multi-step returns** $n \in 1, 3, 5, 7$
- **Target network update period** $\in 2000, 4000, 8000, 16000$ steps
- **Noisy network standard deviation** $\sigma_0 \in [0.01, 0.5]$ (log scale)
- **Prioritization exponent** $\alpha \in [0.0, 1.0]$
- **Hidden layer size** $\in 256, 512, 1024$
- **Number of hidden layers** $\in 1, 2, 3$

All other hyperparameters, including the state representation, remained fixed.

Optimization Strategy

HPO was conducted using the Tree-structured Parzen Estimator (TPE) sampler implemented in the Optuna framework, with the multivariate option enabled to capture correlations between hyperparameters.

To improve search efficiency, the optimization proceeded in a two-phase approach:

1. **Broad exploration:** 50 trials sampled broadly across the search space to identify promising regions.

2. **Focused refinement:** 50 trials concentrated on regions suggested by the first phase. All completed trials from the exploration phase were incorporated into the refinement study, allowing the sampler to leverage prior observations.

Each trial evaluated the agent over a maximum of 10 million environment steps, with early stopping at 500,000 steps if performance was poor or if a 90% success rate was already reached.

Implementation Details

To ensure fair comparison across trials, the replay buffer initialization procedure was modified. In previous experiments, each agent’s buffer was bootstrapped by collecting 20,000 transitions using greedy actions from the randomly initialized network. For hyperparameter optimization, a shared buffer of 20,000 transitions was instead pre-collected using a uniform random policy and used to initialize every trial’s replay buffer identically. This reduces inter-trial variance attributable to different initial experiences, improving the signal-to-noise ratio when comparing hyperparameter configurations, though it introduces a minor bias from the purely exploratory nature of the seeding data. Trials were trained in parallel across multiple workers utilizing both GPU and CPU resources. All trial configurations and results were logged in JSON format and TensorBoard to facilitate post-hoc analysis. The best-performing model across all trials was automatically tracked and saved, replacing previous bests whenever a superior configuration was found.

Evaluation Metric

For each trial, final performance was measured as the mean final state norm, final maximum tension deviation, and success rate over the last 100 episodes.

3.5 Methodological Limitations

Several methodological limitations should be acknowledged when interpreting the experimental results.

Single Training Runs. Due to computational constraints, each configuration in the ablation study and follow-up experiments was trained once. The results provide point estimates rather than statistical comparisons across multiple random seeds. Conclusions are therefore drawn based on relative performance trends and qualitative behavioral differences rather than statistically significant differences. Configurations showing small performance gaps should be interpreted with caution.

Hyperparameter Transfer. The recurrent architectures in the temporal augmentation experiments were trained using hyperparameters originally tuned for feedforward networks. Recurrent networks may benefit from different learning rates, batch sizes, or sequence lengths. Similarly, PPO and TD-MPC2 used literature-recommended hyperparameters rather than task-specific tuning. The ablation study results therefore represent lower bounds on what each algorithm might achieve with dedicated optimization.

Quasi-Static Simulation. The wheel physics model assumes quasi-static equilibrium: spoke adjustments instantaneously propagate to a new equilibrium state without transient dynamics. Real wheel truing involves time-dependent effects including spoke wind-up, nipple friction, and gradual tension redistribution. The learned policies may not transfer directly to physical wheels without adaptation.

These limitations suggest that the experimental results should be interpreted as a comparative study of reinforcement learning design choices in a controlled simulation environment, rather than as a direct evaluation of approaches for practical wheel truing automation.

Chapter 4

Results and Analysis

This chapter presents the experimental results of the ablation study and follow-up experiments. Section 4.1 establishes reference points for interpreting performance. Section 4.2 analyzes the main ablation study across algorithms, action spaces, state representations, and reward functions. Section 4.3 presents the reward weighting parameter sweep. Section 4.4 examines convergence behavior under extended training and reports on temporal augmentation experiments. Finally, Section 4.6 synthesizes findings and addresses the remaining questions.

4.1 Baseline Metrics and Reference Points

4.1.1 Initial State Distribution

The initial wheel configurations, generated by randomly detuning five spokes by up to ± 1 turn from a perfectly true wheel, establish the baseline against which all learned policies are evaluated. Figure 4.1 shows the distribution of initial state norms across training episodes, with mean 0.702 mm and standard deviation 0.226 mm. Figure 4.2 displays the corresponding distribution of maximum spoke tension deviations, with mean 171.99 N and standard deviation 43.07 N.

These statistics provide two critical reference points: any policy achieving final state norms above approximately 0.7 mm has failed to improve upon the initial configuration, and the initial tension deviations establish a baseline for evaluating mechanical integrity. The average optimal path length from these initial configurations is approximately 25 adjustment steps, suggesting that the 41-step episode horizon provides roughly 60% headroom for exploration.

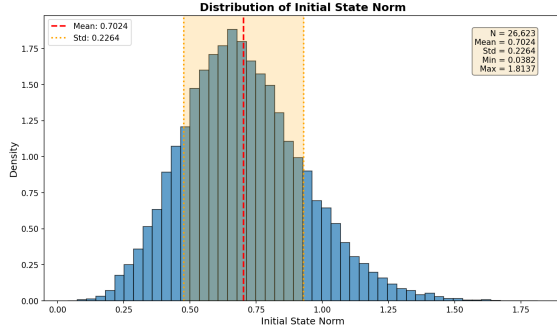


Figure 4.1: Distribution of initial state norm (integrated rim displacement) across training episodes. The mean of 0.702 mm establishes the baseline against which learned policies are evaluated.

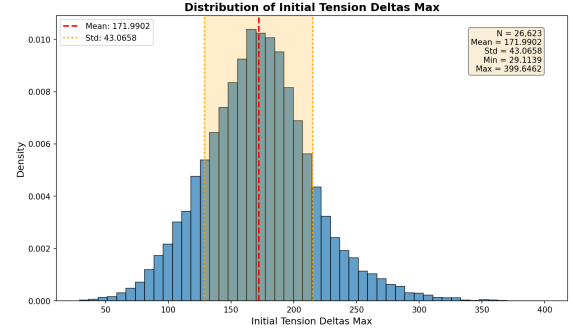


Figure 4.2: Distribution of initial maximum tension deviation from the nominal 800 N. The mean of 171.99 N provides context for evaluating tension uniformity.

4.1.2 Interpretation Framework

The success condition, requiring all displacement components below 0.3 mm, tensions between 700–900 N, and tension spread under 10%, was never achieved by any configuration. All episodes terminated by timeout at step 41. This universal failure to achieve the “golden wheel” standard indicates either that the task is fundamentally more difficult than the 41-step budget permits, or that no policy learned sufficiently precise control to satisfy all constraints simultaneously.

4.2 Ablation Study Results

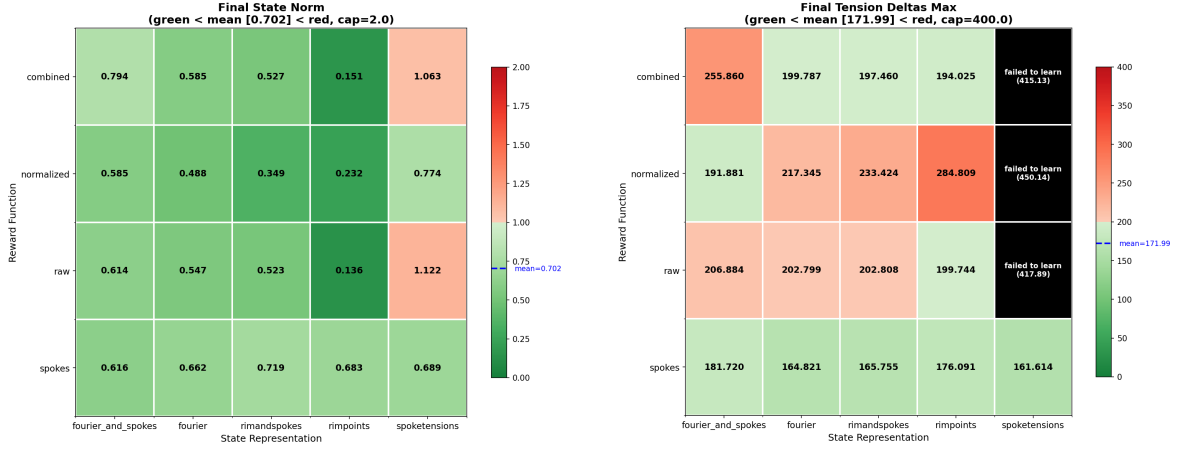
4.2.1 Algorithm Comparison

The ablation study reveals a clear performance hierarchy among the three algorithmic paradigms. Table 4.1 summarizes the best-performing configuration for each algorithm.

Table 4.1: Best-performing configuration for each algorithm, evaluated by final state norm (mean of last 100 training episodes). Improvement percentage is calculated relative to the initial mean of 0.702 mm.

Algorithm	Action Space	Best Config	Final Norm	Tension Max	Improv.
Rainbow DQN	Discrete	raw, rimpoints	0.136 mm	199.7 N	+80.6%
PPO	Discrete	raw, rimpoints	0.220 mm	290.4 N	+68.6%
TD-MPC2	Continuous	raw, rimandspokes	0.728 mm	211.1 N	−3.8%

Value-Based Methods (Rainbow DQN). Rainbow DQN achieved the best overall performance, with the optimal configuration (raw reward, rimpoints state) reaching a final state norm of 0.136 mm, representing an 80.6% reduction from the initial mean. Figure 4.3 displays the performance matrix across all reward-state combinations.

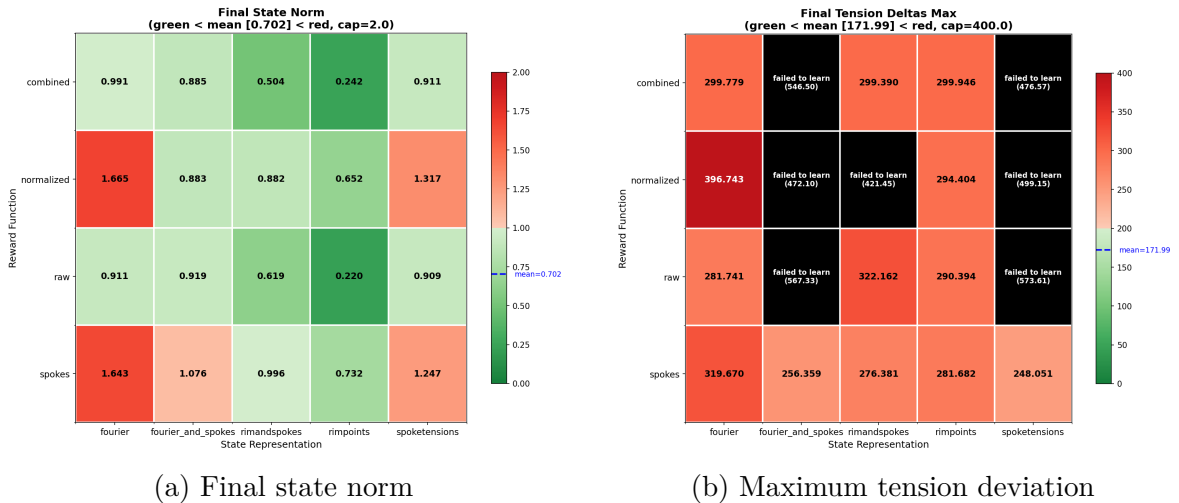


(a) Final state norm across configurations. Green indicates performance better than the initial mean (0.702 mm); red indicates worse.

(b) Maximum tension deviation. The initial mean of 171.99 N serves as reference.

Figure 4.3: Rainbow DQN ablation results. The rimpoints state representation with raw or combined reward achieves the best geometric accuracy across all tested configurations.

Policy Gradient Methods (PPO). PPO with discrete actions achieved the second-best result (0.220 mm, 68.6% improvement), approximately $1.6\times$ worse than DQN despite operating in the identical action space. Figure 4.4 shows the discrete action space results. Of 20 comparable configurations, DQN outperforms PPO in 17, with the three exceptions occurring with the spoketensions-only state representation where both algorithms perform poorly. This systematic advantage likely reflects sample efficiency differences: DQN reuses each transition multiple times through experience replay, while PPO discards data after each update.



(a) Final state norm

(b) Maximum tension deviation

Figure 4.4: PPO with discrete action space. Performance is systematically worse than DQN but follows similar patterns across state representations and reward functions.

Model-Based Methods (TD-MPC2). TD-MPC2 with the continuous action space achieved final state norms around 0.73-0.75 mm, approximately at the initial mean, indicating marginal learning at best (Figure 4.5). Notably, TD-MPC2 shows less catastrophic failure than PPO in the all-spokes action space (Section 4.2.2), suggesting the planning component provides some robustness to high-dimensional continuous control.

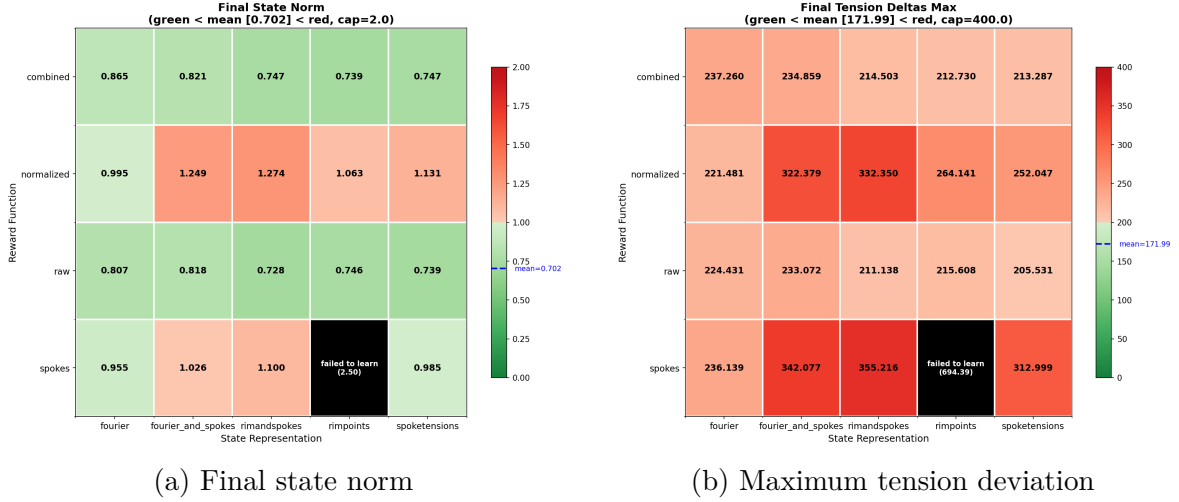


Figure 4.5: TD-MPC2 with continuous action space. Performance hovers near the initial state mean, indicating minimal effective learning within the training budget.

4.2.2 Action Space Analysis

The Dominance of Discrete Actions. Discrete action formulation outperforms continuous alternatives by wide margins. Table 4.2 quantifies this gap for PPO with the rimpoints state and combined reward.

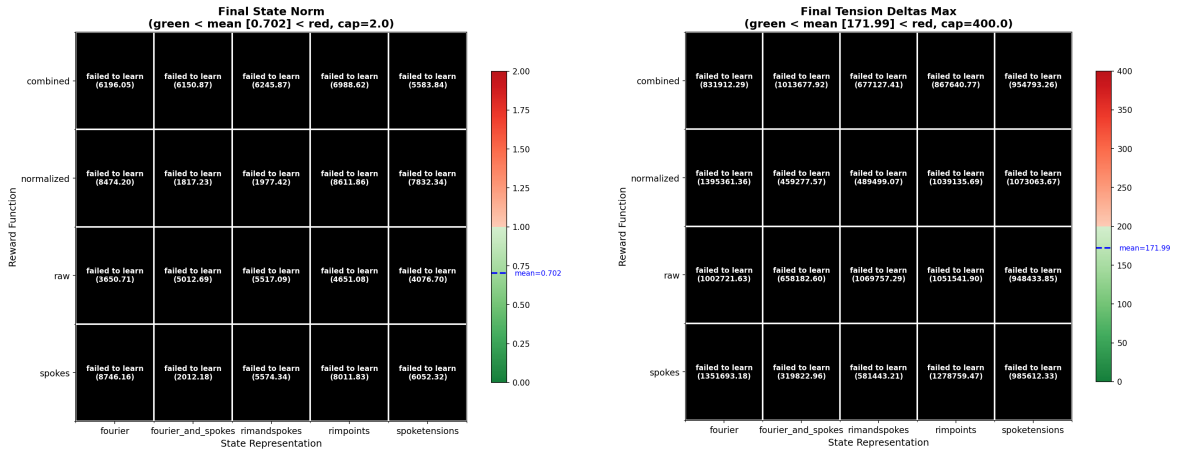
Table 4.2: Action space comparison for PPO (rimpoints state, combined reward). The discrete formulation provides dramatic advantages over continuous alternatives.

Action Space	Final State Norm	Relative to Discrete
Discrete (72 actions)	0.242 mm	1.0×
Continuous (2D)	1.023 mm	4.2× worse
All-Spokes (36D)	6989 mm	28,824× worse

This hierarchy suggests that discrete actions provide an implicit curriculum: the agent learns which spoke to adjust before learning fine-grained control. The continuous formulation removes this structure, while the all-spokes formulation demands coordination across 36 dimensions simultaneously.

The All-Spokes Divergence Phenomenon. PPO with all-spokes actions produces final state norms in the thousands of millimeters, approximately four orders of magnitude worse than discrete PPO (Figure 4.6). Several mechanisms may explain this divergence:

1. **Credit assignment explosion:** When all 36 spokes are adjusted simultaneously, the reward signal provides no information about which adjustments contributed positively or negatively.
2. **Exploration collapse:** With 36 continuous dimensions and entropy coefficient 0.01, the policy may converge to a narrow region before discovering effective strategies.
3. **PPO clipping constraints:** Probability ratio clipping may be overly conservative in high dimensions, preventing large policy updates needed to escape poor local optima.



(a) Final state norm. Values in thousands indicate complete failure.

(b) Maximum tension deviation

Figure 4.6: PPO with all-spokes action space. All configurations show catastrophic divergence, with final state norms exceeding initial values by factors of 10^3 – 10^4 .

TD-MPC2 with all-spokes shows substantially less divergence, with final norms of 1-50 mm rather than thousands (Figure 4.7). This suggests that model-based planning provides robustness: the MPPI procedure may find reasonable actions even when the learned policy is poor.

4.2.3 State Representation Analysis

Perhaps the most surprising finding is that adding spoke tensions to the state representation consistently degrades performance. Table 4.3 quantifies this effect for DQN with combined reward.

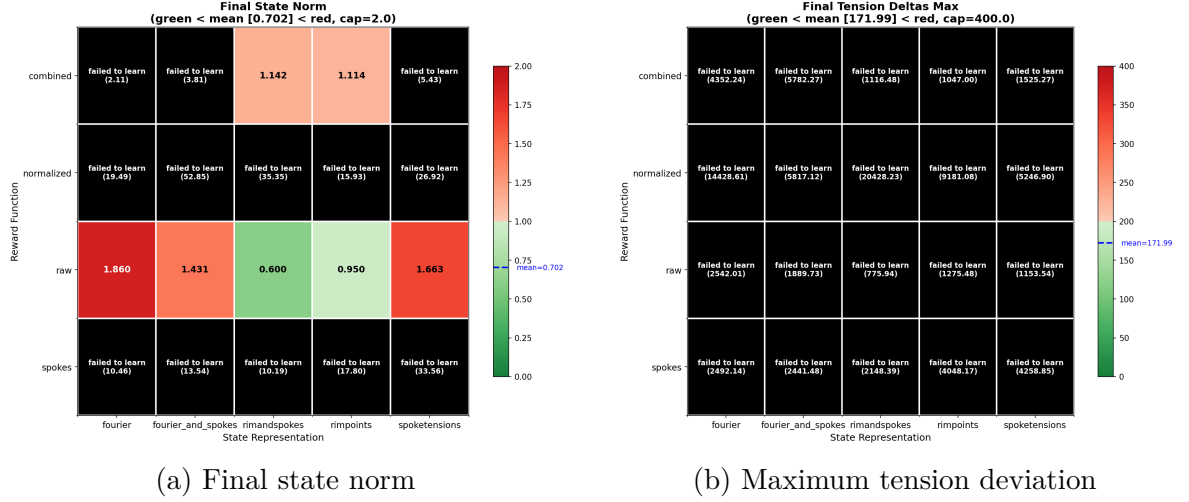


Figure 4.7: TD-MPC2 with all-spokes action space. While performance is poor, the divergence is far less catastrophic than PPO.

Table 4.3: State representation comparison for DQN (combined reward). Adding spoke tensions to the observation consistently degrades performance despite providing strictly more information.

State Representation	Dimension	Final Norm	Effect
rimpoints	720	0.151 mm	Baseline
rimandspokes	756	0.527 mm	+249% worse
fourier	122	0.585 mm	+287% worse
fourier_and_spokes	158	0.794 mm	+425% worse
spoketensions	36	1.063 mm	+603% worse

This pattern contradicts naive intuitions about information sufficiency. We hypothesize two contributing factors. First, spoke tensions and rim displacements are physically coupled through the stiffness equations (Section 3.1.1), so providing both introduces redundant, correlated features that the network must learn to appropriately weight or ignore. Second, the higher-dimensional input spaces may require more training data than the 10^6 step budget provides.

The Fourier representation (122D) also underperforms raw points (720D), despite capturing $> 99.999\%$ of the signal variance. This may reflect the loss of spatial locality: raw displacement at each rim position maps directly to spoke locations, while Fourier coefficients distribute this information across frequency components, complicating credit assignment.

4.2.4 Reward Function Analysis

The reward functions span a spectrum from dense (raw, combined) to sparse (normalized, spokes). As expected, denser signals facilitate faster and more reliable learning. Table 4.4 summarizes performance with the rimpoints state representation.

Table 4.4: Reward function comparison (rimpoints state, discrete actions). Dense reward signals (raw, combined) substantially outperform sparse alternatives.

Reward	DQN Norm	DQN Tension	PPO Norm	PPO Tension
raw	0.136 mm	199.7 N	0.220 mm	290.4 N
combined	0.151 mm	194.0 N	0.242 mm	299.4 N
normalized	0.232 mm	284.8 N	0.652 mm	294.4 N
spokes	0.683 mm	176.1 N	0.732 mm	281.7 N

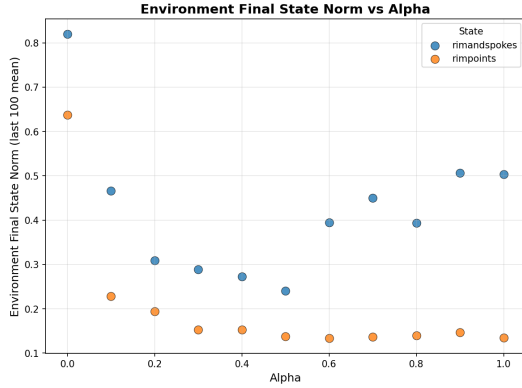
The **spokes** reward, which provides signal only about adjustment magnitude rather than geometric effect, essentially fails to improve upon the starting state. Comparing **raw** and **combined** rewards reveals a subtle trade-off: raw achieves the best geometric accuracy (0.136 mm) but combined produces marginally better tension uniformity (194.0 N vs 199.7 N) at the cost of slightly worse geometry (0.151 mm).

4.3 Alpha Parameter Sweep

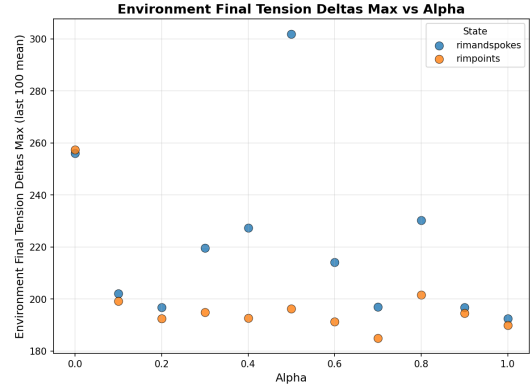
The generalized combined reward $r = -(\alpha \cdot \|\text{rim}\| + (1 - \alpha) \cdot \text{tension_deviation})$ was swept over $\alpha \in \{0.0, 0.1, \dots, 1.0\}$ with both DQN and PPO using the rimpoints and rimandspokes state representations.

Figure 4.8 displays the results for DQN. Performance is relatively flat across a wide range of α values, with $\alpha \in [0.6, 0.8]$ showing slightly lower final state norms. At $\alpha = 0.0$

(pure tension penalty), geometry degrades as expected since the reward provides no direct signal about rim displacement.



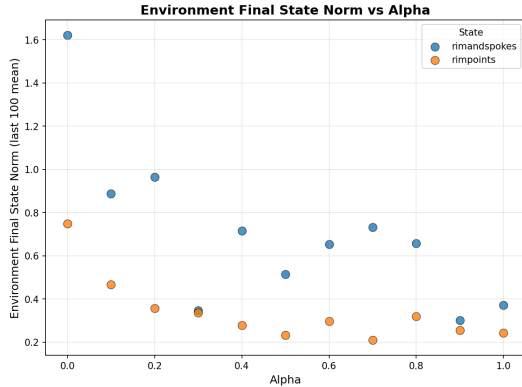
(a) Final state norm (mean of last 100 episodes)



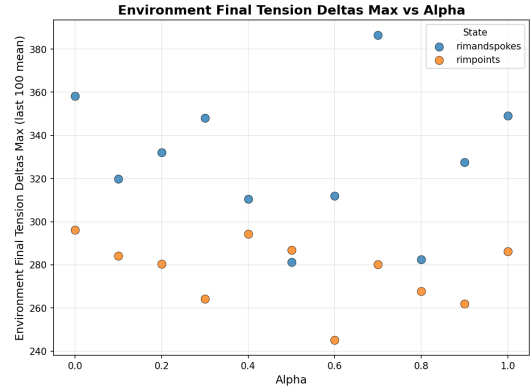
(b) Maximum tension deviation from 800 N

Figure 4.8: Alpha parameter sweep for DQN. The rimpoints representation (orange) consistently outperforms rimandspokes (blue) across all α values.

Figure 4.9 shows corresponding results for PPO, which follow a qualitatively similar but noisier pattern.



(a) Final state norm (mean of last 100 episodes)



(b) Maximum tension deviation from 800 N

Figure 4.9: Alpha parameter sweep for PPO. Performance is noisier and generally worse than DQN, but the optimal α region is similar.

Based on these results, $\alpha = 0.7$ was selected for subsequent experiments. This value achieves final state norms approximately 15-20% lower than $\alpha = 1.0$ while providing some tension regularization, consistent with practical wheel truing where geometry is the primary objective but extreme tension imbalance indicates structural problems.

4.4 Extended Training and Temporal Augmentation

The ablation study at 10^6 steps revealed that several configurations had not yet plateaued, raising questions about asymptotic performance. Additionally, if the environment is non-Markovian with respect to rim displacements alone, temporal context should improve performance. To address both questions, extended training experiments to 5×10^6 steps were conducted, comparing baseline feedforward policies with temporally augmented variants (frame stacking with $k = 4$, LSTM, and GRU) for DQN and PPO.

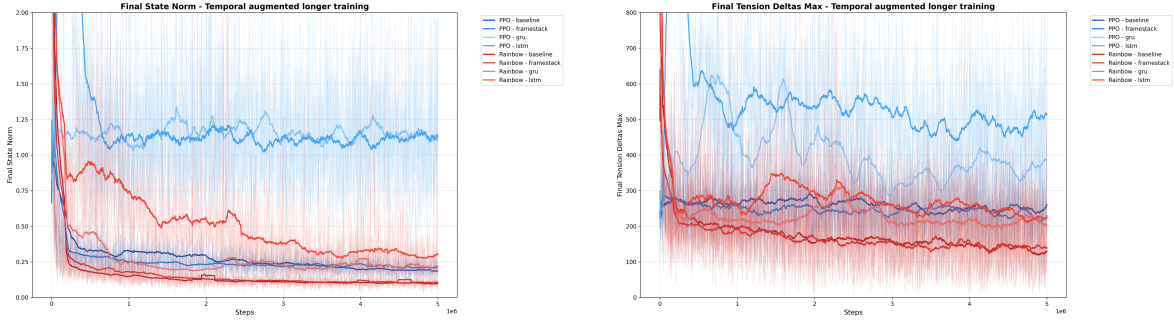


Figure 4.10: Extended training and temporal augmentation results over 5×10^6 steps. Final state norm (left) and maximum tension delta (right) across PPO (blue) and Rainbow (red) with different temporal processing methods: baseline (darkest), frame stacking, LSTM, and GRU (lightest). Raw data points shown with 20% transparency, overlaid with 100-step moving averages. Neither temporal augmentation method improves over the feedforward baseline.

4.4.1 Convergence Behavior

The two algorithms exhibit markedly different convergence profiles. DQN continues learning throughout the 5×10^6 step period without a clear plateau, suggesting that with sufficient budget it may achieve substantially better performance. PPO plateaus before 2×10^6 steps with no meaningful improvement thereafter, confirming that the performance gap reflects fundamental algorithmic differences rather than insufficient training.

4.4.2 Temporal Augmentation Results

Neither frame stacking nor recurrent architectures (LSTM, GRU) improve upon the feedforward baseline for either algorithm. The rimandspokes representation also continues to underperform rimpoints at 5×10^6 steps, though there is some indication of narrowing.

4.4.3 Implications for Effective Markovianity

The failure of both temporal augmentation and direct tension observation to improve performance provides evidence that the wheel truing environment is effectively Markovian with respect to rim displacements. Despite the theoretical many-to-one mapping from spoke tensions to displacements, the current rim profile appears sufficient for near-optimal action selection. Two interpretations are consistent with this finding: either the rim displacement profile contains enough information to discriminate between tension configurations requiring different actions (the degenerate cases being rare), or the current policies operate in a regime where Markovian approximation is adequate and benefits would only emerge with substantially better base performance. The consistency of this null result across multiple methods suggests the first interpretation is more likely.

4.5 Synthesis

4.5.1 Explaining Algorithm Performance Differences

DQN vs. PPO in Discrete Action Spaces. The consistent $1.6\times$ performance gap between DQN and PPO in the discrete action space likely reflects sample efficiency differences. Off-policy learning with prioritized experience replay allows DQN to extract more learning signal from the same number of environment interactions. PPO’s on-policy constraint discards most of the data generated during training: each transition is used only once before being discarded, while DQN may replay important transitions dozens of times.

Continuous Action Space Failures. The 2D continuous formulation conflates spoke selection (inherently discrete) with adjustment magnitude (continuous), requiring the policy to simultaneously learn two qualitatively different mappings. The implicit structure of discrete actions, separating “where” from “how much”, appears essential for efficient learning. PPO’s continuous action space achieves final norms of approximately 1 mm, representing minimal improvement from the initial distribution.

All-Spokes Divergence. The 36-dimensional action space presents an extreme credit assignment challenge. Combined with insufficient exploration (entropy coefficient 0.01 is likely too low for 36D) and potentially overly conservative PPO clipping, the policy converges to ineffective regions of action space before discovering coordinated adjustment strategies. The fact that TD-MPC2 shows less divergence suggests that planning-based action selection provides robustness that pure policy learning lacks.

We hypothesize the following failure cascade for PPO with all-spokes actions: (1) the policy initially outputs near-zero actions due to safe initialization; (2) random

perturbations occasionally improve the wheel, creating gradient signal toward those action patterns; (3) with entropy coefficient 0.01, the policy converges to a narrow action distribution before exploring broadly; (4) the converged policy makes coordinated but ineffective adjustments; (5) small errors compound over the 41-step episode, causing the wheel state to diverge catastrophically.

4.5.2 Explaining the State Representation Paradox

The spoke tensions are not independent of rim displacements—they are physically coupled through the wheel mechanics described in Section 3.1.1. This coupling creates redundant, correlated features that complicate optimization without providing information unavailable in the displacement signal alone. The higher-dimensional input spaces require more training data to learn effective representations, and the 10^6 step budget may be insufficient.

The extended training experiments provide partial evidence for this hypothesis: the performance gap between rimpoints and rimandspokes persists at 5×10^6 steps, but a narrowing trend may be present. Definitive testing would require training to convergence, which was computationally infeasible.

4.5.3 Best-Performing Configuration

The optimal configuration identified through ablation is:

- **Algorithm:** Rainbow DQN with IQN
- **Action space:** Discrete (72 actions)
- **State representation:** rimpoints (720D)
- **Reward function:** raw or combined with $\alpha = 0.7$

This configuration achieves final state norms of approximately 0.14–0.15 mm, representing an 80% reduction from initial configurations, though still short of the “golden wheel” success criterion.

4.6 Hyperparameter Optimization

Chapter 5

Discussion and Conclusion

Bibliography

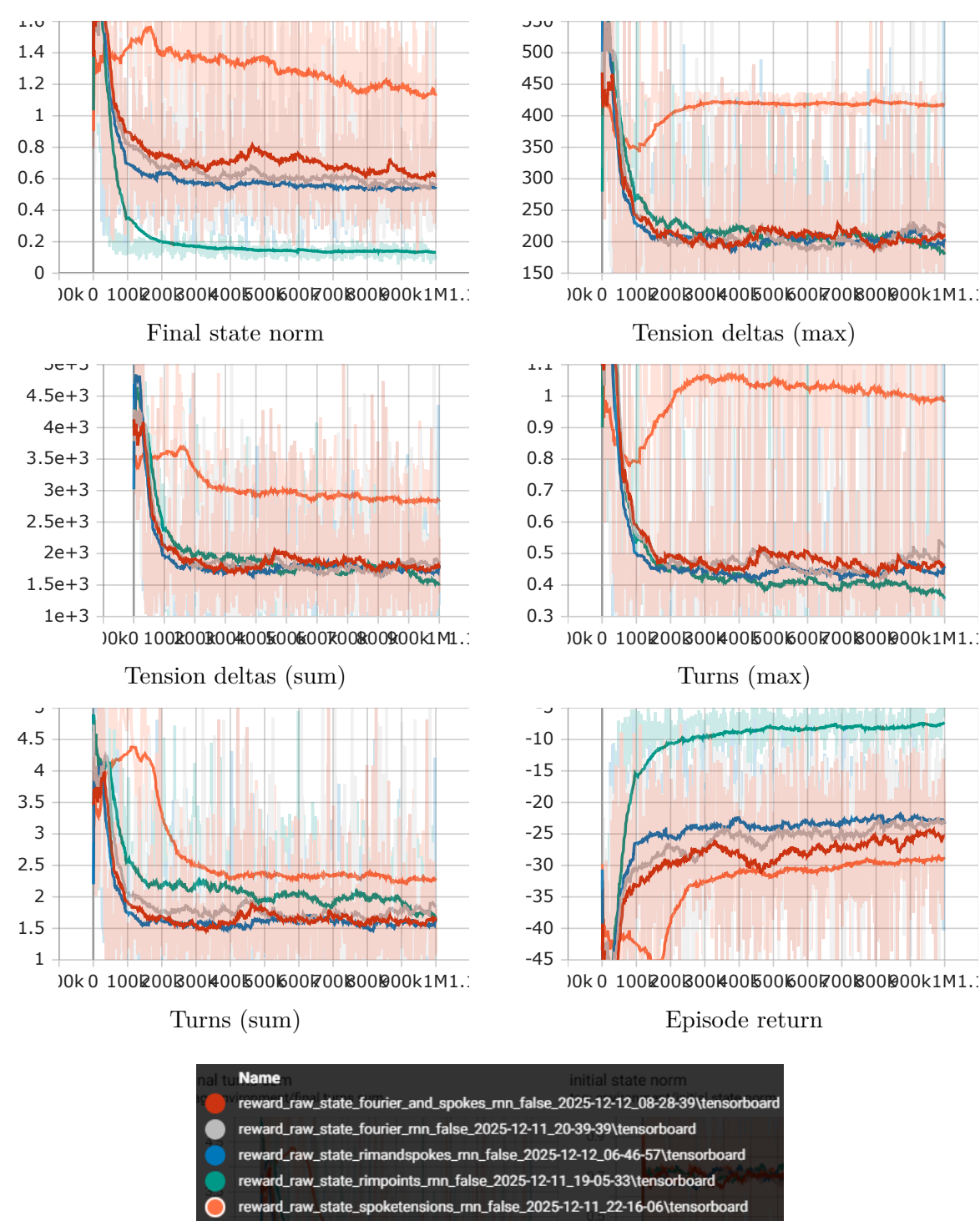
- [1] Matthew Ford. “Reinventing the Wheel: Stress Analysis, Stability, and Optimization of the Bicycle Wheel”. In: 2018. URL: <https://api.semanticscholar.org/CorpusID:216797463>.
- [2] Will Dabney et al. “Implicit Quantile Networks for Distributional Reinforcement Learning”. In: *International Conference on Machine Learning*. PMLR. 2018, pp. 1096–1105.
- [3] Hado Van Hasselt, Arthur Guez, and David Silver. “Deep Reinforcement Learning with Double Q-Learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 30. 1. 2016.
- [4] Ziyu Wang et al. “Dueling Network Architectures for Deep Reinforcement Learning”. In: *International Conference on Machine Learning*. PMLR. 2016, pp. 1995–2003.
- [5] Tom Schaul et al. “Prioritized Experience Replay”. In: *arXiv preprint arXiv:1511.05952* (2015).
- [6] Meire Fortunato et al. “Noisy Networks for Exploration”. In: *International Conference on Learning Representations*. 2018.
- [7] John Schulman et al. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *arXiv preprint arXiv:1506.02438* (2015).
- [8] Nicklas Hansen, Hao Su, and Xiaolong Wang. “TD-MPC2: Scalable, Robust World Models for Continuous Control”. In: *International Conference on Learning Representations* (2024).
- [9] Diganta Misra. “Mish: A Self Regularized Non-Monotonic Activation Function”. In: *arXiv preprint arXiv:1908.08681* (2019).
- [10] Sepp Hochreiter and Jürgen Schmidhuber. “Long short-term memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [11] Kyunghyun Cho et al. “Learning phrase representations using RNN encoder-decoder for statistical machine translation”. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1724–1734.

- [12] James Bergstra et al. “Algorithms for hyper-parameter optimization”. In: *Advances in Neural Information Processing Systems*. Vol. 24. 2011.
- [13] Takuya Akiba et al. “Optuna: A next-generation hyperparameter optimization framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019, pp. 2623–2631.

Appendix

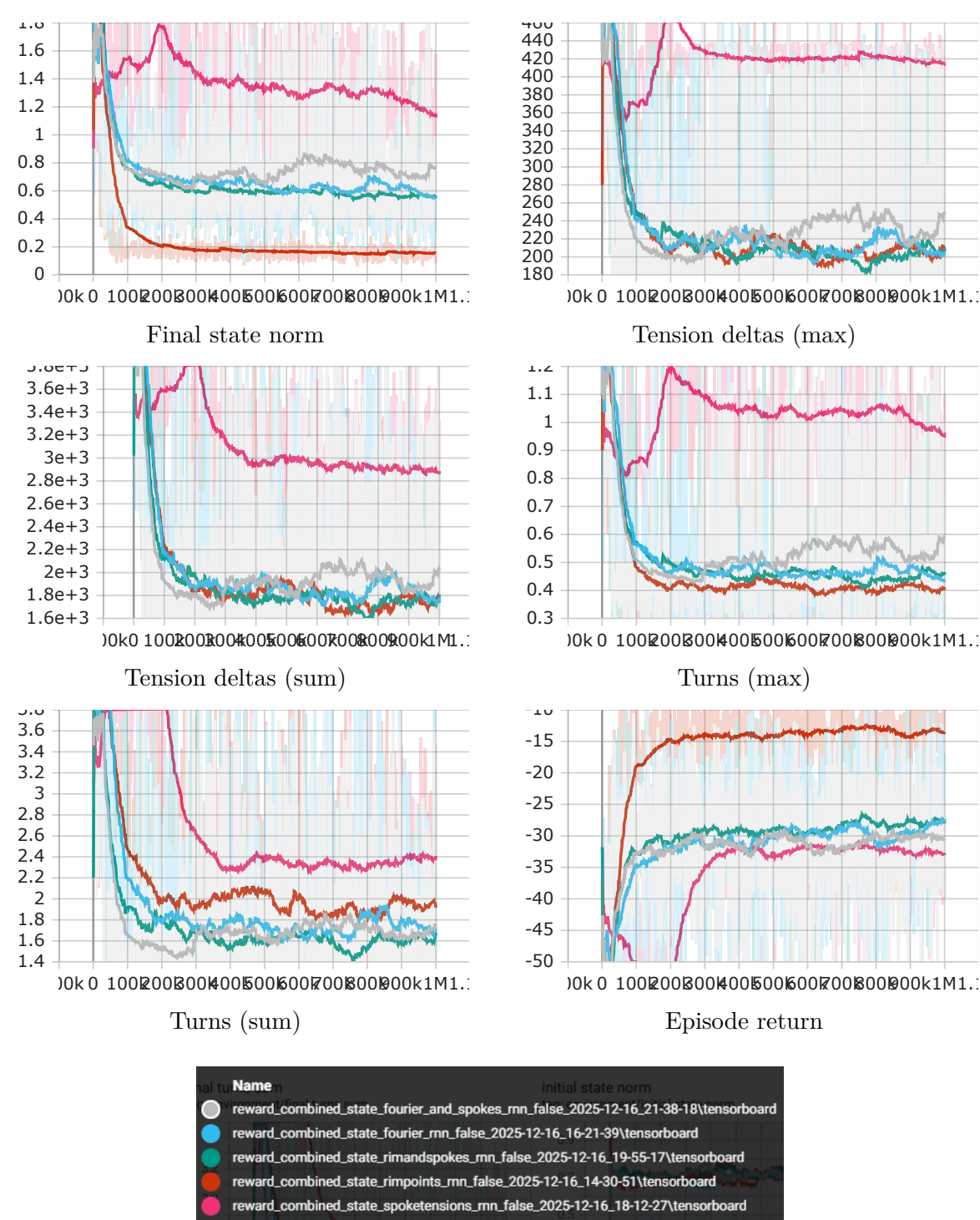
TODO:

- (low prio) redo the plots
- plots need explanation on how moving average is calculated



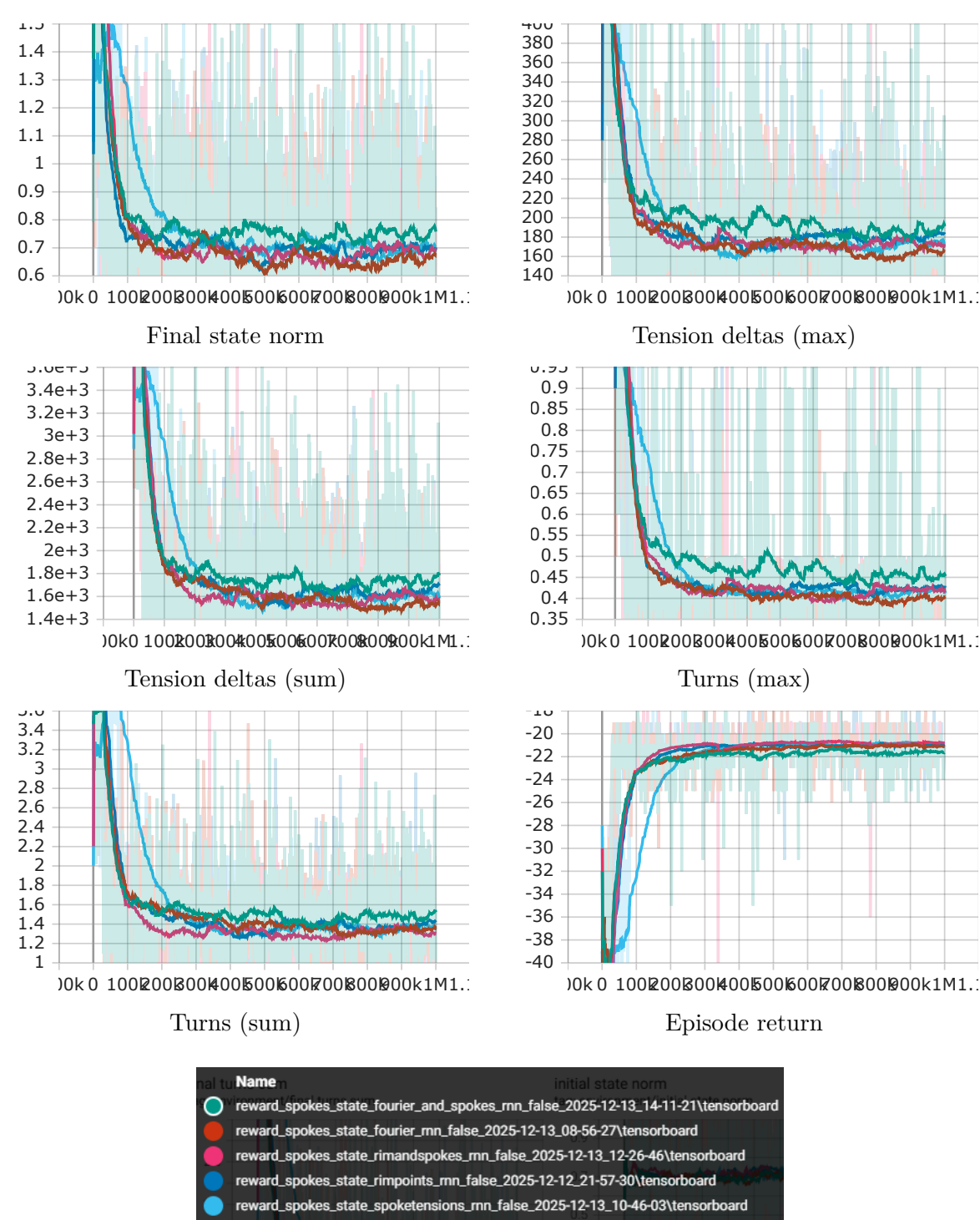
Legend

Overview of DQN training metrics.



Legend

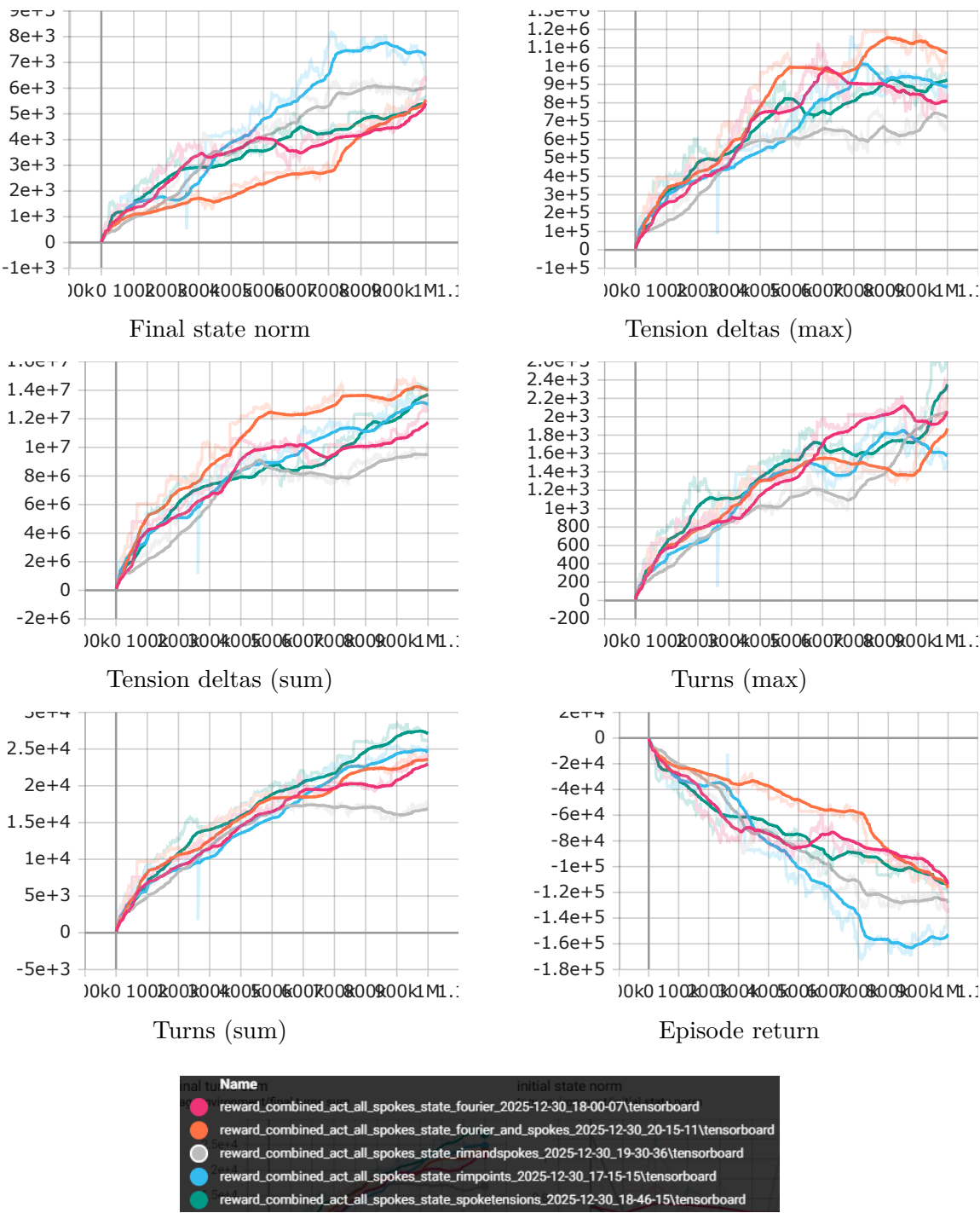
Overview of DQN training metrics.



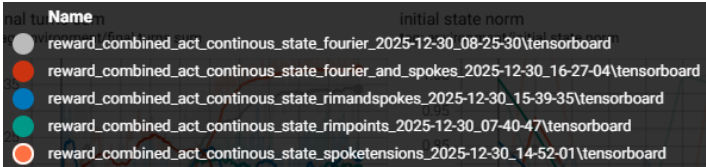
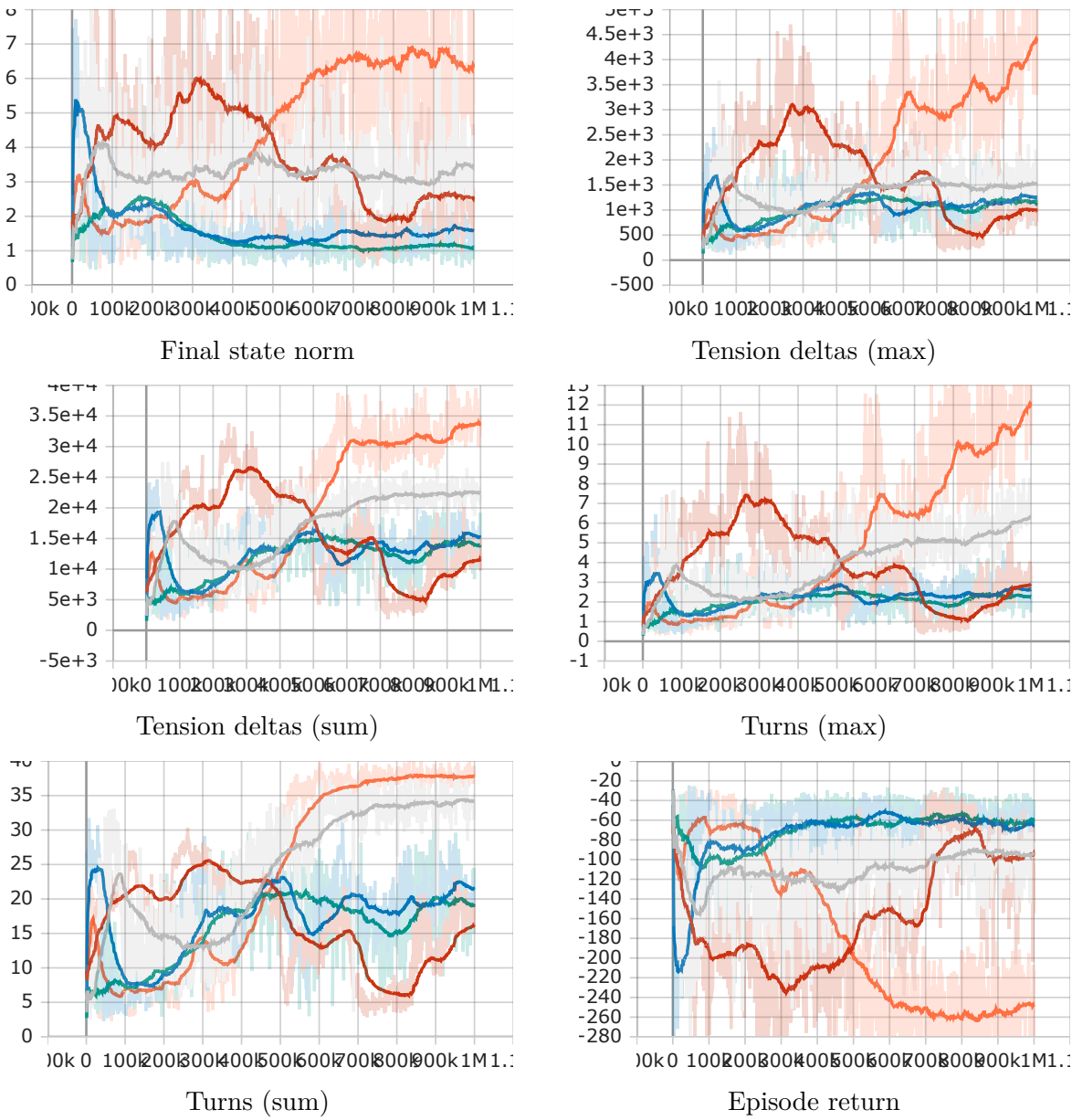
Legend

Overview of DQN training metrics.

PPO Figures

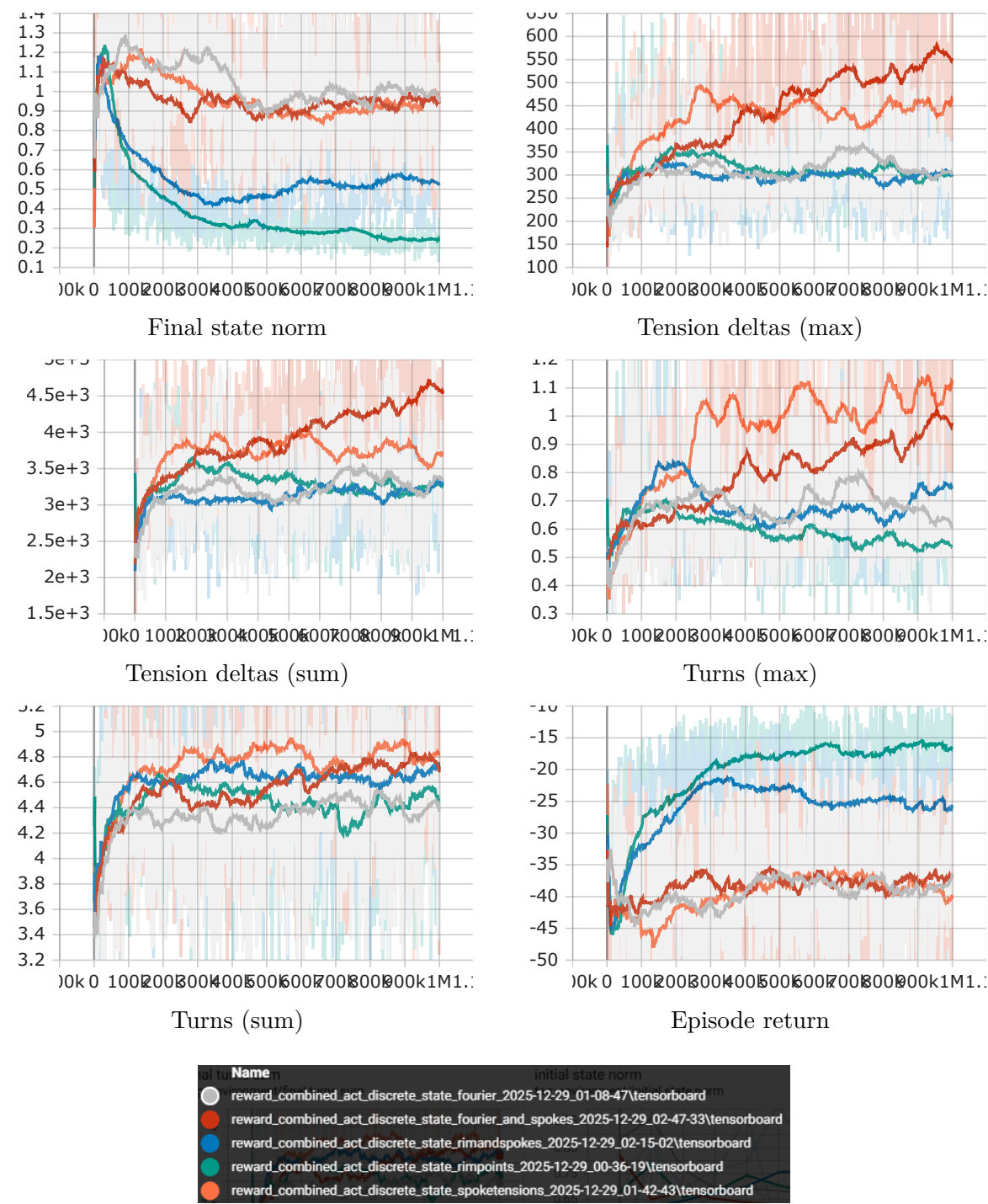


Overview of PPO training metrics (combined_all_spokes).



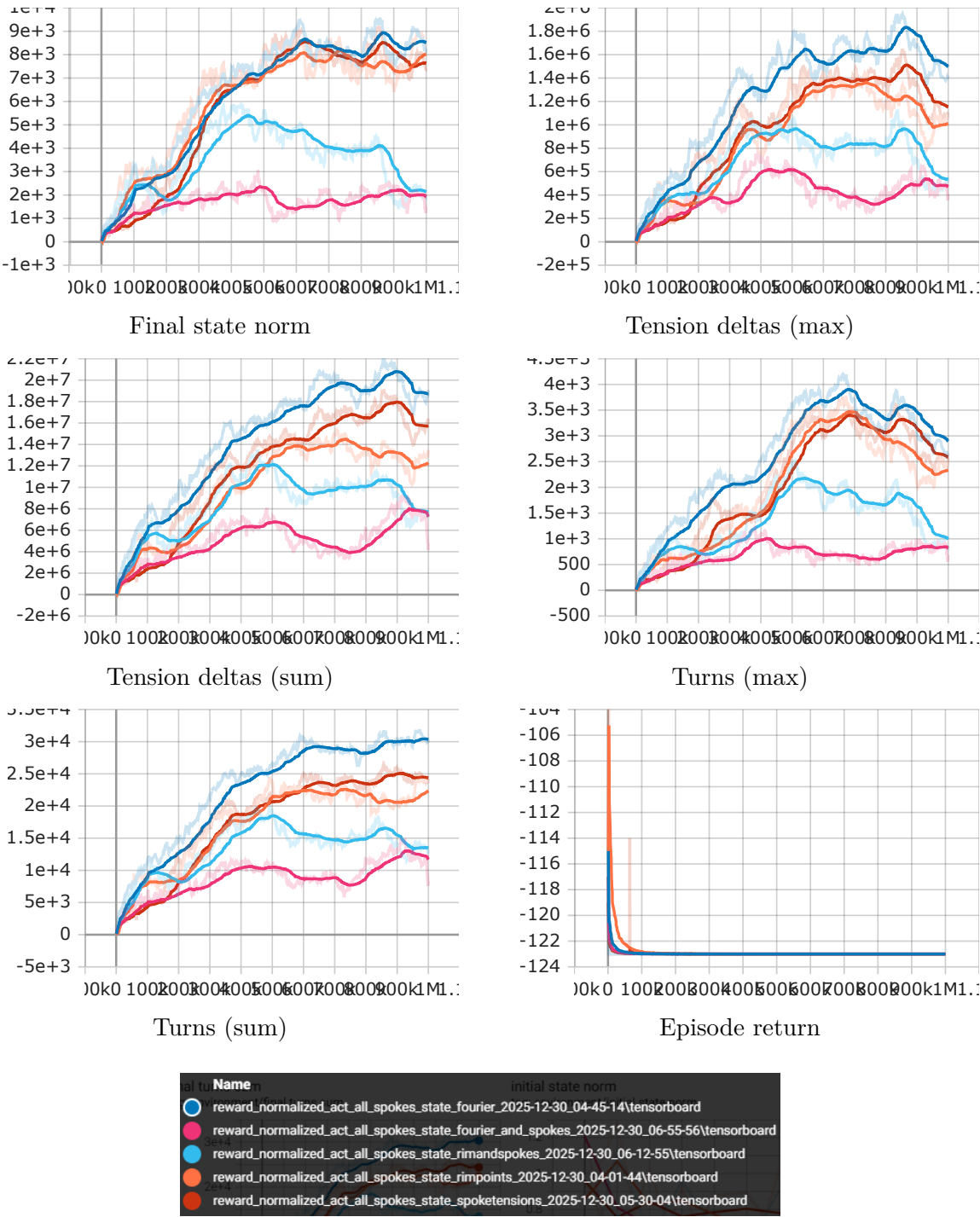
Legend

Overview of PPO training metrics (combined_cont).

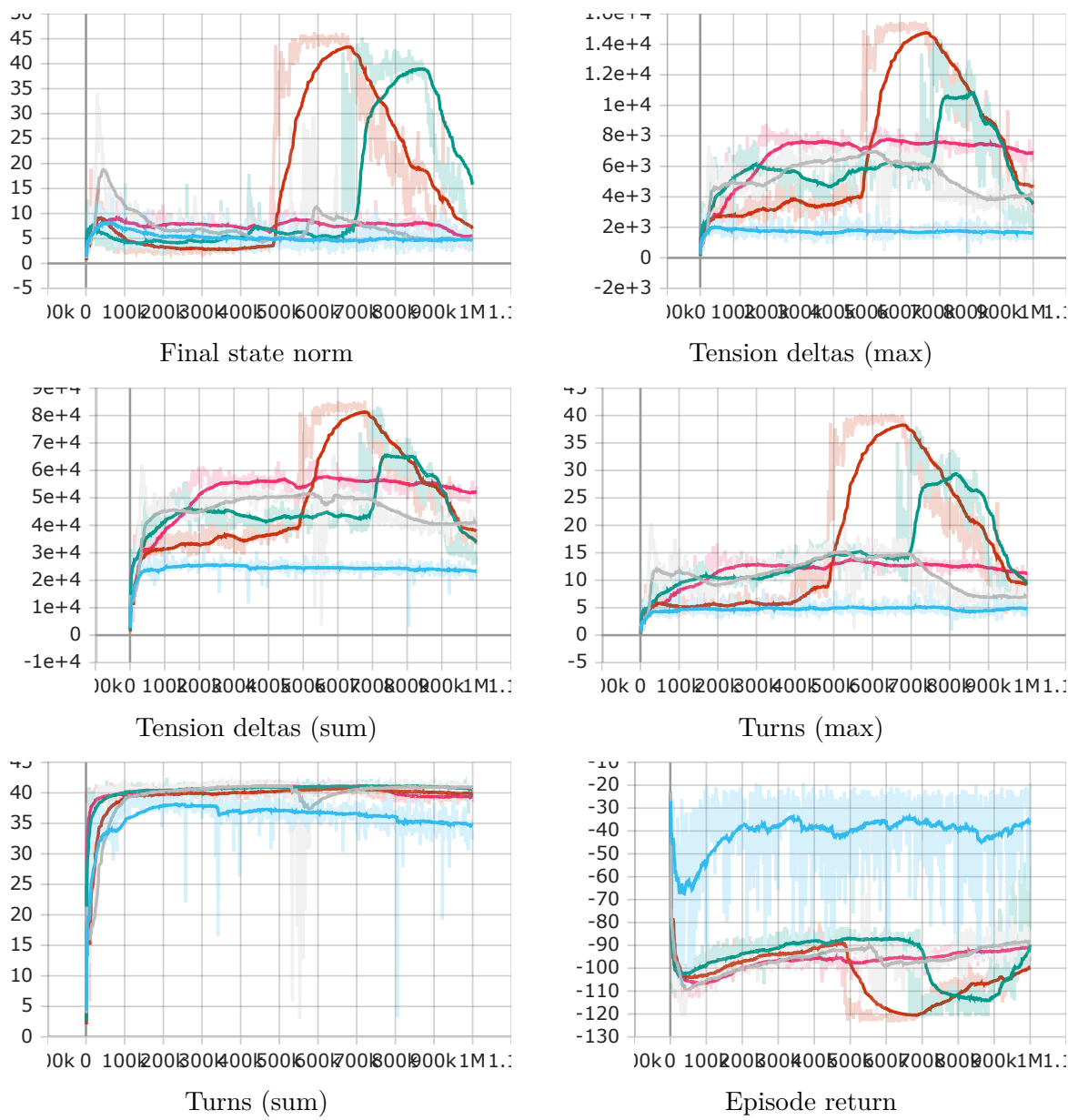


Legend

Overview of PPO training metrics (combined_discrete).



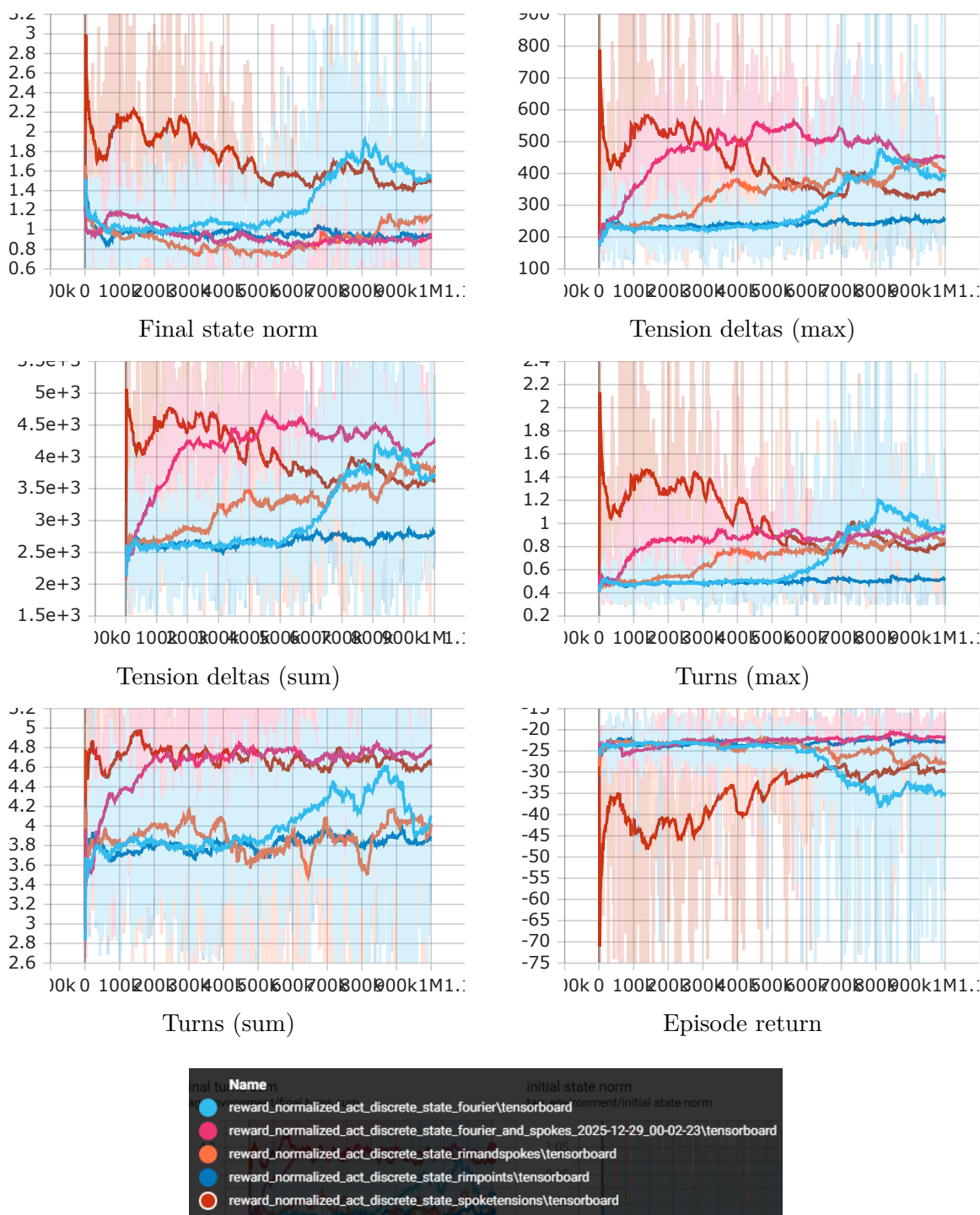
Overview of PPO training metrics (norm_all_spokes).



Name	initial state norm
reward_normalized_act_continuous_state_fourier_2025-12-30_01-00-46\tensorboard	
reward_normalized_act_continuous_state_fourier_and_spokes_2025-12-30_03-15-55\tensorboard	
reward_normalized_act_continuous_state_rimandspokes_2025-12-30_02-31-25\tensorboard	
reward_normalized_act_continuous_state_rimpoints_2025-12-30_00-15-52\tensorboard	
reward_normalized_act_continuous_state_spoketensions_2025-12-30_01-47-01\tensorboard	

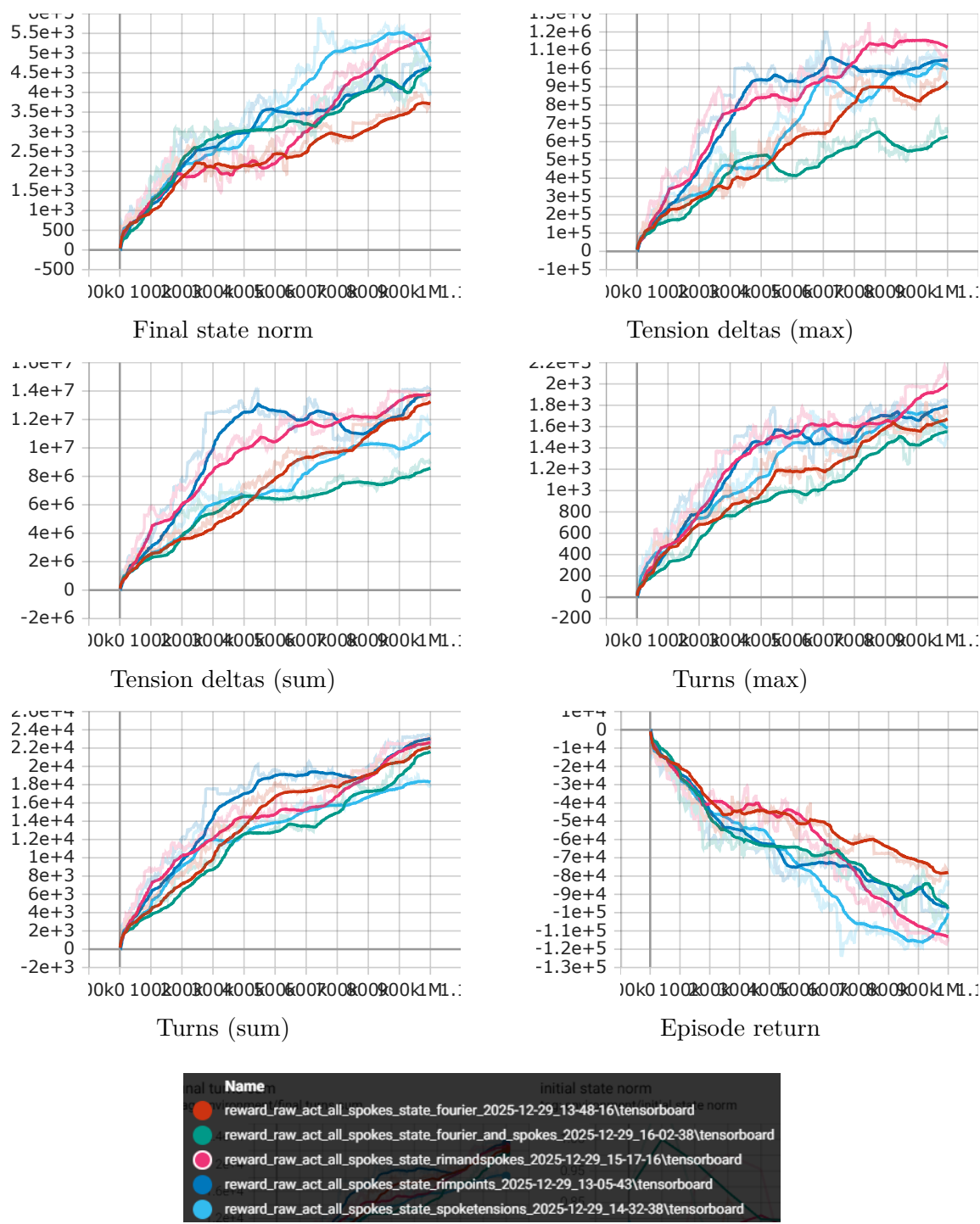
Legend

Overview of PPO training metrics (norm_cont).



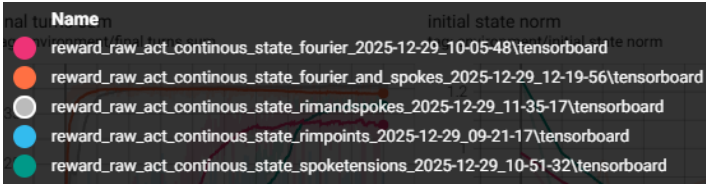
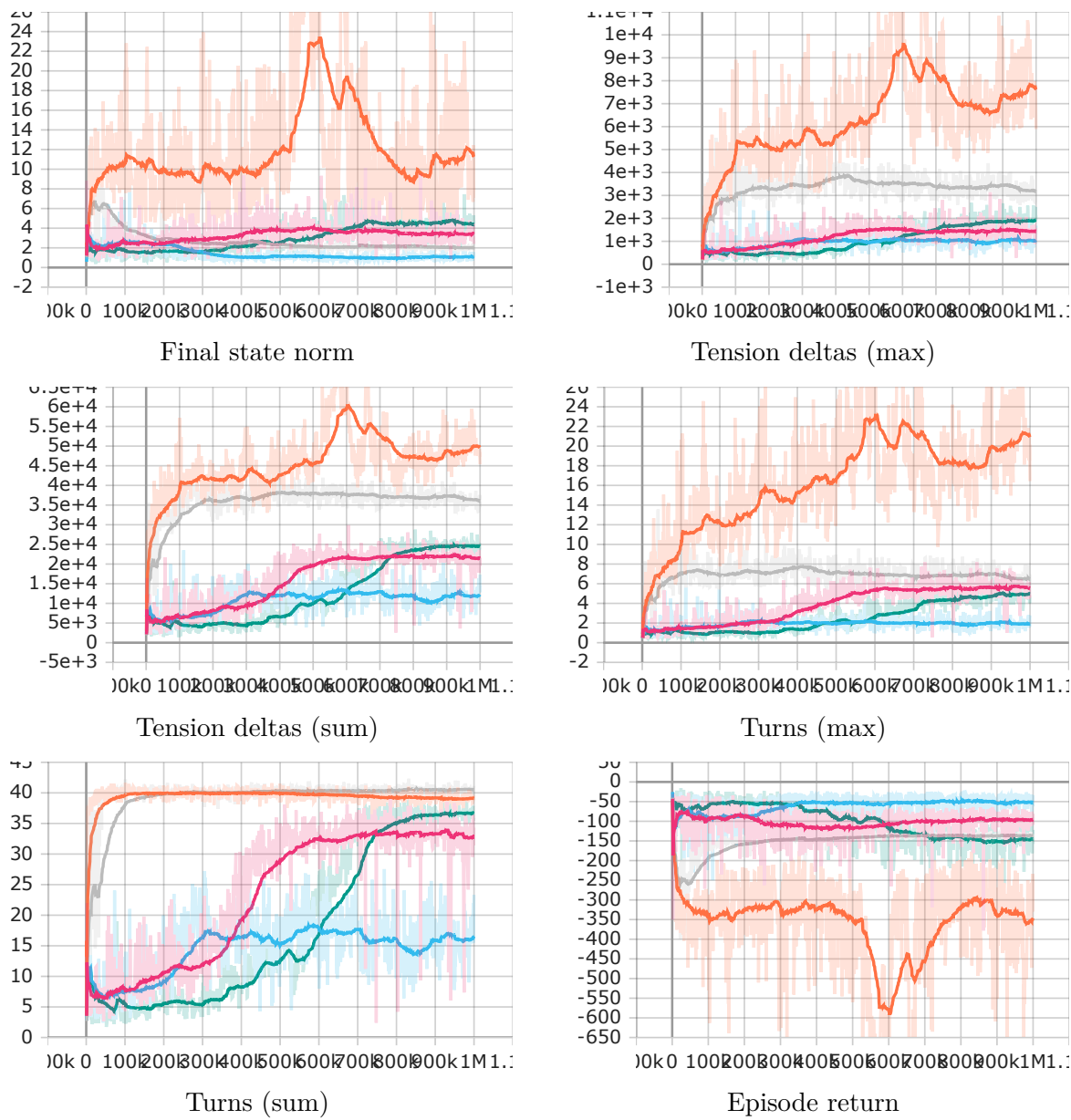
Legend

Overview of PPO training metrics (norm_discrete).



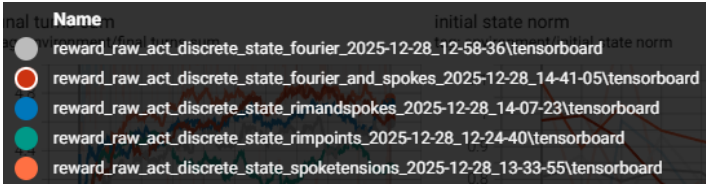
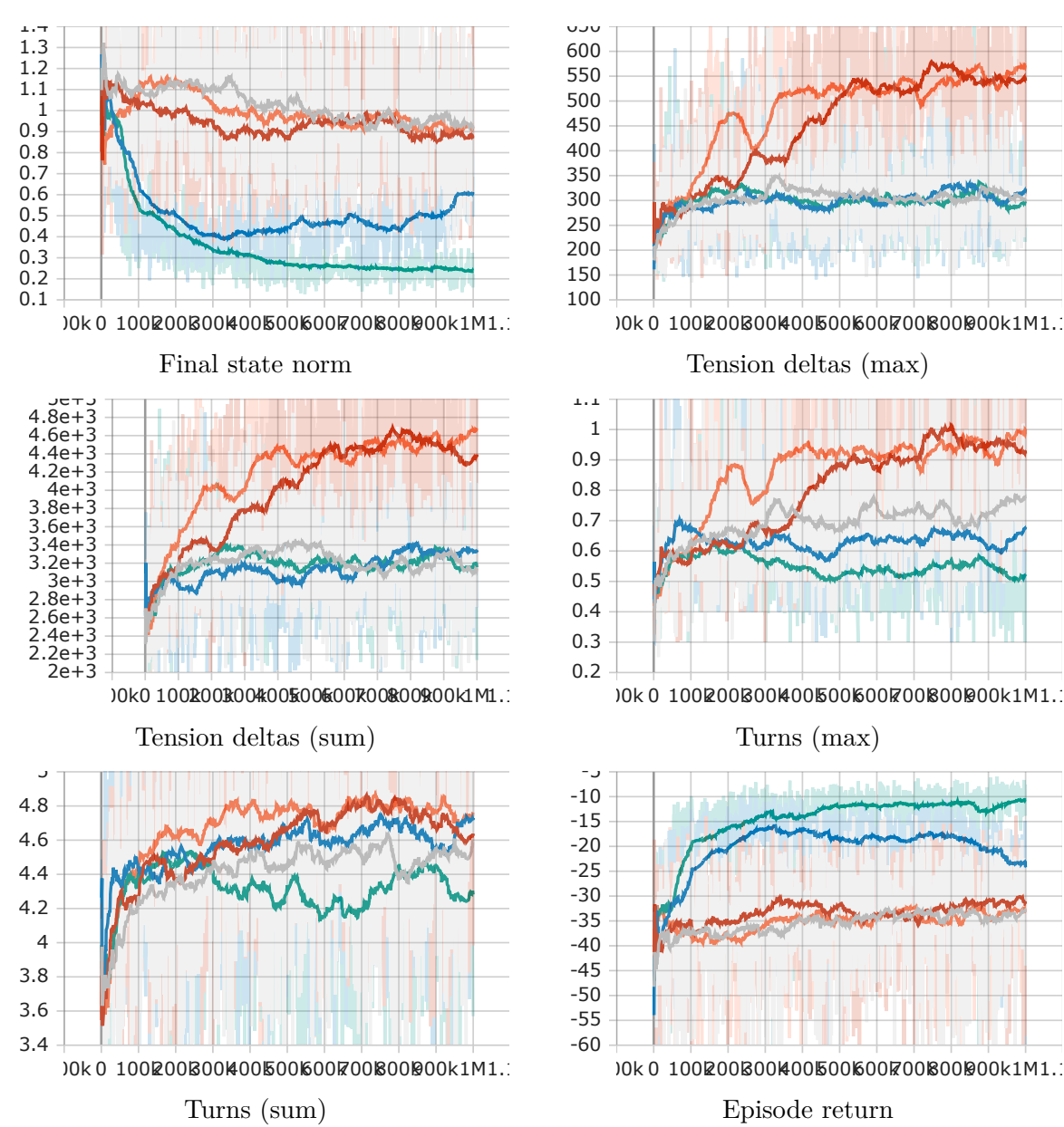
Legend

Overview of PPO training metrics (raw_all_spokes).

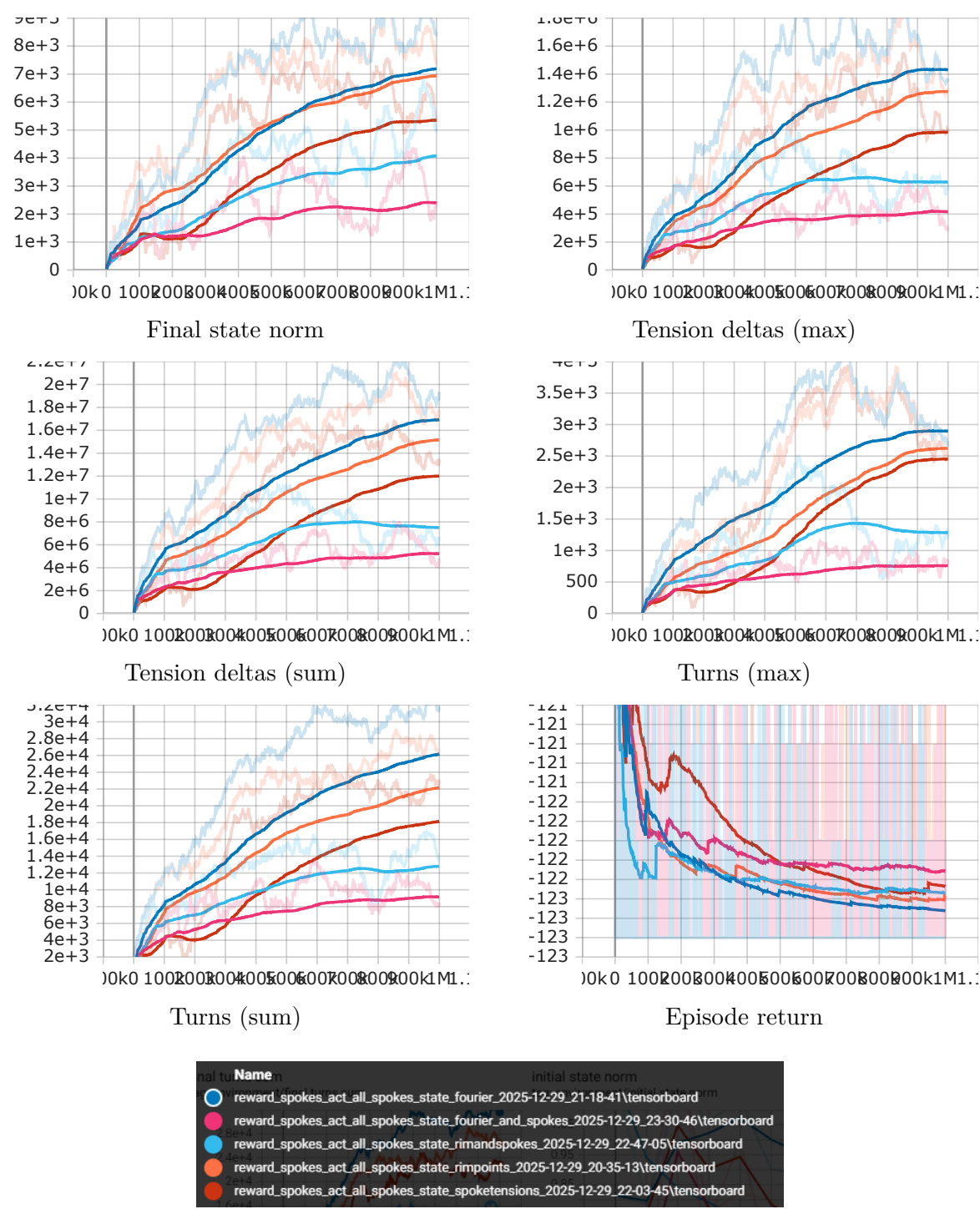


Legend

Overview of PPO training metrics (raw_cont).

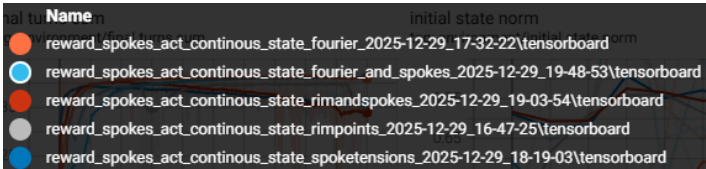
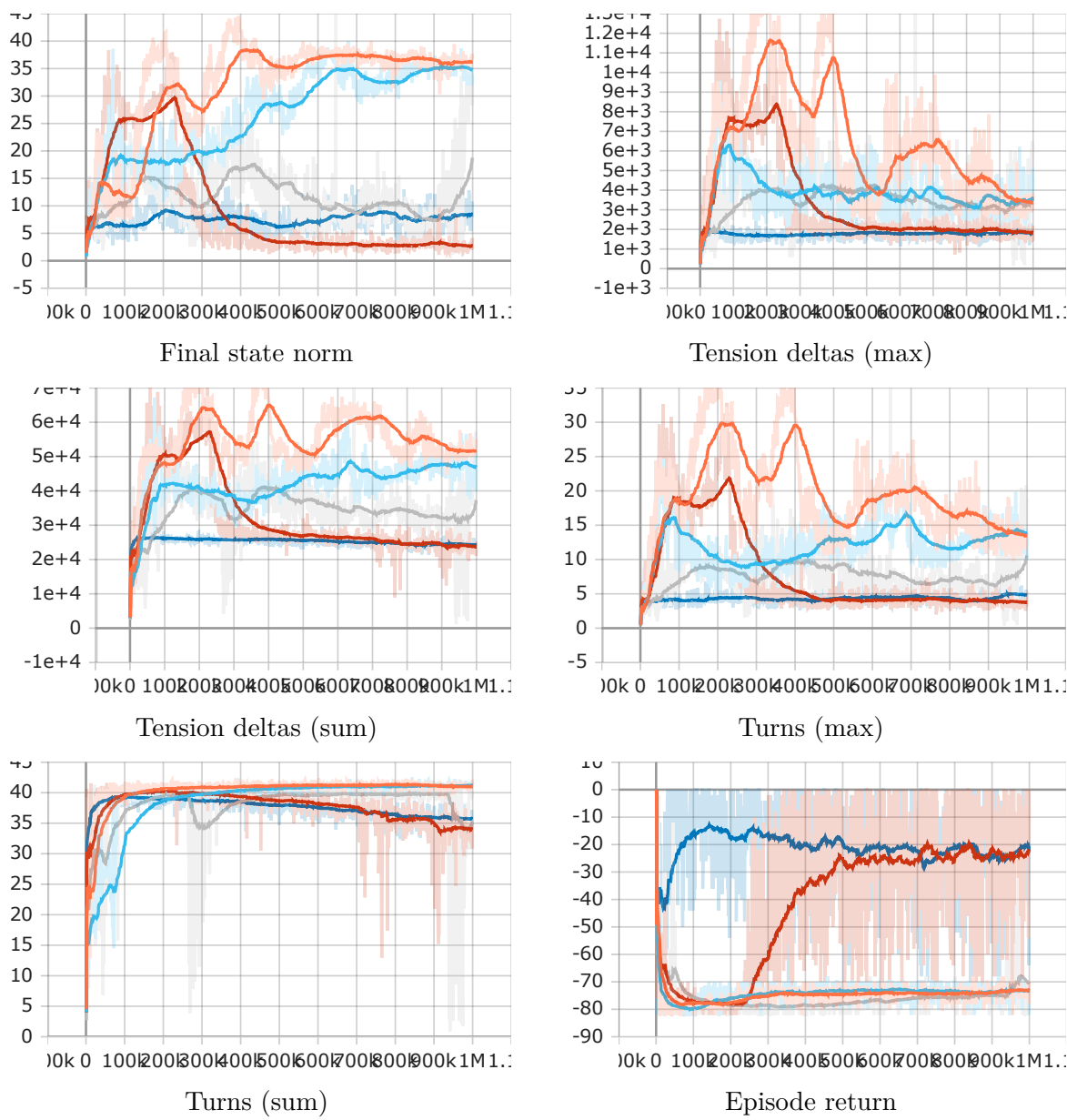


Overview of PPO training metrics (raw_discrete).



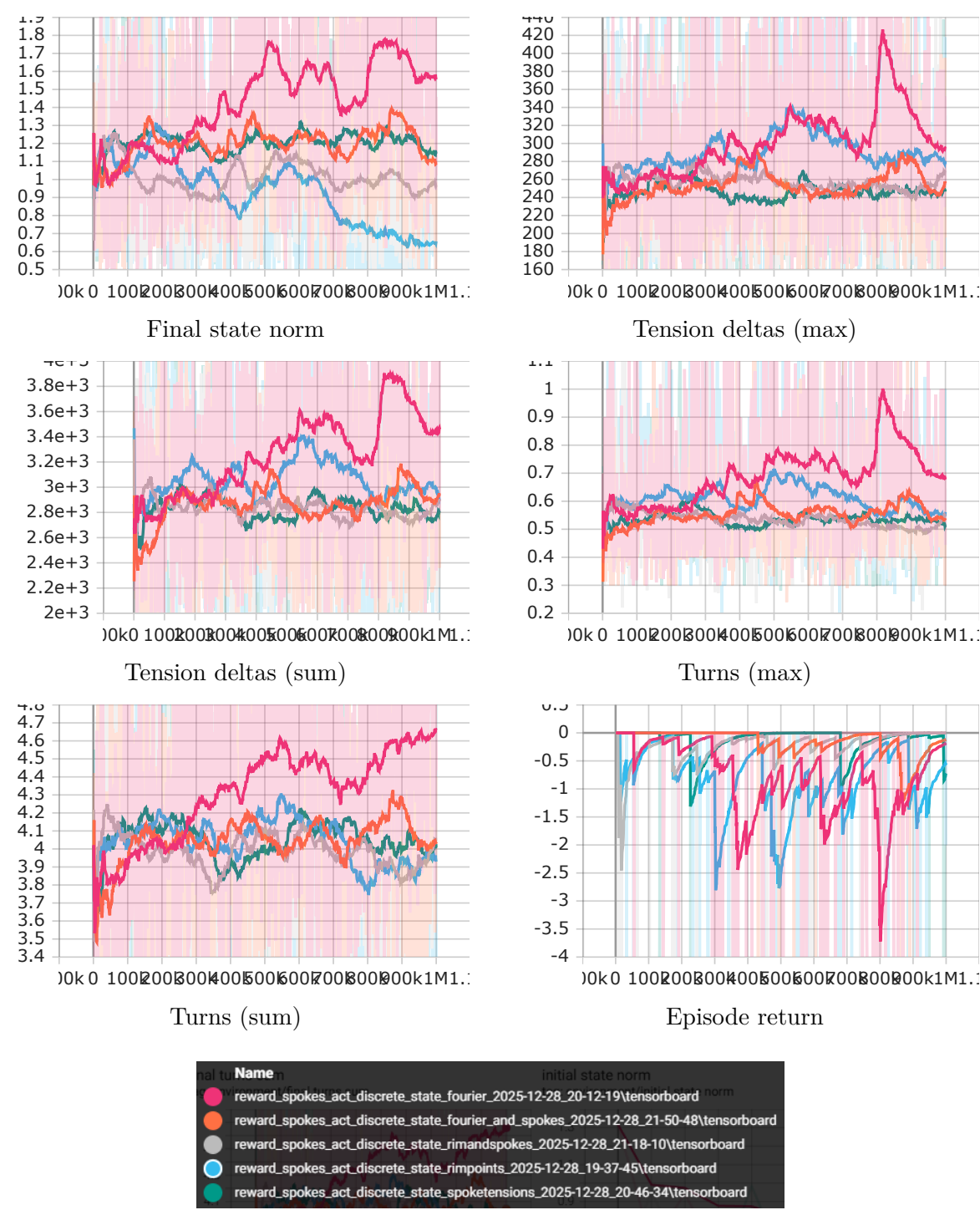
Legend

Overview of PPO training metrics (spokes_all_spokes).



Legend

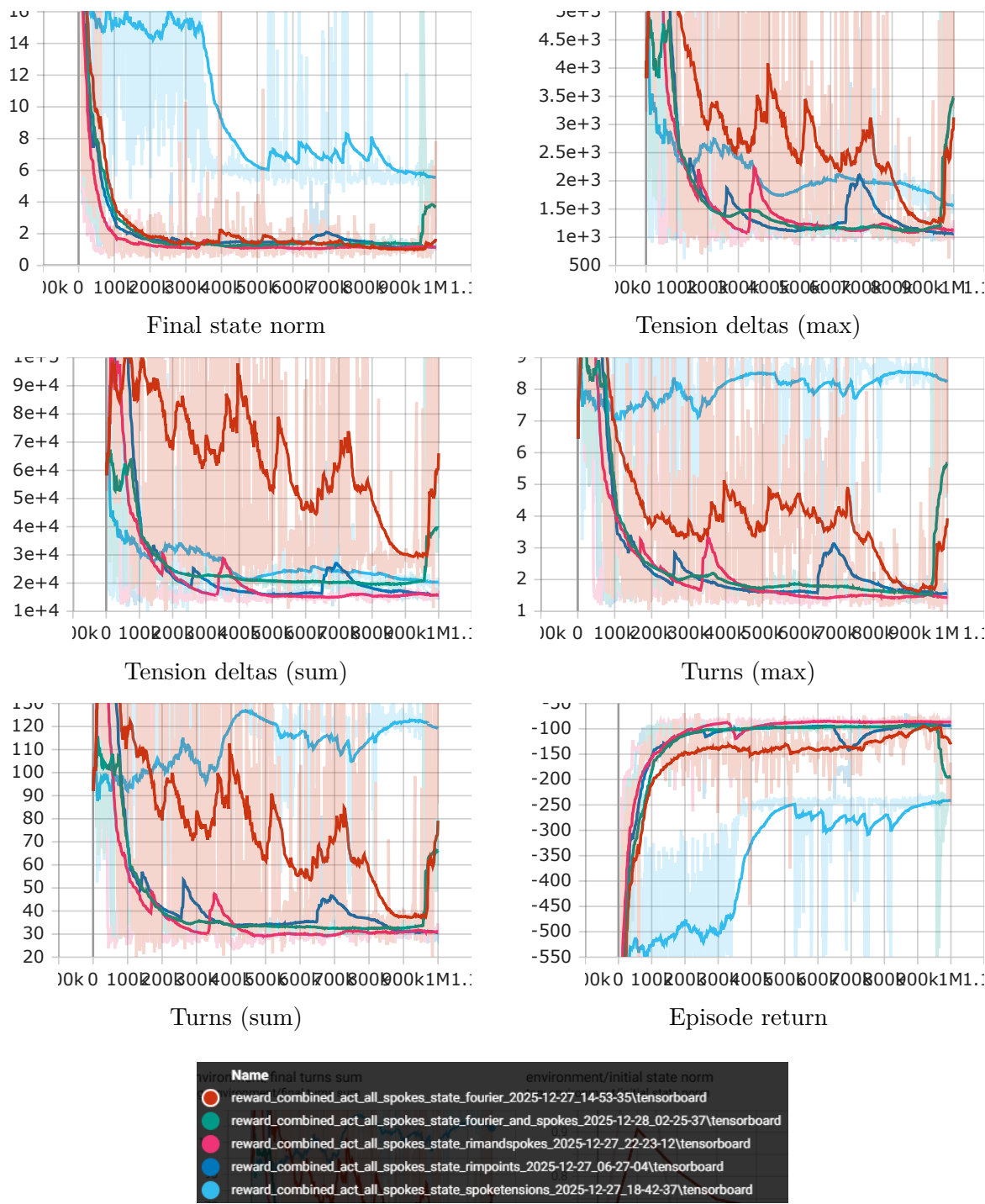
Overview of PPO training metrics (spokes_cont).



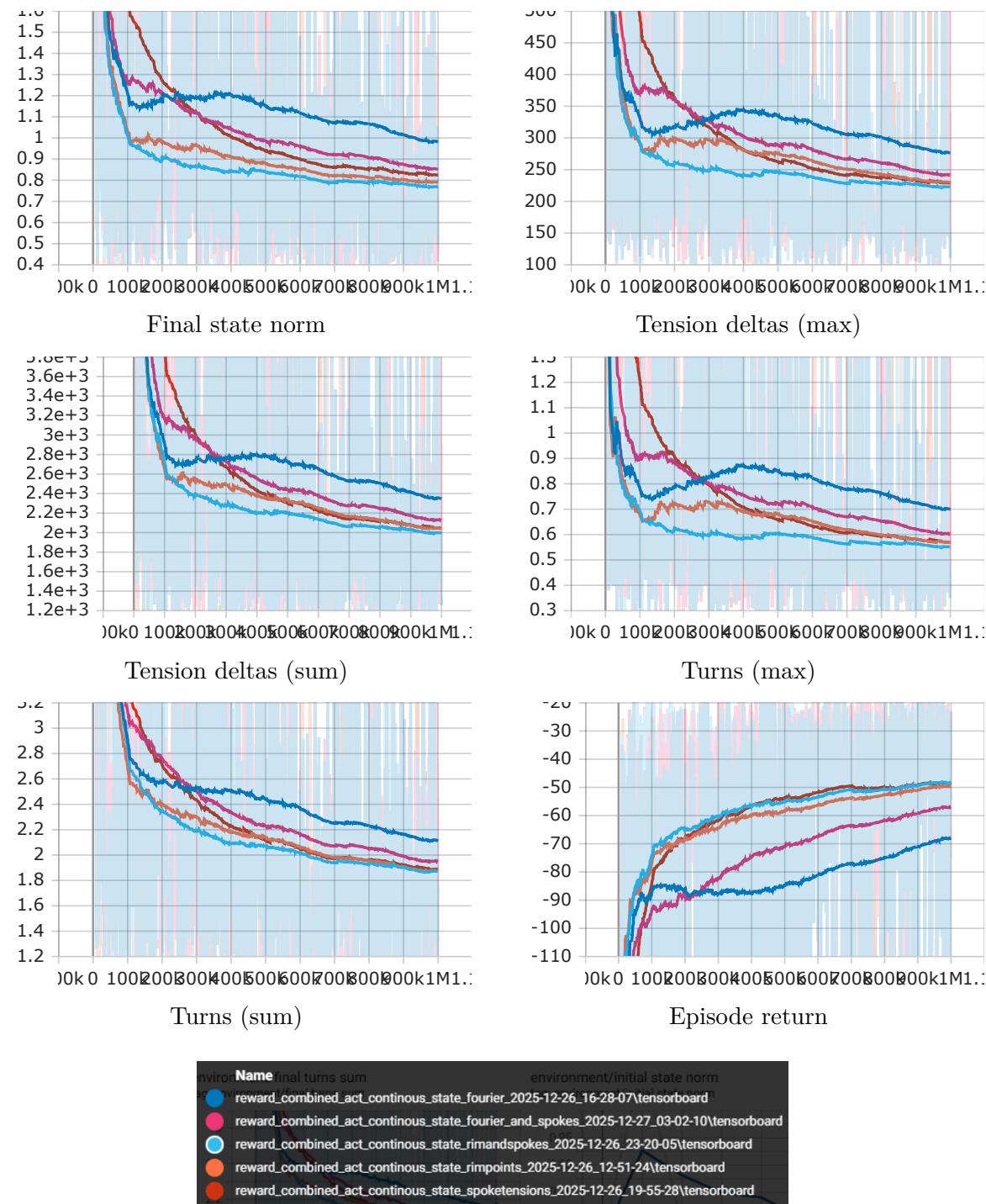
Legend

Overview of PPO training metrics (spokes_discrete).

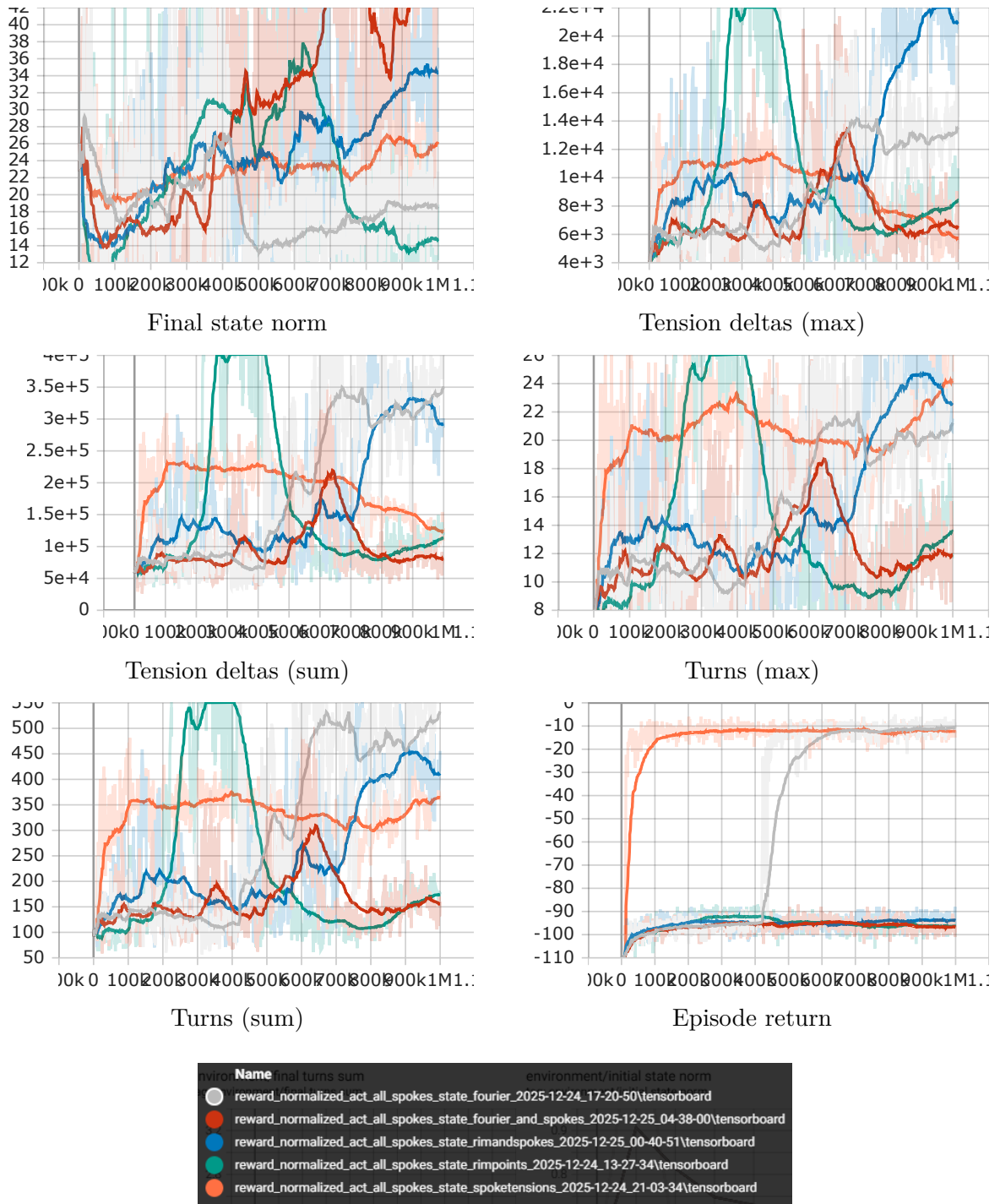
TD-MPC Figures



Overview of TD-MPC training metrics (combined_all_spokes).

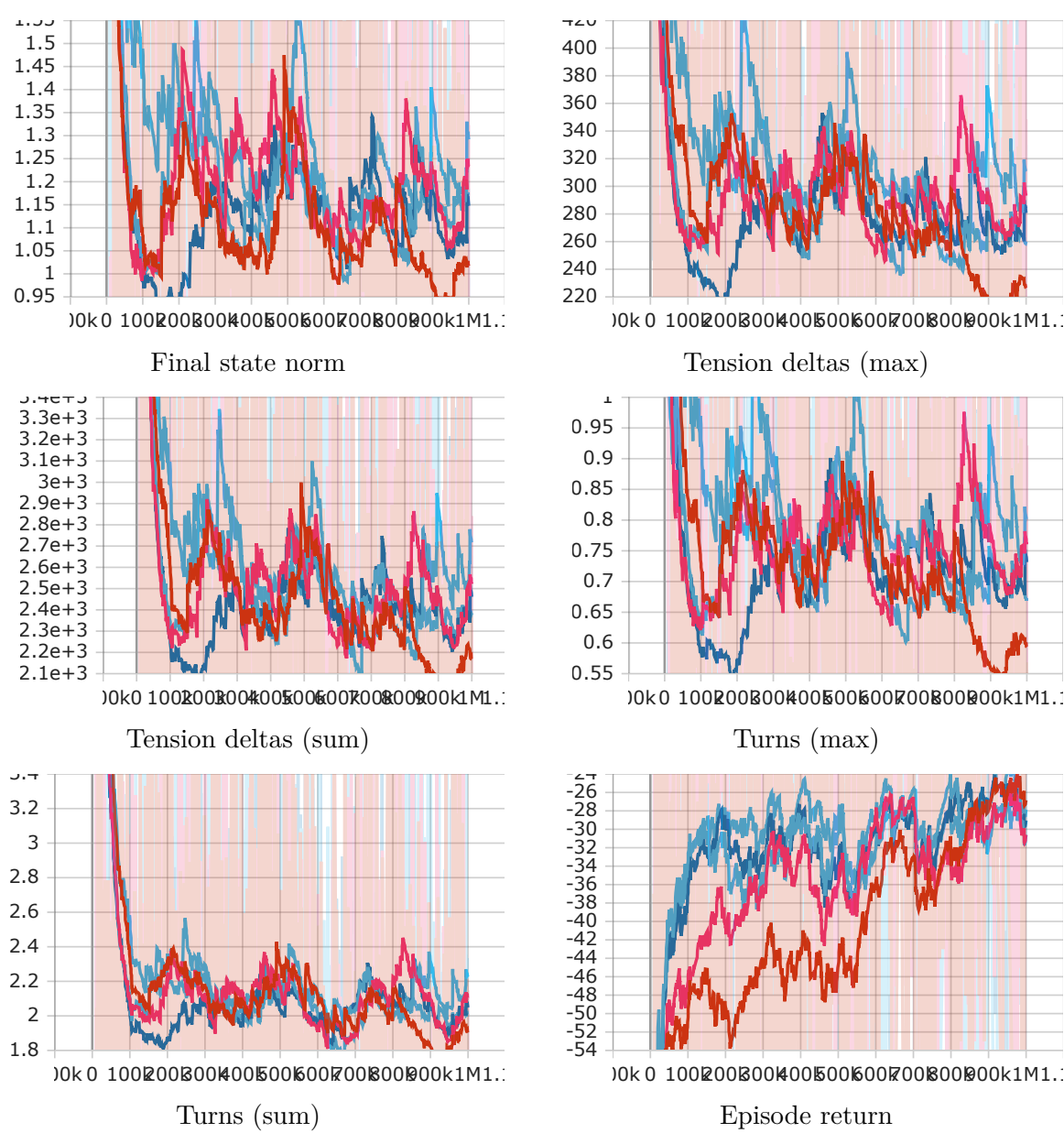


Overview of TD-MPC training metrics (combined_cont).



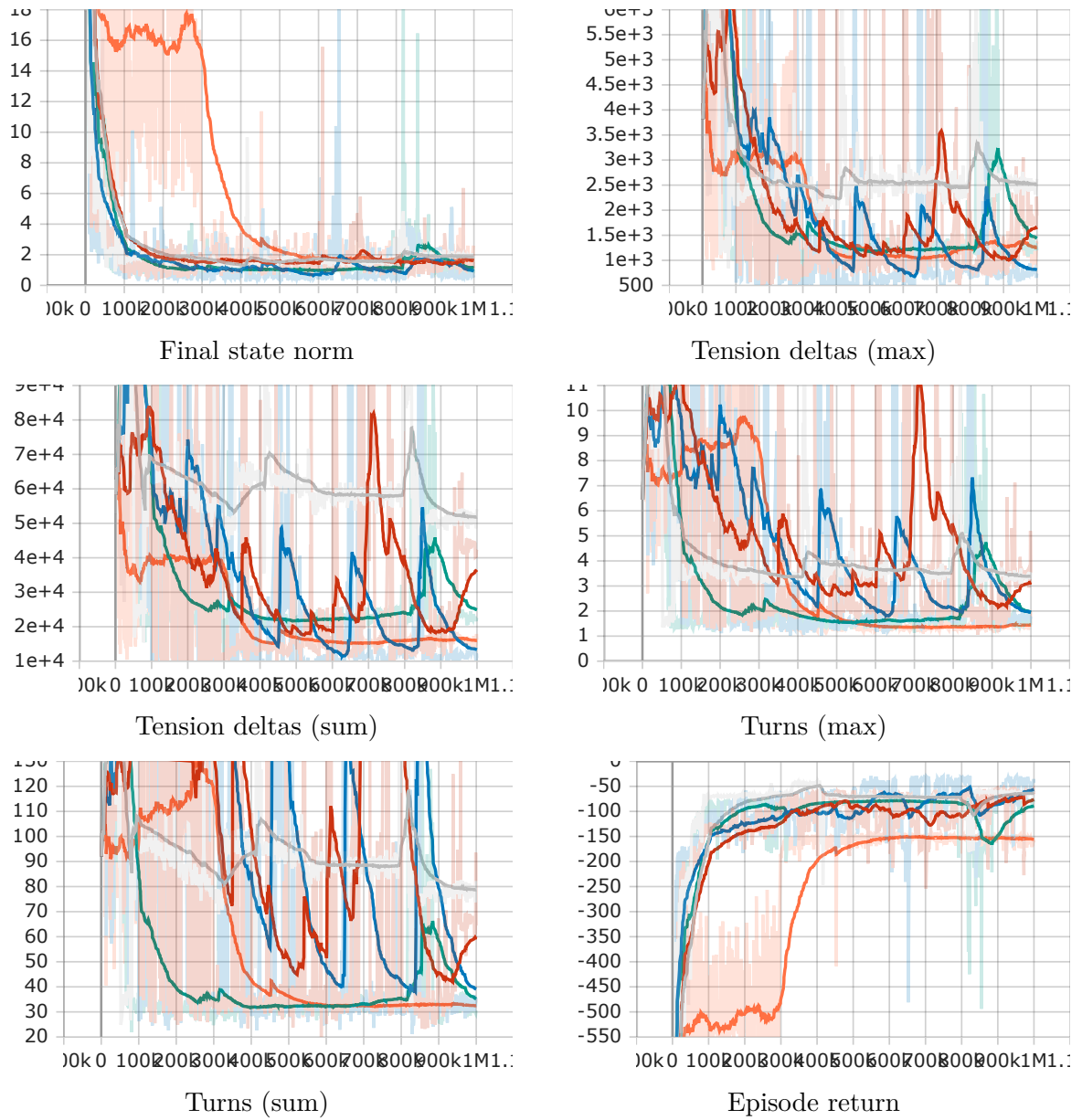
Legend

Overview of TD-MPC training metrics (norm_all_spokes).



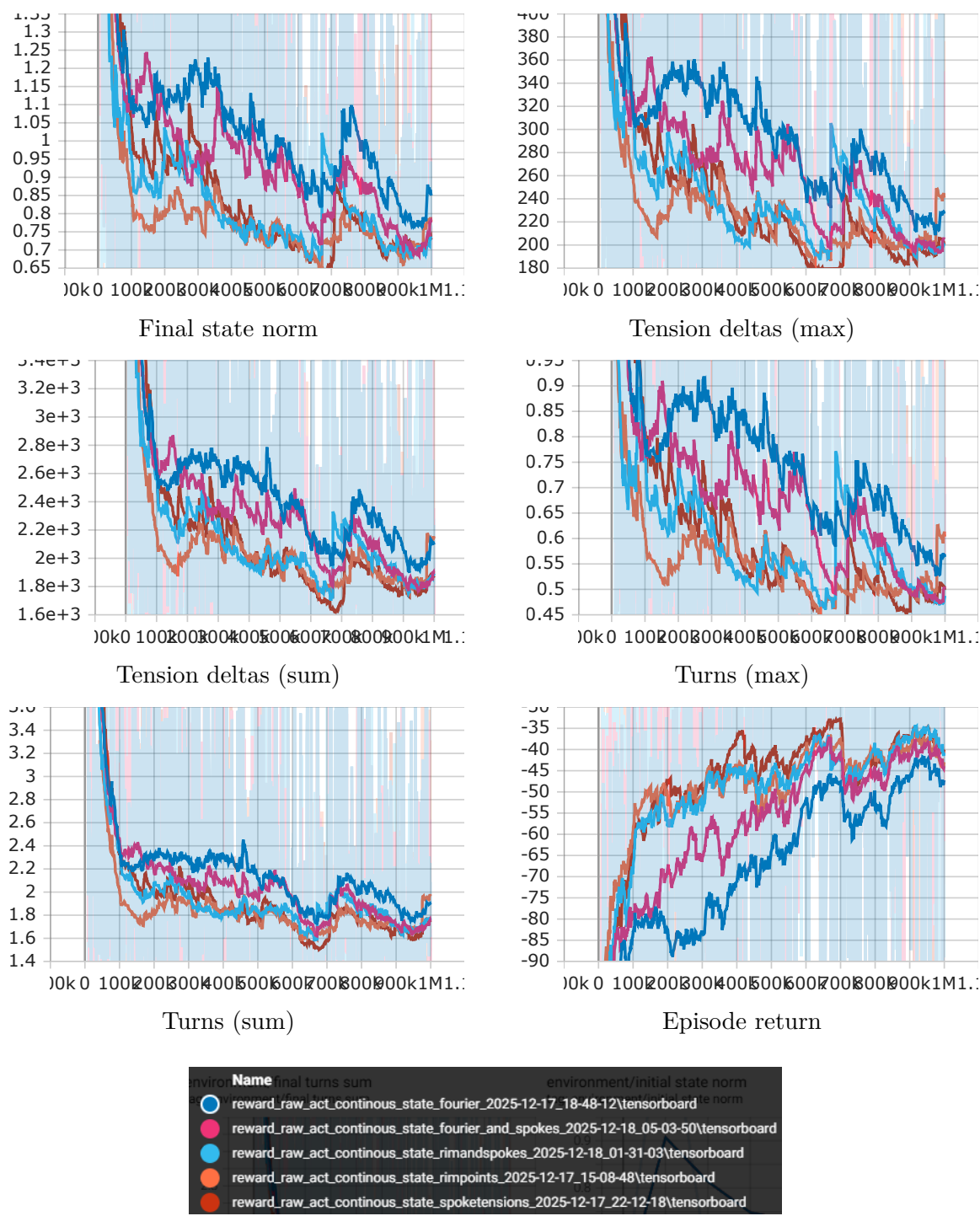
Legend

Overview of TD-MPC training metrics (norm_cont).



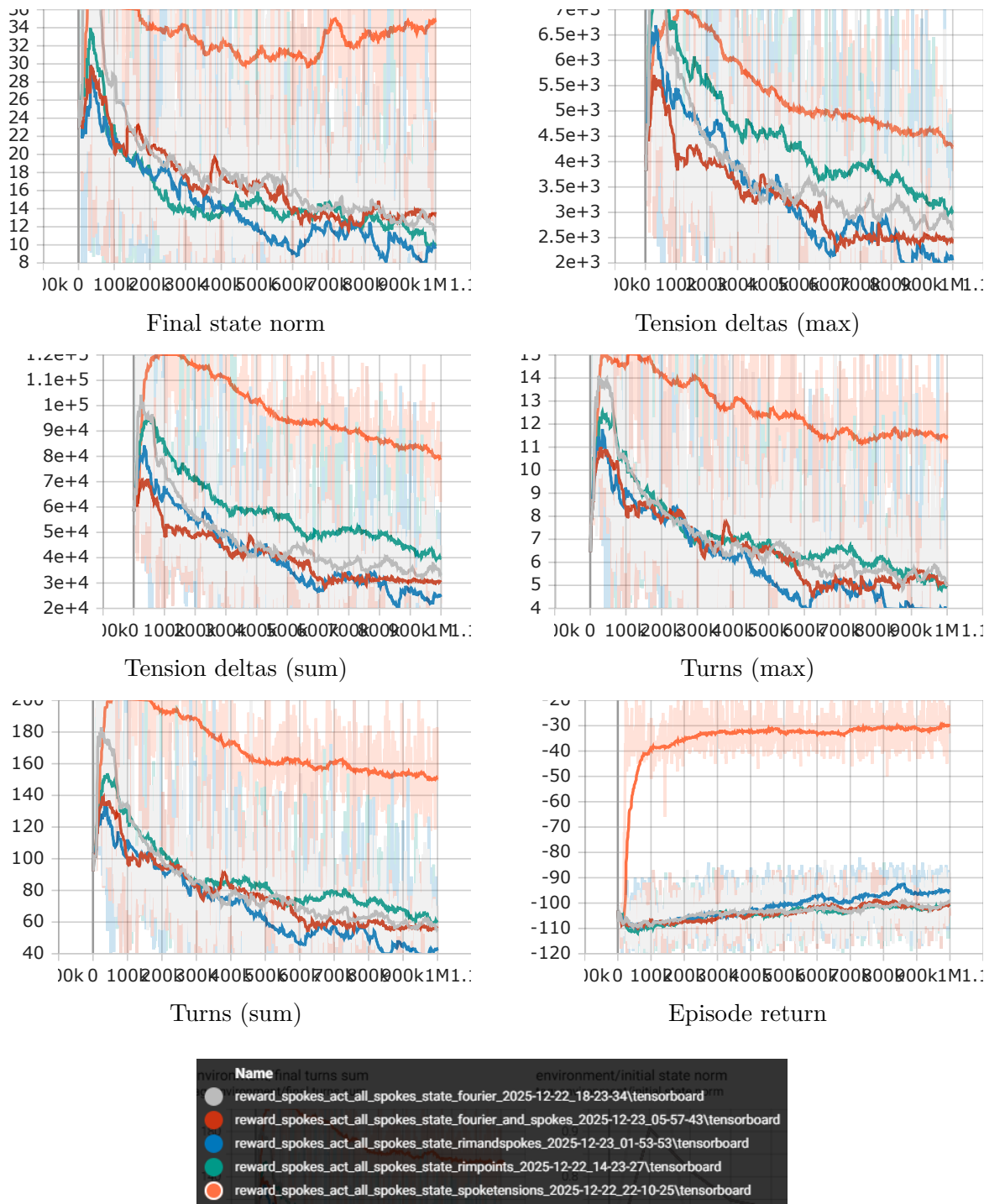
Legend

Overview of TD-MPC training metrics (raw_all_spokes).



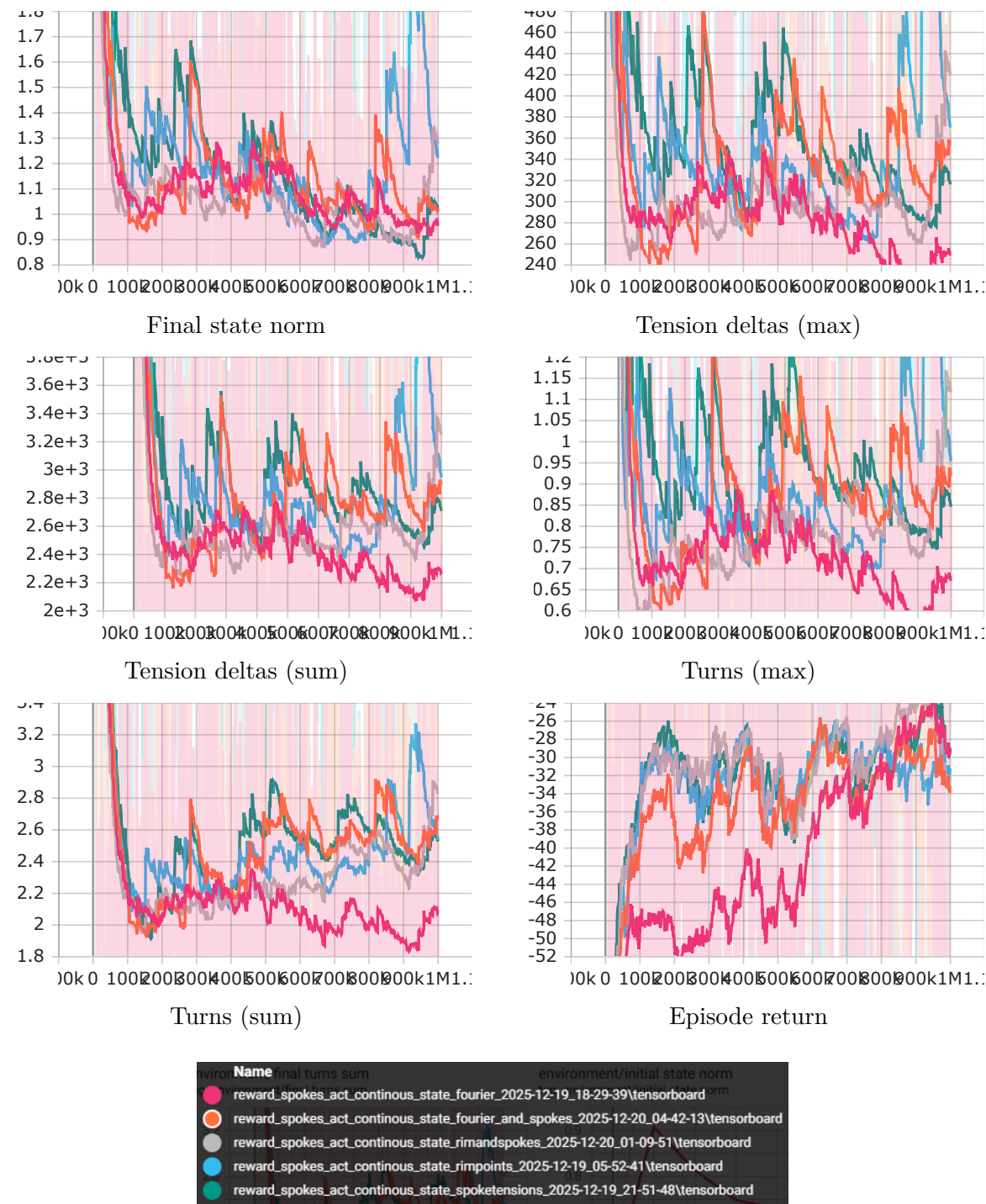
Legend

Overview of TD-MPC training metrics (raw_cont).



Legend

Overview of TD-MPC training metrics (spokes_all_spokes).



Legend

Overview of TD-MPC training metrics (spokes_cont).

.1 Code Listings

Complete code can be found under: <https://github.com/ToniRoz/bike>